Eduardo Ogasawara, Rebecca Salles,
Fabio Porto, Esther Pacitti

Event Detection in Time Series

February 22, 2024

Springer Nature

dedication

Foreword

The foreword covers introductory remarks preceding the text of a book that are written by a *person other than the author or editor* of the book. If applicable, the foreword precedes the preface which is written by the author or editor of the book.

Place, month year *Firstname Surname*

Preface

The book is designed for readers with an interest or background in data analysis, particularly those focused on time series analysis. This includes but is not limited to data scientists, statisticians, researchers, and professionals in fields where time series data is prevalent, such as finance, meteorology, economics, and engineering. It provides a detailed introduction to event detection in time series, discussing types of time series (univariate and multivariate). It explores various event types, including anomalies, change points, and motifs, and examines their detection methods. Event detection strategies are categorized and presented considering different scenarios such as offline, online, and prediction. The importance of accurate evaluation of these methods is emphasized, considering metrics such as precision, recall, F1-score, and AUC-ROC, alongside a discussion on the computational complexity of these methods. The lecture introduces more in-depth topics, including time series analysis, different types of event detection, motif discovery, and evaluation metrics, culminating in an introduction to the Harbinger framework and a summary of challenges in the research area.

Place(s),
month year

Firstname Surname
Firstname Surname

Acknowledgements

The authors thank Prof. Dayse Pastori, Diego Carvalho, Eduardo Bezerra, Joel Santos, and Rafaelli Coutinho for their time reviewing the book.

The authors thank CNPq, CAPES, and FAPERJ for partially sponsoring this research.

Contents

1 Introduction 1
1.1 Overview 1
1.2 Types of events 3
1.2.1 Anomalies 5
1.2.2 Noise anomalies 5
1.2.3 Volatility anomalies 6
1.2.4 Change points 7
1.2.5 Motifs 8
1.3 Data structure 10
1.3.1 Granularity 10
1.3.2 Dimensionality 11
1.4 Detection strategies 12
1.5 Detection scenarios 13
1.6 Detection evaluation 14
1.6.1 Accurateness 14
1.6.2 Expensiveness 15
1.7 Book structure 15

2 Time series analysis 17
2.1 Time series components 17
2.2 Stationarity 18
2.3 Data preprocessing 24
2.3.1 Temporal aggregation 25
2.3.2 Trend component 26
2.3.3 Variance stabilization 27
2.3.4 Detrending and differencing 28
2.3.5 Decomposition 30
2.3.6 Sliding windows 36
2.3.7 Data normalization 36
2.3.8 Data splitting 39
2.4 Time series prediction 40

2.4.1 Statistical models 44
2.4.2 Machine learning models 46

3 Anomaly detection 51
3.1 Basics of anomalies 51
3.1.1 Point and sequence anomalies 52
3.1.2 Contextual and out-of-context anomalies 52
3.1.3 Univariate and multivariate anomaly detection 53
3.1.4 Labeled and unlabeled time series 54
3.1.5 Supervised, semi-supervised, and unsupervised 55
3.2 Methods for anomaly detection 55
3.2.1 Regression 56
3.2.2 Classification 59
3.2.3 Clustering 60
3.2.4 Statistical 63
3.2.5 Spectral 64
3.2.6 Information theory 66
3.3 Advanced topics in anomaly detection 67
3.3.1 Detection methods for volatility anomalies 68
3.3.2 Detection methods for sequence anomalies 68
3.3.3 Detection methods for multidimensional time series 69
3.3.4 Extreme Values 70

4 Change points and concept drift detection 73
4.1 Basics of change points 73
4.2 Change point methods 75
4.2.1 At most one change (AMOC) 76
4.2.2 Pruned exact linear time (PELT) 76
4.2.3 Chow test 78
4.2.4 Generalized fluctuation test 79
4.2.5 Seminal change point 80
4.2.6 Change finder 81
4.3 Concept drift 82
4.4 Concept drift detectors 86
4.4.1 DDM 86
4.4.2 HDDM 87
4.4.3 ADWIN 88
4.4.4 ECDD 88
4.4.5 CUSUM 89
4.4.6 Page-Hinkley 89
4.5 Advanced topics in change points 90

5 Motif discovery . . . 93
5.1 Basics of motifs . . . 93
5.2 Time series indexing . . . 94
5.2.1 Normalization . . . 95
5.2.2 Symbolic aggregation approximation . . . 95
5.2.3 Piecewise aggregate approximation . . . 95
5.3 Similarity measures . . . 96
5.3.1 Euclidean distance . . . 97
5.3.2 Hamming distance . . . 97
5.3.3 Dynamic time warp . . . 97
5.4 Motif discovery methods . . . 98
5.4.1 Brute force discovery . . . 99
5.4.2 Indexed based discovery . . . 99
5.4.3 Random projection . . . 100
5.4.4 Early abandon distance . . . 101
5.4.5 Matrix profile . . . 101
5.5 Advanced topics in motif discovery . . . 103
5.5.1 Multidimensional motif discovery . . . 103
5.5.2 Variable motif size . . . 104
5.5.3 Ranking motifs and occurrences . . . 104
5.5.4 Big Data . . . 105
5.5.5 Novel data structures . . . 105

6 Evaluation metrics . . . 107
6.1 Basic metrics . . . 107
6.2 Time tolerance . . . 111
6.3 SoftED . . . 112
6.4 Other initiatives for event detection metrics . . . 115

7 Online event detection . . . 117
7.1 Characterizing the online versus offline scenario . . . 117
7.2 Online detection . . . 118
7.3 Online prediction . . . 121
7.3.1 Online prediction using time series classification . . . 121
7.3.2 Online prediction using time series prediction . . . 122
7.4 Online detection evaluation . . . 123
7.5 Open challenges in online event detection . . . 124

8 Epilogue . . . 127
8.1 Summary of research area . . . 127
8.2 Summary of challenges . . . 130
8.3 Harbinger . . . 132

References . . . 133

Index . . . 157

Acronyms

R^2	R-squared.
ACF	Autocorrelation function.
ADFT	Augmented Dickey-Fuller test.
ADWIN	Adaptive windowing.
AIC	Akaike information criterion.
AMOC	At most one change.
AN	Adaptive normalization.
AR(1)	First order autoregressive model.
ARCH	Autoregressive conditional heteroscedasticity model.
ARIMA	Autoregressive integrated moving average.
ARMA	Autoregressive moving average.
AR	Autoregressive.
AUC	Area under the curve.
BCT	Box-Cox transform.
BIC	Bayesian information criterion.
BPT	Breusch-Pagan test.
CF	Change finder.
CUSUM	Cumulative sum control chart.
CWT	Continuous wavelet transform.
Chow test	Chow test (F test).
Conv1D	Convolutional neural network for time series.
DBScan	Density-based spatial clustering of applications with noise.
DDM	Drift detection method.
DFT	Discrete Fourier transform.
DIF	Simple differencing.
DTW	Dynamic time warping.
DTree	Decision trees.
DT	Detrending.

DWT	Discrete wavelet transform.
ECDD	EWMA for concept drift detection.
EDist	Euclidean distance.
EE	Extreme Event.
ELM	Extreme learning machines network.
EMD	Empirical mode decomposition.
EMMA	Enumeration of motifs through matrix approximation.
ETS	Exponential smoothing state space model.
EV	Extreme Values.
EWMA	Exponentially weighted moving average.
FBIAD	Forward and backward inertial anomaly detector.
FFT	Fast Fourier transform.
FHDDM	Fast Hoeffding drift detection method.
FT	Fourier transform.
GARCH	Generalized autoregressive conditional heteroscedasticity model.
GEV	Generalized extreme value.
GFT	Generalized fluctuation test.
GRU	Gated recurrent unit.
GST	Global surface temperature time series.
HAAR	Haar filter.
HDDM	Hoeffding drift detection method.
HDist	Hamming distance.
HHT	Hilbert-Huang transform.
HItem	Hamming item.
HPF	High-pass filter.
HW	Holt-Winter's exponential smoothing.
IID	Independent and identically distributed.
IMF	Intrinsic mode function.
K-means	K-means.
KNN-CAD	K-nearest neighbors conformal anomaly detector.
KNN	K-nearest neighbors.
KZFT	Kolmogorov-Zurbenko Fourier transform.
LMM	Linear mixed model.
LPF	Low-pass filter.
LR	Linear regression.
LSTM	Long short-term memory neural network.
LT	Logarithmic transform.
LogReg	Logistic regression.
MAS	Moving average smoother.
MA	Moving average.
MBIC	Modified Bayesian information criterion.
MDDM	McDiarmid Drift Detection Method.
MDL	Minimum description length.

MEP	Maximum entropy partitioning.
MGT	Monthly global temperature time series.
MLE	Maximum likelihood estimation.
MLM	Machine learning methods.
MLP	Multilayer perceptron network.
MP	Matrix profile.
MRA	Multiresolution analysis.
MSE	Mean square error.
Min-Max	Min-max normalization.
NAB	Numenta anomaly benchmark.
NB	Naive Bayes.
NNET	Neural network.
NOAA	National Centers for Environment Information.
OLS	Ordinary least squares.
PAA	Piecewise aggregate approximation.
PACF	Partial autocorrelation function.
PCA	Principle component analysis.
PCT	Percentage change transform.
PDF	Probability density function.
PELT	Pruned exact linear time.
PPT	Phillips-Perron test.
PP	Prediction performance.
Page-Hinkley	Page's cumulative sum test.
RBF	Radial basis function network.
RFR	Random forest regression.
RF	Random forest.
ROC	Operating characteristic curve.
RSS	Residual sum squared.
SARIMA	Seasonal autoregressive integrated moving average.
SAX	Symbolic aggregate approximation.
SCP	Seminal change point.
SM	Structural model.
SOM	Self organizing maps.
STAMP	Scalable time series anytime matrix profile.
SVM	Support vector machine.
SW	Sliding window.
SoftED	Soft evaluation event detection.
TC	Temporal component.
TF	Theta forecasting.
THW	Taylor's double seasonal Holt-Winter.
VAE	Variational autoencoder.
VMD	Variational mode decomposition.
WFT	Windowed Fourier transform.
WPT	Discrete wavelet packet transform.
WT	Wavelet transform.

YGT	Yearly global temperature time series.
Z-Score	Standard score normalization.
sMAPE	Symmetric mean absolute percentage error.

Chapter 1
Introduction

Time series events are related to some phenomena observed in a time series. This introductory chapter provides the general principles of event detection. This chapter presents and organizes event detection by a general taxonomy. Section 1.1 provides an overview of events. Section 1.2 characterizes the different types of events, including anomalies, change points, and motifs. Section 1.3 provides the general data structure for time series event detection. Since time series event detection is a broad subject that involves searching for events through event detection methods, Sections 1.4, 1.5, 1.6 respectively cover detection strategies, detection scenarios, and detection evaluation. Finally, Section 1.7 provides the book structure.

1.1 Overview

Detecting events in time series demands the study of advanced and specialized methods and techniques [59]. It has been extensively explored over the years and has attracted increasingly more interest over the last decade [38]. Consider the Global surface temperature time series (`GST`), available from the National Centers for Environment Information (`NOAA`), which provides comprehensive data collections of global coverage over land (Global Historical Climatology Network-Monthly) and ocean (Extended Reconstructed Sea Surface Temperature) surface [199].

The `GST` is depicted in Figure 1.1 and is an example of a time series. A time series is a sequence of observations of an object of interest collected over time. When observations are related to a single variable, a time series is referenced as univariate. Conversely, it is a multivariate time series when observations are related to two or more variables. Commonly, the behavior of a time series is studied as a function of its past data [120].

Formally, we may consider a time series X as a sequence of n observations, $< x_1, x_2, x_3, \ldots, x_n >$, where x_1 and x_n represent, respectively, the observations of the series at the first (oldest) and last (newest) time points [93, 244]. The length n of a time series X is $|X|$. A specific time series observation is referenced as x_t, indexed

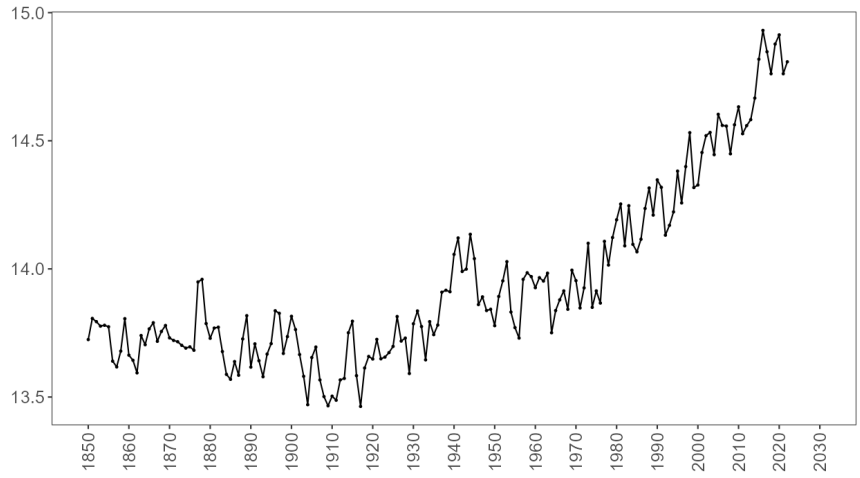


Fig. 1.1 `YGT` (in degrees Celsius) obtained from `NOAA` [199].

in time by t ($t \in \{1, \ldots, n\}$). It is common to interpret the sequences that compose a time series as a time series process [244].

The frequency of a time series defines when the observations are regularly collected. For example, a monthly time series is when observations are collected each month. Observations are called low-frequency time series when collected daily or slowly. Conversely, high-frequency time series are the ones in which the collection of observations occurs more than once during the day.

From these definitions, it is possible to observe that the global temperature is a monthly time series. It is a low-frequency time series measured in degrees Celsius. The `YGT` has been available since 1850. It is a relevant time series given the relevancy of discussion around global warming [196]. It can be used to discuss the outcomes of event detection methods while analyzing this time series.

For most real-world time series, it is often possible to observe the occurrence of a significant change in the behavior of a time series at a certain point or time interval. Such behavior change generally characterizes the occurrence of an event [117]. Unlike general outliers or noisy data [116, 125, 271], this event represents a phenomenon with a specific meaning to a certain domain. For example, it can mean a security breach or a failure in an industrial process [21, 205, 265].

Time series event detection is the process of identifying such events. It encompasses identifying past events (offline), discovering events in real-time (online), and even predicting future events. It is recognized as a basic function in surveillance and monitoring systems. Due to that, it has gained much attention in research for application domains involving critical systems [217]. The event detection problem becomes particularly relevant, especially for applications based on sensor data analysis [74, 275, 301]. Such applications can be observed in chemistry, reflection seismic, and oil drilling and exploration, where monitoring operations are essential. More-

over, event detection is crucial for practical applications that affect day-to-day life, such as medical diagnostic problems, fault detection, structural damage identification, intrusion detection, fraud detection, and astronomy object detection.

Event detection methods found in literature usually follow five general strategies: regression, classification, clustering, statistical, and domain. Unfortunately, assessing event detection methods across different domains is complex [302]. Due to the importance of event detection in several applications, many authors have analyzed, compared, and reviewed several methods. The number of papers can confirm the relevance of event detection subject [9, 203, 264, 265].

Event detection is a wide and dynamic area. However, papers and reviews of event detection in time series frequently focus on particular aspects of the problem. They usually limit themselves to study solutions for detecting only a specific type of event or methods that adopt a specific subset of detection strategies [21, 265]. Also, they do not present a common and general formalization of events and their different types. Moreover, most existing event detection algorithms and reviews do not properly apply to streaming applications [8]. Online event detection problem is currently prominent, but the available literature is still youthful [29, 180, 302].

In such a broad context, this chapter aims to gather representative concepts on event detection currently available in the literature to provide an overview of the event detection problem [29, 205, 261, 302]. The concepts are analyzed according to event type specializations, the approached event detection strategies, and evaluation metrics. This chapter generalizes events and their specialization as anomalies, change points, and motifs. The available literature is organized in a taxonomy. It presents a context for reviewed studies. It is depicted in Figure 1.2. It pinpoints the basis for deeper discussions in the following chapters.

The taxonomy was built based on a systematic mapping study. Firstly, studies on event detection for time series were searched using a query string involving the keywords “event detection” and “time series”. A second search was performed to find studies on the specific area of event prediction using the keywords “event prediction” and “time series”. Moreover, survey papers on event detection and event prediction were gathered based on snowballing. Finally, an additional snowball search was performed mainly on detection performance metrics due to their increasing importance.

The taxonomy is divided into five major categories. The first two categories regard event type and general data structure associated with events. The following categories group the studies according to their scenario, strategies, and evaluation. These categories are detailed in the next sections.

1.2 Types of events

Events are pervasive in real-world time series [117]. Generally, events can be observed in anomalies, change points, or motifs (frequent patterns). The first two commonly occur at a time point. Events related to intervals are generally studied as motifs or discords [79, 167, 290, 291]. Typically, event detection methods in the

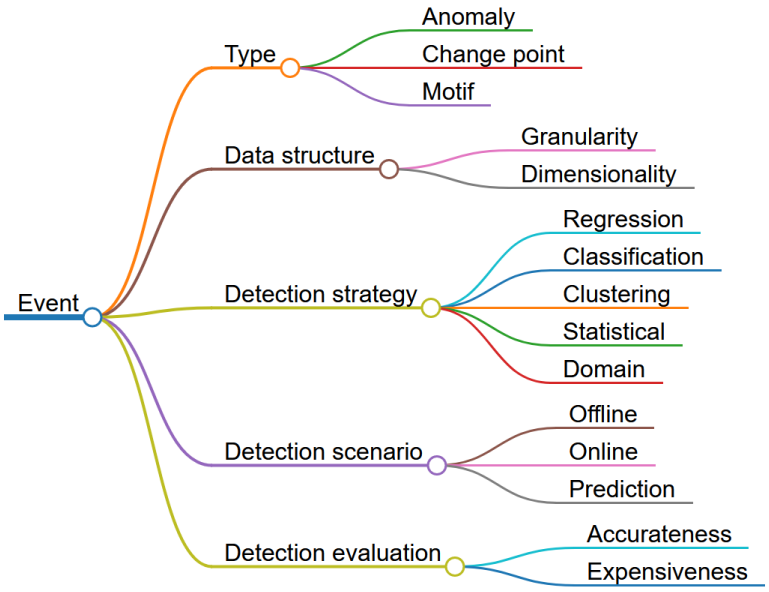


Fig. 1.2 Taxonomy of time series events

literature specialize in identifying a specific type of event. Although some methods can detect multiple events in time series, they are generally related to the detection of anomalies and change points [25, 26, 62, 255].

For the moment, the goal is to provide a general formalization of the time point events. Let x_t be observations for a time series X. A Temporal component (TC) for x_t is expressed as $tc(x_t)$. The TC can refer to the observation itself, its instant trend, and its instant volatility, respectively represented as $tc_o(x_t)$, $tc_{tr}(x_t)$, and $tc_v(x_t)$.

Consider the typical time series behavior in which autoregressive assumption holds [114]; a TC for x_t is expected to be relatively related to previous observations. Such assumptions can be express as $ep(x_t, k) = E(tc(x_t) \mid tc(x_{t-k}), \ldots, tc(x_{t-1}))$, where ep is the expected (E) TC for x_t from the previous k TC.

Using the same assumptions, it is expected that a TC for x_t can be explained from the following observations. It is similar to autoregressive assumption applied to a reversed time series [164]. So, a given $tc(x_t)$ is related to the following k observations. Such assumption can be expressed such as $ef(x_t, k) = E(tc(x_t) \mid tc(x_{t+1}), \ldots, tc(x_{t+k}))$, where ef is the expected (E) TC for x_t from the following k TC.

In general, time point events $\{t\}$ in a time series X can be expressed as $e(X, k, \sigma)$ using the Equation 1.1, where k represents the length of nearby observations. Suppose an TC for x_t escapes the expected value above a threshold σ based on previous or following k observations. In that case, it can be considered an event. Equation 1.1 also considers an event if the expected TC from previous and following k observations are different above a threshold σ.

$$e(X,k,\sigma) = \{t, |tc(x_t) - ep(x_t,k)| > \sigma \\ \vee |tc(x_t) - ef(x_t,k)| > \sigma \\ \vee |ep(x_t,k) - ef(x_t,k)| > \sigma\} \tag{1.1}$$

1.2.1 Anomalies

Most commonly, events detected in time series refer to anomalies. Time series anomalies are unexpected or abnormal patterns or events in a time series. These anomalies can be caused by various factors, such as errors in data collection, changes in underlying trends or patterns, or external factors, such as weather events or economic shifts [209, 222, 304].

Anomalies appear not to be generated by the same process as most of the observations in the time series [59]. Anomalies can be formalized as isolated observations of nearby data. In this case, an event identified in x_t can be considered an anomaly if it escapes expected TC before and after time point t according to σ threshold, as described in Equation 1.2.

$$a(X,k,\sigma) = \{t, |tc(x_t) - ep(x_t,k)| > \sigma \wedge |tc(x_t) - ef(x_t,k)| > \sigma\} \tag{1.2}$$

1.2.2 Noise anomalies

Noise anomalies, or residual anomalies, are unexpected or abnormal variations in the part of a time series that are not considered by the seasonal and trend components. They correspond to unexpected or abnormal variations in the residual component of a time series[1] [59].

Let $\hat{X}$ be an estimate of a time series X, produced by adjusting a model α, with $\hat{x}_t = \alpha(x_t)$. From $\hat{X}$, it is possible to derive the residual time series W ($< \omega_1, \ldots, \omega_n >$). When the model is well-adjusted, it is expected that W has a Gaussian distribution with zero mean[2] [267]. Noise anomalies (na) can be identified using outliers analysis over the residual time series W as described in Equation 1.3.

$$na(X) = outliers(W),\ \omega_t = x_t - \hat{x}_t \tag{1.3}$$

The simplest form of characterizing outliers is through a box plot using Equation 1.4, where $Q_1(W)$ and $Q_3(W)$ are the first and third quartiles, respectively, and IQR

[1] Time series components are detailed in Section 2.1

[2] In this case, it is considered a Gaussian white noise

is the interquartile distance [6, 116]. Equation 1.4 characterizes the time points in which observations are outliers in a box plot analysis.

$$outliers(W) = \{t, \omega_t \notin [Q_1(W) - 1.5 \cdot IQR(W), Q_3(W) + 1.5 \cdot IQR(W)]\} \quad (1.4)$$

Figure 1.3 presents the occurrence of noise anomalies (marked as red) from the YGT considering the residual time series obtained from an adjusted Autoregressive integrated moving average (ARIMA) model. As can be observed, such anomalies commonly correspond to local minimum and maximum values concerning the modeled time series.

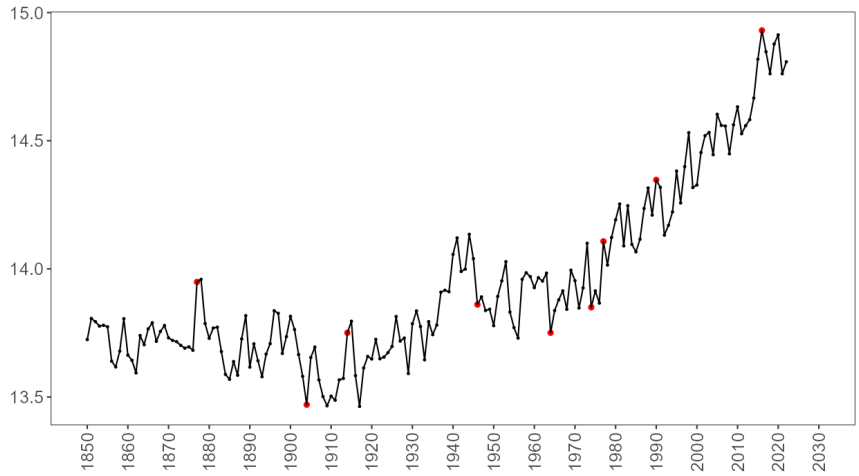


Fig. 1.3 Anomalies observed at YGT from the residual analysis of an ARIMA model

1.2.3 Volatility anomalies

Volatility anomalies are unexpected or abnormal fluctuations in the volatility of a time series. Volatility refers to the degree of variation or fluctuation in a time series. It can be measured using statistical methods such as standard deviation or variance.

Most financial time series exhibit non-linear properties, as the volatility of these series varies widely over time. Thus, there is a demand for studying the volatility of time series. Econometric models appear to address the non-linearity of data, including stochastic volatility, such as Autoregressive conditional heteroscedasticity model (ARCH) and Generalized autoregressive conditional heteroscedasticity model (GARCH), the latter being the most well-known and applied model [54]. The studies

applied to the financial area associate high volatility with risk. Such situations characterize anomalies in the time series.

Volatility anomalies (va) can be formalized by specializing Equation 1.2 to the volatility component. It is described in Equation 1.5. In this case, an event identified in time t can be considered a volatility anomaly if TC for the volatility component (tc_v) escapes expected volatility before (ep_v) and after (ef_v) time point t.

$$va(X, k, \sigma) = \{t, |tc_v(x_t) - ep_v(x_t, k)| > \sigma \wedge |tc_v(x_t) - ef_v(x_t, k)| > \sigma\} \quad (1.5)$$

A detailed discussion on anomalies is dedicated in Chapter 3.

1.2.4 Change points

Change points are time intervals in a time series where there is a significant change in the statistical properties. They represent a transition between different states in a process that generates the time series [85, 255, 265]. In this case, a change point event identified in time t follows the expected behavior observed before or after the time point t, but not both at the same time according to $cp(X, k, \sigma)$ in Equation 1.6. It can also refer to a significant difference between the expected trend before or after time point t.

$$\begin{aligned} cp(X, k, \sigma) = \{t, (|tc(x_t) - ep(x_t, k)| > \sigma \veebar\ |tc(x_t) - ef(x_t, k)| > \sigma) \\ \vee\ (|ep_{tr}(x_t, k) - ef_{tr}(x_t, k)| > \sigma)\} \end{aligned} \quad (1.6)$$

Change points are also related to concept drift [132, 278]. It is possible to define change point detection as a concept drift hypothesis test. The null hypothesis H_0 characterizes the absence of drift, and the alternative hypothesis H_A negates H_0. Formally, H_0 and H_A are as presented in Equation 1.7, where $\mathbb{P}_{n(t)}$ is the probability density function of observations nearby t [21][3].

$$\begin{aligned} H_0 &: \forall\, t, k\ (t \neq k) \mid \mathbb{P}_{n(t)} \approx \mathbb{P}_{n(k)} \\ H_A &: \forall\, t\ \exists\, k\ (t \neq k) \mid \mathbb{P}_{n(t)} \neq \mathbb{P}_{n(k)} \end{aligned} \quad (1.7)$$

From this perspective, change point detection is related to monitoring concept drifts. Both problems focus on observing considerable changes in time series behavior. In particular, concept drifts may happen due to environmental events such as change points. In this context, Gama et al. [104] review existing strategies for handling concept drift using adaptive learning processes. Other surveys describe

[3] Such observations could be a sliding window (see Section 2.3.6 for definition) containing t

concept drifts in the context of nonstationarity by presenting efficient algorithms and applications for learning and adapting to evolving environments [87].

Figure 1.4 presents the occurrence of change points (marked as dashed grey lines) from the YGT considering using Chow test statistic [298]. As observed, trends before and after change points are significantly different.

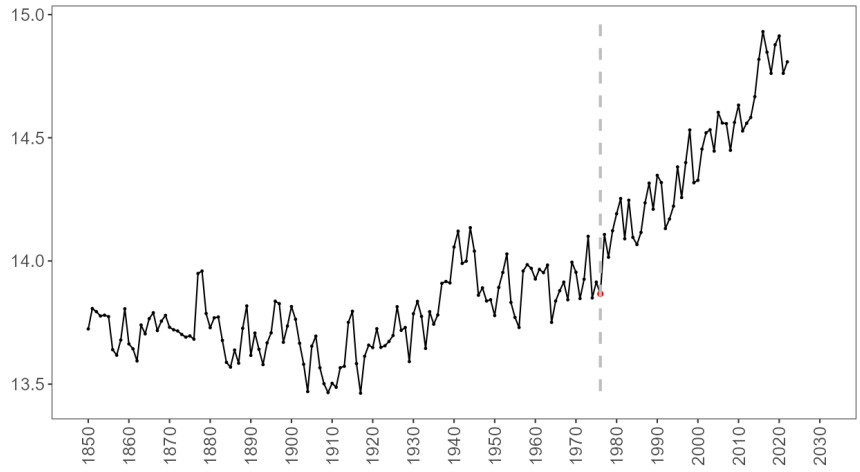


Fig. 1.4 Change points observed at YGT using Chow test statistic [298]

Among the studies that focus on identifying change point events or drifts in time series is the one by Guralnik and Srivastava [117] that gives one of the seminal papers on change-point discovery. Aminikhanghahi and Cook [21] comprehensively review different change point detection methods for time series. Also, Truong et al. [265] review algorithms for the offline detection of multiple change points in multivariate time series. Change point detection is discussed in Chapter 4.

1.2.5 Motifs

Events detected in time series can also be observed by the presence of *motifs*. Time series motifs are recurring patterns in a time series that have some distinctive shape and can be used to identify similar or identical patterns in other time series. In other words, a motif is a repeating subsequence within a longer time series. Motifs can be used to understand the underlying structure of a time series and to identify important features such as trends, seasonal patterns, or outliers [264].

Although they have a certain frequency, they are generally rare because they are commonly unknown and are discovered when scanning the time series. Identifying motifs can be useful in various applications, including anomaly detection, classifica-

tion, and predicting future behavior. For example, identifying motifs in physiological signals such as electrocardiograms (ECGs) can help diagnose heart conditions or predict future cardiac events. In financial applications, identifying motifs in stock prices can help identify trends and predict future price movements.

Given a sequence q and a time series X, q is a *motif* in X, if and only if q occurs in X at least σ times[4] [43, 191].

Figure 1.5 presents the occurrence of a motif in the `YGT`. The detected motif has a sequence length of three [166]. In this example, a sequence is considered a motif if it at least occurs thrice. The beginning of each motif is marked in red, and the following observations of each sequence are marked in purple.

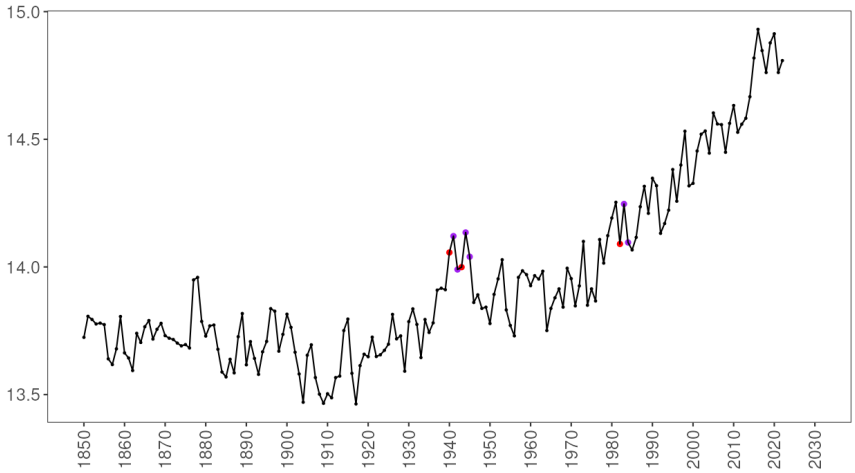


Fig. 1.5 Motif of size three observed at `YGT` [166]

The general approach for discovering motifs is related to directly exploring time series distances or correlations between subsequences. Due to this brute force exploration, they generally have quadratic complexity $O(n^2)$. It is common to adopt indexing techniques, such as Symbolic aggregate approximation (`SAX`) [166], for faster computation time while introducing similarity tolerance for the sequences. It enables hashing techniques to decrease computational time. Besides, some advanced techniques, such as Matrix Profile [290, 291], adopt enhanced data structure for fast computation. The formalization of motifs and their discovery is discussed in Chapter 5.

[4] A more formal definition of motif is provided in Chapter 5.

1.3 Data structure

As discussed in Section 1.1, a time series is a collection of observations taken at regular intervals over time. Events correspond to an observation or a sequence of observations that are characterized according to a specific type (anomalies, change points, or motifs). Both time series and events can be represented in various data structures depending on the context and purpose of the analysis [136].

One common data structure for time series is a simple table or matrix, where each row represents a specific time point, and each column represents a variable or feature being measured at that time point. Under this structure, an additional column can represent the presence of events, with the characterization of the type of event. It is a typical format for many time series and associated events [136].

Another common data structure for time series is a specialized object or class in a programming language or statistical software package. These objects are specifically designed to handle time series and often have built-in functions and methods for manipulating and analyzing the data. Examples include the Pandas library in Python or the ts object in R [224, 266].

In addition to these basic structures, more advanced techniques such as decomposition or state space models can represent and analyze time series, particularly when dealing with multiple variables or complex dependencies over time [244].

1.3.1 Granularity

Generally, anomalies fall into two general categories: punctual and interval [29]. They are related to the granularity of the time series. Punctual events are individual data instances that are interpreted as events concerning the rest of the data. In other words, they are discrete events recorded at a particular timestamp or time interval. Punctual events are typically represented in a timestamped event sequence data structure. This structure stores information about each event, including the time it occurred and any associated data.

Interval events refer to events that occur over a defined period rather than at a specific time. Interval events are typically recorded as a start and end time, along with the time series. Examples of interval time series events include continuous daily rainfall or high hourly electricity consumption.

Analyzing interval time series events requires different techniques than analyzing punctual events. One common approach is to aggregate the data into larger time intervals, such as weekly or monthly aggregates, and then analyze these aggregates using time series modeling techniques such as `ARIMA`.

The `YGT` presented so far is derived from the `MGT`. Figure 1.6 presents a sample of the Monthly global temperature time series (`MGT`). Figure 1.6a presents the `MGT` from January 1971 until December 1980. Figure 1.6b presents the `YGT` for the same period. It is temporal aggregated from the `MGT`. The value for the `YGT` is the mean of `MGT` each year.

year\month	1	2	3	4	5	6	7	8	9	10	11	12
1971	13.9	13.8	13.8	13.8	13.9	13.8	13.9	13.9	13.9	13.8	13.9	13.9
1972	13.7	13.7	14.0	14.0	13.9	14.0	14.0	14.0	13.9	14.0	14.0	14.0
1973	14.3	14.3	14.3	14.2	14.1	14.2	14.1	14.0	14.0	14.0	13.9	14.0
1974	13.8	13.7	13.9	13.9	13.9	13.8	13.9	14.0	13.9	13.8	13.8	13.8
1975	14.0	14.0	14.0	14.0	14.0	14.0	13.9	13.9	13.9	13.8	13.7	13.8
1976	13.9	13.8	13.8	13.9	13.8	13.9	13.9	13.9	13.9	13.7	13.9	14.0
1977	14.1	14.1	14.2	14.2	14.2	14.2	14.1	14.1	14.1	14.0	14.1	14.0
1978	14.1	14.1	14.1	14.0	14.0	14.0	14.0	13.9	14.0	14.0	14.1	14.0
1979	14.0	13.8	14.1	14.0	14.1	14.1	14.1	14.2	14.2	14.2	14.2	14.4
1980	14.3	14.3	14.2	14.2	14.3	14.2	14.2	14.1	14.1	14.1	14.2	14.1

(a)

year	value
1971	13.8
1972	13.9
1973	14.1
1974	13.8
1975	13.9
1976	13.9
1977	14.1
1978	14.0
1979	14.1
1980	14.2

(b)

Fig. 1.6 Examples of global temperature time series under different granularity between 1971-1980: (a) Monthly global temperature (`MGT`); (b) Yearly global temperature (`YGT`)

1.3.2 Dimensionality

Event detection might occur in unidimensional or multidimensional time series. A unidimensional time series is a time series where only one variable is measured in each observation. This variable can be a continuous or discrete numeric value, and it can be measured at regular or irregular intervals.

Unidimensional time series events are typically represented as a sequence of values over time. The simplest data structure used to represent it is a vector. Although simple, unidimensional time series events are important as they can provide valuable insights into various phenomena.

A multidimensional time series is a time series where multiple variables are measured in each observation. Each variable can be continuous, discrete, or categorical values. Examples of multidimensional time series events include a city's hourly temperature, humidity, and wind speed readings.

Multidimensional time series events are typically represented as a matrix or table with multiple columns. Each column represents a measured variable, and each row represents an observation or measurement taken at a specific time. It is similar to the table representation in the Relational Model, but the order of data is relevant.

Figure 1.7 illustrates the representation of different time series. Figure 1.7a presents the `YGT` as a vector. It is a univariate time series. Figure 1.7b presents the `YGT` with an additional column to characterize the time associated with each observation. Finally, Figure 1.7c presents a multidimensional time series. It has a column for the `YGT` and the worldwide crude oil production. In a multidimensional time series, one might be interested in finding associations among the different dimensions. In this example, if there exists any association between crude oil consumption and the global temperature increase.

value
13.8
13.9
14.1
13.8
13.9
13.9
14.1
14.0
14.1
14.2

(a)

time	value
1971	13.8
1972	13.9
1973	14.1
1974	13.8
1975	13.9
1976	13.9
1977	14.1
1978	14.0
1979	14.1
1980	14.2

(b)

time	global temperature	crude oil production
1971	13.8	2491
1972	13.9	2634
1973	14.1	2870
1974	13.8	2875
1975	13.9	2740
1976	13.9	2966
1977	14.1	3069
1978	14.0	3108
1979	14.1	3229
1980	14.2	3111

(c)

Fig. 1.7 Examples of time series dimension representation: (a) univariate time series as vector; (b) univariate time series with timestamp column; (c) multivariate time series

1.4 Detection strategies

Due to the increasing demand for event detection in real-world applications, many event detection methods have been developed and are currently available in the literature. Several methods have been described and compared by Chandola et al. [59]. Moreover, Ariyaluran Habeeb et al. [29] overview the state-of-the-art machine learning-based big data processing technologies for event detection.

It is possible to classify the event detection methods into five general strategies: regression, classification, clustering, statistical, domain [59, 217]. Besides, event detection methods can also be classified into theory-driven or data-driven according to the approach adopted. Theory-driven models are based on established theories, principles, or concepts from a particular field. These models aim to represent and explain real-world phenomena by explicitly incorporating theoretical constructs into the model. Conversely, data-driven models are created by analyzing large datasets. While theory-driven techniques are mostly in domain analysis and statistics-based, machine learning generally enables data-driven techniques.

Data-driven methods based on machine learning are not necessarily restricted to certain kinds of problems, and they apply to time series event detection. Some of the main data-driven methods used for event detection are K-nearest neighbors conformal anomaly detector (`KNN-CAD`), `K-means`, Neural network (`NNET`), Support vector machine (`SVM`), Extreme learning machines network (`ELM`), Convolutional neural network for time series (`Conv1D`), and Long short-term memory neural network (`LSTM`). Particularly, `NNET`, `ELM`, `SVM`, `Conv1D`, and `LSTM` are also used for time series prediction [194].

Data-driven techniques can also be organized according to the training strategy. In supervised mode, the techniques assume the availability of a training data set with labeled instances (for example, the indication that each observation has an associated event or not). In semi-supervised mode, they assume that the training data only label instances for the normal class. On the other hand, techniques that operate in unsupervised mode do not require labels in training data and, thus, are most widely

applicable. The techniques in this category implicitly assume that normal instances are more frequent than anomalies [59].

Domain strategies use expert knowledge or established models of the domain problem to identify new data samples as abnormal in case they differ enough from expected. The literature presents several studies that employ this strategy. Gabarda and Cristóbal [102] present a domain-specific technique for finding events in a seismic dataset using time-frequency methods. Zhang et al. [300] identify human fall events based on a model of body posture evolution.

1.5 Detection scenarios

The detection scenarios are an important characterization of the condition in which the event detection methods are used. They drive requirements for data management, computational resources, and constraints of the event detection methods. It is possible to organize the detection scenario into three major classes: offline event detection, online event detection, and event prediction.

Offline event detection is when events are discovered after the time series has been collected. It involves analyzing the time series retrospectively to identify patterns or changes that may indicate the occurrence of an event. The process of offline event detection typically involves two major steps. First, the time series is preprocessed. Then, a time series analysis technique is applied to identify events. The technique may include one of the strategies previously mentioned.

Once potential events have been identified, further analysis is done to confirm whether they are real events or false positives. This analysis can involve additional data sources, expert knowledge, or further statistical testing.

Online event detection is when events are discovered in a time series as the observations are collected. It comprises continuously monitoring the time series. The process of online event detection is typically comprised of the two major steps depicted in offline detection. However, it has computational constraints to enable real-time reactions when potential events are discovered. For example, predictive maintenance can detect changes in machine behavior that may indicate a potential failure, allowing maintenance to be scheduled proactively.

More formally, let X be a streaming time series composed of a continuous stream of inputs: $< \ldots, x_{t-2}, x_{t-1}, x_t, x_{t+1}, x_{t+2}, \cdots >$. At each point in time t, a model trained on previous observations $< \ldots, x_{t-2}, x_{t-1} >$ is used to determine whether the current behavior of the system is unusual. This determination must be made by the next input x_{t+1}. For this, the time series model must be continuously updated [8].

Unlike offline event detection, data is not split into static train/test sets, and algorithms cannot look ahead. Practical applications impose additional challenges. Typically, the sensor streams are large in number and at high velocity, leaving little opportunity for expert intervention, manual parameter tweaking, and manual data labeling. Thus, operating in an unsupervised, automated fashion is necessary [8]. Furthermore, in streaming applications, early detection of events is often important,

where events are identified as soon as possible. Detection of events can give information that is crucial to decision-making. This information must be given early enough that it is actionable, preventing possible critical system failures. However, there is still a trade-off between early detections and false positives. An algorithm that often makes inaccurate detections is likely to be ignored.

Finally, time series event prediction forecasts future events based on historical time series. It involves using statistical, machine learning, or deep learning techniques to model the patterns and trends in the time series and make predictions about future events.

The process of time series event prediction typically involves several steps. First, the time series is preprocessed and cleaned to remove any noise or outliers that could affect the accuracy of the analysis. Then, various time series analysis techniques are applied to model the patterns and trends in the data. These techniques can be from `ARIMA` to deep learning models such as `LSTM`.

Once the model has been trained and validated, it can be used to predict future events. Time series event prediction is used in various applications, including finance and healthcare. In healthcare, for example, it can be used to forecast patient outcomes or the spread of diseases.

1.6 Detection evaluation

The evaluation of event detection methods is related to determining the effectiveness of the event detection algorithms. Additionally, they differ according to computational time and memory used.

1.6.1 Accurateness

The evaluation of time series event detection assesses the accuracy and effectiveness of event detection algorithms in identifying events in time series. It involves comparing the output of the algorithm with ground truth data, which experts have manually labeled to indicate the occurrence of events. The evaluation of event detection algorithms is challenging due to the subjective nature of event labeling. Many applications lack ground truth labels.

The evaluation typically involves several metrics. The metrics are calculated using the ground truth data and the output of the algorithm to evaluate the accuracy of the event detection algorithm. These metrics include precision, recall, F1-score, and area under the receiver operating characteristic curve (AUC-ROC). Precision is the ratio of true positive events to the total detected events. At the same time, recall is the ratio of true positive events to the total number of actual events. The F1-score is a weighted average of precision and recall that considers both measures. The AUC-

ROC measures the algorithm's ability to discriminate between positive and negative events [233, 260].

An important aspect of any event detection method is how the events are reported. Typically, the outputs produced by event detection methods are either scores or labels. Scoring detection methods assign an anomaly score to each instance in the data. It varies according to the degree to which that instance is considered an anomaly. On the other hand, labeling detection methods assign a label (normal or anomalous) to each data instance [59].

Detection methods might vary performance under different time series [95]. Therefore, there is a demand for comparing the results provided by them. Benchmarking and comparing event detection performance is a complex task [159]. Standard classification metrics are generally used for evaluating the performance of an algorithm to distinguish normal from abnormal data samples [59, 260]. Detection evaluation measures have also been investigated in the areas of sequence data anomaly detection [8, 116], time series mining and representation [83], and sensor-based human activity learning [277]. Metrics found in the literature are mainly designed to evaluate the detection of punctual anomalies. However, many real-world event occurrences extend over some time (range-based). Motivated by this, Tatbul et al. [260] expand the well-known Precision and Recall metrics to measure the accuracy of detection algorithms over event ranges.

1.6.2 Expensiveness

Event detection methods vary according to the computational resources required. The complexity of time series event detection can be analyzed using several measures, including time complexity, space complexity, and computational complexity. Time complexity is the time required to execute the algorithm on a given input size. In contrast, space complexity refers to the amount of memory the algorithm requires. Computational complexity refers to the resources the algorithm requires, including time, memory, and other computational resources.

Several techniques can be used to manage the complexity of time series event detection. These include optimizing the algorithm and its implementation, using parallel processing to distribute the computational workload, and using data reduction techniques to reduce the size of the time series.

Detection evaluation is covered in Chapter 6.

1.7 Book structure

This introductory material provided an overview of the subject. The taxonomy depicted in Figure 1.2 categorizes the main concepts from which the book goes in an in-depth analysis. Figure 1.8 provides the general book structure. Chapter 2 provides

the basics of time series analysis. Although some advanced readers might skip it, the chapter provides background material from which other sessions can directly discuss the studied concepts without a time series background introduction. It covers time series decomposition, time series preprocessing, and time series prediction.

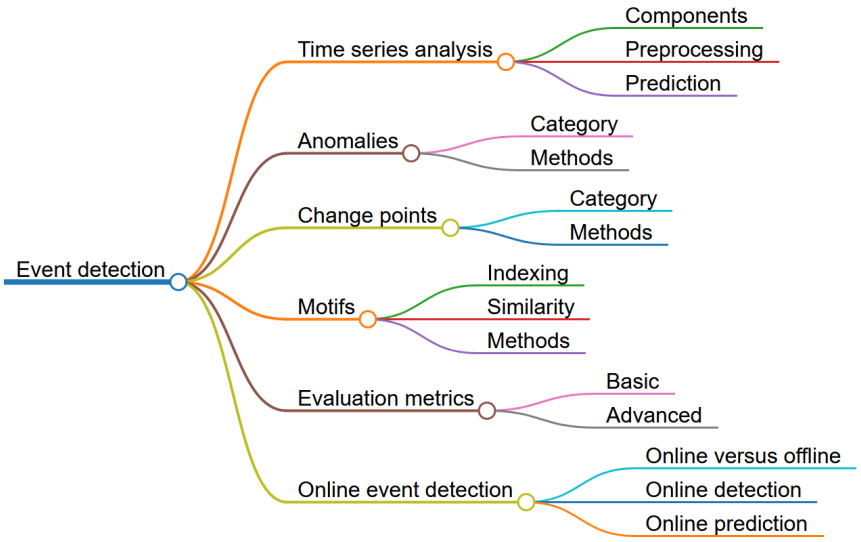


Fig. 1.8 Book structure

Chapter 3 provides the categories of anomalies and the main anomaly detection methods. Chapter 4 provides the categories of change points and the main change point detection methods. It also covers concept drift in time series since it is related. Chapter 5 discusses time series indexing and similarity, which are the main enabling concepts for motif discovery. Later, the chapter covers the main motif discovery methods.

Chapter 6 covers the main evaluation metrics on event detection. It also discusses time tolerance from two important aspects. The first is relating the number of observations needed to detect an event. The second indicates that temporal close detections as false positives and false negatives should not rule with an iron fist since they are close. Chapter 7 discusses online event detection. It starts comparing online with offline detection. It then differentiates between online detection (detecting events as soon as possible) and online prediction (predicting events prior to their occurrences). Finally, Chapter 8 summarizes the research area and the main challenges discussed in the book. It ends by presenting the Harbinger framework, made available as an R package, from which all examples presented in this book were built.

Chapter 2
Time series analysis

An overview of the time series analysis is needed to comprehend event detection methods better. This chapter covers time series components, described in Section 2.1. Then, it formalizes stationarity in Section 2.2. Section 2.3 presents major time series data preprocessing techniques, including temporal aggregation, extraction of trend components, time series variance stabilization, detrending and differencing, time series decomposition, sliding windows, data normalization, and data splitting. Finally, Section 2.4 describes the basics of time series prediction using statistical and machine learning principles.

2.1 Time series components

It is very common to study time series according to their major components. They are commonly organized into three types of components: *(i)* trend (β_t); *(ii)* seasonality (π_t); *(iii)* residual (ω_t) as described by Equation 2.1 [130].

$$x_t = \beta_t + \pi_t + \omega_t \tag{2.1}$$

The trend component refers to the time series's long-term movement or direction. It represents the underlying pattern or trend of the data, independent of any seasonal or cyclical fluctuations. When the trend is linear, it is a monotonic function or a function in which there can be at most one extreme within a given data period [187]. Generally, the trend is taken as a disposition of the time series dataset, which remains in future observations [97]. The trend can be rising, falling, or stable over time. For example, a country's gross product may steadily increase over several years, indicating a positive trend. A neonatal death rate may steadily decrease over time, indicating a negative trend. Both these situations are good for policymakers. So, the relevance of positive or negative trends is much more related to the semantics of the studied time series rather than its values [130].

It is important to identify and account for trends in time series to make an accurate analysis. There are several processes to determine the trend component. One of the most known is linear regression, choosing an average line for the time series. Others include moving averages and other concepts discussed later in this chapter. Additionally, detrending can remove the trending component of the time series. This approach targets enabling the focus on the remaining time series components [114].

Seasonality refers to patterns that repeat at fixed intervals, such as daily, weekly, monthly, or yearly. These patterns usually relate to periodical factors influencing the underlying data. For example, sales may increase every December due to Christmas, or beverages and ice cream sales may increase during summer. Seasonality is also expressed according to amplitude, frequency, and phase. The amplitude refers to the magnitude of seasonal variation. The frequency refers to the length of the seasonal cycle. Finally, the phase refers to the timing of seasonal peaks and troughs within each cycle [130].

Finally, the residual is the difference between the time series and the sum of the trend and seasonal components. The residual component represents the random or unpredictable fluctuations that the trend or seasonal components cannot explain. It can be referred to as a stochastic process [244]. It is sometimes called the "noise" in the time series. When the noise has a Gaussian distribution with zero mean, it is called Gaussian white noise [267].

Figure 2.1 presents the decomposition of the `YGT` from 1970 until nowadays. The decomposition is produced using the R package forecast [130]. The trend component is extracted using a moving average. The seasonal component is computed by averaging through the defined frequency. Since it is a monthly time series, the frequency is 12. The residual component is determined by removing the trend and seasonal components of the time series.

By intuitively looking at Figure 2.1, the temperature presents a rising trend. The seasonal component presents a major sinusoidal yearly (with some spikes). The mean of residual equals zero, and its variance is limited, which seems to be a white noise [267]. These intuitive analyses are discussed in the following chapters.

2.2 Stationarity

Most methods applied for time series analysis assume that the behavior of a time series presents a level of regularity over time, which is generally approached with the study of the concept of stationarity [114]. A widely adopted definition of stationarity established that given a time series X [114, 244]:

i the mean function, $E(X) = \mu$, is constant and does not depend on time;
ii the variance function, $VAR(X) = E\left((X - \mu)^2\right) = \sigma^2$, is constant and does not depend on time;
iii the autocovariance function, $\gamma(X_s, X_t)$, between the time-shifted time series X_t and X_s depends only on the difference $|s - t|$.

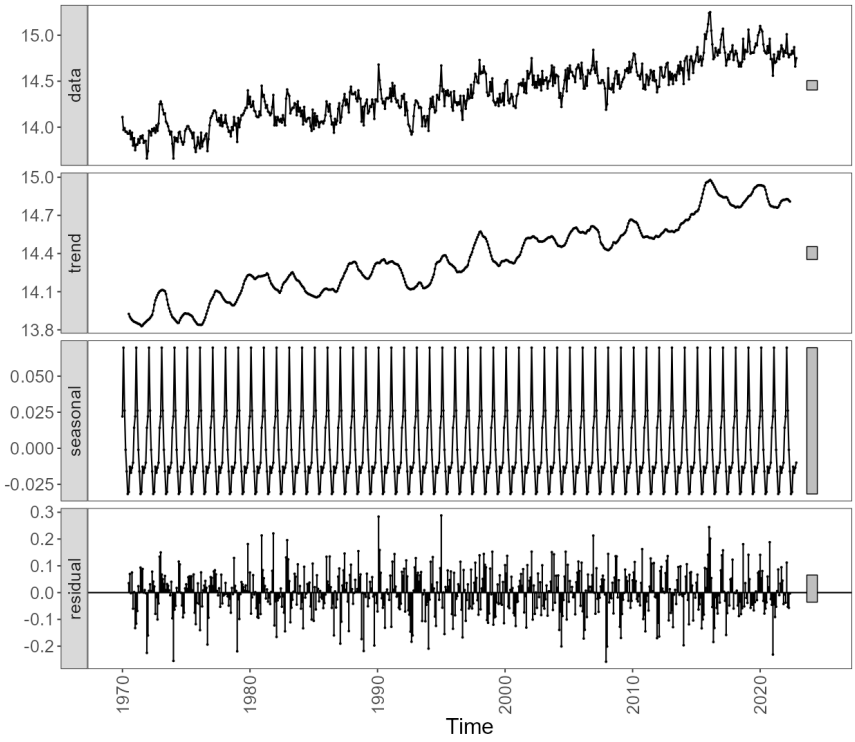


Fig. 2.1 `YGT` decomposed using R package forecast [130]. It presents the time series, its trend, seasonal, and residual components.

In other words, in a stationary time series, the statistical properties of the mean, variance, and covariance remain constant over time [288]. These constraints are important since they enable statistical inference based on any sampled time series subset [120]. In other words, it is a time series where the data is unaffected by external factors such as trends, seasonality, or other patterns.

Suppose a time series does not follow any constraints a stationary process imposes. In that case, it is considered a nonstationary time series. Non-stationarity may manifest in many different ways. Generally, it implies that a time series mean or variance functions depend on time t [235]. The changes are often due to deterministic trends, structural breaks, level shifts, or changes in variance (a condition known as heteroscedasticity). They can also be due to the presence of unit roots [120].

An example of a stationary time series may be visualized in Figure 2.2a, where one may observe mean and variance functions independent of time. It represents a First order autoregressive model (`AR(1)`), which is defined as in Equation 2.2, where α is a constant and $\omega_t \sim N(0, \sigma_\omega^2)$ [114]. In this example, $\sigma_\omega^2 = 2$, $\theta = 0.5$, and $\alpha = 0$. Since $0 < \theta < 1$, any relevant impacts of past observations eventually become negligible and do not affect the global behavior of the time series. It follows that this

model presents constant statistical properties such as $E(x_t) = \mu_t = \alpha/(1-\theta) = 0$ and $VAR(x_t) = \sigma_\omega^2/(1-\theta^2) \sim 2$, hence being considered stationary.

$$x_t = \alpha + \theta x_{t-1} + \omega_t, \;\; 0 < \theta < 1 \tag{2.2}$$

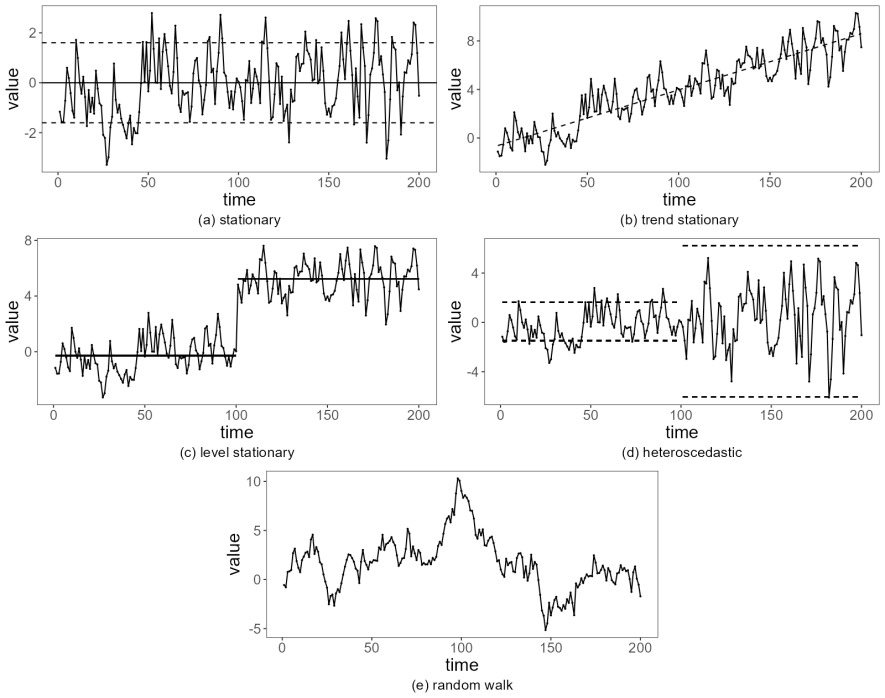


Fig. 2.2 Examples of time series (adapted from Salles et al. [235]): stationary (a); trend stationary (b); level stationary (c); heteroscedastic (d); and difference stationary (e). Except for (a), all examples are related to nonstationary time series. The solid and dashed black lines represent the time series mean and variance functions.

Figures 2.2b, 2.2c, 2.2d and 2.2e show representative nonstationary time series. They correspond to nonstationary samples due to trend, structural break, heteroscedasticity, and unit root.

A trended model might be considered the simplest form of nonstationary time series [235]. This model represents a process that has stationary behavior around a deterministic trend. This trend shifts the mean of a time series, causing it to increase or decrease over time. Commonly, the deviations of a systematic trend may be a stationary variable, known as a detrended variable, which may be analyzed instead of the original time series. In that case, usual stationary models are applicable [120, 288]. A time series that presents this behavior is called trend stationary. The time series represented in Equation 2.3 describes the example of Figure 2.2b, where

βt is the trend term and $\omega_t \sim N(0, \sigma_\omega^2)$ represents white noise. The mean function of the process $E(x_t) = \mu_t = \beta t$ varies over time, violating the stationary constraints.

$$x_t = \alpha + \theta x_{t-1} + \beta t + \omega_t, \;\; 0 < \theta < 1 \tag{2.3}$$

Structural breaks may also cause non-stationarity in a time series that happens at specific points in time, usually due to environment changes [235]. These structural breaks may eventually result in level shifts in a time series, which cause the mean function to be different for different portions of the series. In that case, a time series can be partitioned. One can separately analyze each data portion with different statistical properties, provided that the timing of a structural break is known [120]. Figure 2.2c shows an example of a level stationary time series where one can see the level shifts of the mean function. The plot in Figure 2.2c represents the model in Equation 2.4, which is derived from Equation 2.2 by the addition of a dummy variable d_t, with the level shift $\delta = 5$ and the time of the structural break $t_b = 100$. This model gives another example of non-stationarity in the mean.

$$x_t = \alpha + \theta x_{t-1} + \delta d_t + \omega_t, \;\; 0 < \theta < 1, \;\; d_t = \begin{cases} 0, & t \leq t_b, \\ 1, & t > t_b. \end{cases} \tag{2.4}$$

Another cause of non-stationarity resulting from structural breaks is the change in variance over time, commonly known as heteroscedasticity [120, 235, 244]. Heteroscedasticity arises from environmental changes that make the volatility of time series observations increase/decrease over time. The time series which presents this condition is called heteroscedastic. Analogous to level shifts, the different variance properties in different portions of a time series can be addressed by partitioning the series or modeling the changes in variance with a dependency on the structural breakpoints. An example of a heteroscedastic time series is depicted in Figure 2.2d, where different variance properties on the first and last portions of the series are easily observable. The series presented in Figure 2.2d represent the same model defined in Equation 2.2 and the same series of Figure 2.2a, but in this case, ω_t is set as $\omega_t \sim N(0, \sigma_\omega^2 = 2)$ for $t = 1, \ldots, 100$ and $\omega_t \sim N(0, \sigma_\omega^2 = 4)$ for $t = 101, \ldots, 200$. This model gives an example of a time series that presents non-stationarity in the variance.

An important non-stationarity often observed in real-world series is caused by a unit root in the characteristic polynomial of a time series model [235]. Without a unit root, time series observations tend to fluctuate around deterministic components such as a mean or a trend. Conversely, observations do not revert to a historical level when a unit root is present and may wander in any direction. The presence of a unit root implies that the time series suffers from the influence of long-run components or stochastic trends [120]. In that case, removing a stochastic trend, usually done by applying a process called differencing, is often helpful to coerce such time series to stationarity. Therefore, nonstationary time series that present unit roots are also known as difference stationary [44]. A difference stationary time series is presented in Figure 2.2e, which represents the so-called random walk model, formulated as in Equation 2.5. This model assumes that the value of a time series x_t at a time t can

be explained by the value of the series at the time $t - 1$ plus a random movement represented by ω_t [244].

$$x_t = \alpha + x_{t-1} + \omega_t = \alpha + \sum_{i=1}^{t} \omega_i \tag{2.5}$$

The random walk model in Equation 2.5 is also derived from the `AR(1)` model in Equation 2.2 by defining $\theta = 1$. The definition of $\theta = 1$ implies that any impacts caused by past observations permanently affect the global behavior of a time series. In this case, the mean function of the process is not fixed, and the variance function tends to increase over time [120] as can be observed by Equation 2.6. This model gives an example of a process that presents a unit root and is nonstationary both in mean and variance [288].

$$VAR(x_t) = VAR(\alpha) + \sum_{i=1}^{t} VAR(\omega_i) = t\sigma_\omega^2 \rightarrow \infty \; as \; t \rightarrow \infty \tag{2.6}$$

The mean and variance evaluation for stationary time series are relatively intuitive. The test for autocovariance can be computed using Autocorrelation function (`ACF`) and Partial autocorrelation function (`PACF`) of the residuals of an `AR(1)` model along with the corresponding confidence intervals for a pair of compared lags [114]. Such a discussion is outside the scope of this book, but it can be visually explored by plotting `ACF`. The `ACF` measures the correlation between a time series and its lagged values. In other words, it measures how much observations in a time series are related to their previous observations. Whenever `ACF` is above the dashed threshold, that particular lag is related to the current value. While working with stationary time series, as lags increase, `ACF` rapidly decreases to zero. Conversely, in nonstationary time series, such convergence goes slowly or might not occur. Figures 2.3a, 2.3b, 2.3c, 2.3d, and 2.3e provide a visual example of `ACF` for the examples of Figure 2.2 (stationary, trend stationary, level stationary, heteroscedastic, nonstationary), whereas Figure 2.3f is related to `YGT`. For both Figures 2.2a and 2.2d, the `ACF` rapidly goes to zero. The other examples indicate potential nonstationary time series from the perspective of `ACF`. The heteroscedasticity of 2.2d is not captured by `ACF`.

In addition, there are well-established statistical tests that can be used in time series for testing stationarity. The Augmented Dickey-Fuller test (`ADFT`) and Phillips-Perron test (`PPT`) investigate whether a time-ordered set of observations contains a unit root and is, therefore, nonstationary [91]. These tests have the null hypothesis of nonstationary time series. Suppose the p-value is lower than a predefined significance level (5%). In that case, it goes to the alternative hypothesis of stationary time series. Besides, the Breusch-Pagan test (`BPT`) evaluates a linear regression model of the time series to check for heteroscedasticity [114]. Since none of these tests checks all aspects of non-stationarity, they are usually applied in conjunction to confirm the stationarity of a time series. Table 2.1 evaluates the time series presented. All series marked with X were considered nonstationary according to the respective test. Since the trend stationary time series is a simple case of non-stationarity, none of these tests considered it nonstationary. The level stationary case was detected by `ADFT`.

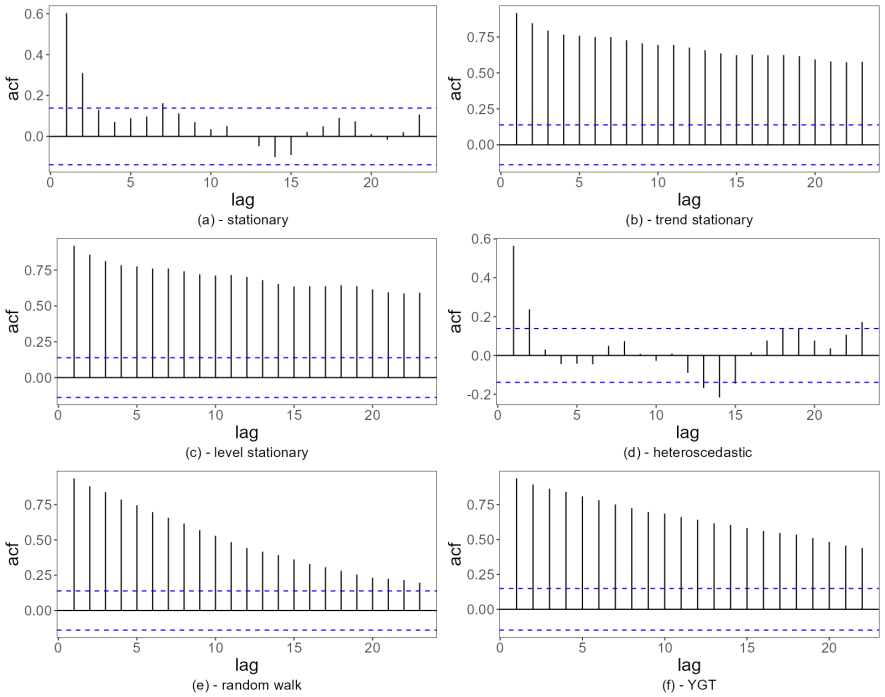


Fig. 2.3 Examples of the ACF for stationary time series of Figures 2.2a (a), 2.2b (b), 2.2c (c), 2.2d (d), and 2.2e (e) and the ACF of YGT (f).

The heteroscedasticity was only detected by BPT. All three applied tests detected the difference stationary. The YGT was only marked as nonstationary by BPT.

Table 2.1 Tests for non-stationarity time series. Marked values correspond to non-stationarity of the time series detected by the statistical test

Time series	ADFT	PPT	BPT
stationary			
trend stationary			
level stationary	X		
heteroscedastic			X
difference stationary	X	X	X
YGT			X

It is also important to remark that frequently, the dependence of a time series on past data may occur by multiples of some underlying seasonal lag S. In that case, a time series presents periodic components. Therefore, its statistical properties, such as mean and variance, may periodically change, creating a dependence on time t. It makes seasonality another form of non-stationarity, often found in time series [288].

Generally, if not properly addressed, any non-stationarity can have a relevant impact on time series prediction applications. Overlooking non-stationarity properties in a time series may lead to misleading statistical inferences and bad or unexpected prediction results. The next section categorizes some relevant data preprocessing for the event detection subject. Many of them are prepared to address nonstationary time series.

2.3 Data preprocessing

Adequate data preprocessing is an important activity in any application aiming at data analytics. It commonly encompasses more than 60% of the data mining process. Data cleaning, feature selection, sampling, outlier removal, normalization, and data transformation are commonly performed during preprocessing [118, 223].

Time series data preprocessing is important for modeling time series [93]. The main goal of data preprocessing is to guarantee the quality of the data before it is fed to any learning algorithm [105, 223]. This context encompasses problems of classification (prediction of discrete data) and regression (modeling or prediction of continuous data) [52, 93, 118]. This chapter focuses solely on transformation methods that target the regression problem. The reason is that such methods are useful for modeling and aiding the actual regression of the time series. These tasks are relevant during time series event detection. While there are many tasks involved during the time series data preprocessing, it is possible to summarize the most common ones:

- Data cleaning: is basic for general data analytics activities and is well-covered by literature [118, 281]. It involves handling missing values, outliers, and errors in the data. Missing values can be filled in using techniques such as interpolation or imputation. Some outliers can be detected using statistical methods. They are commonly removed or smoothed. However, outliers in the scope of this book are generally addressed as anomalies. They are discussed in Chapter 3.
- Temporal aggregation: depending on the frequency of the data, it may need to be resampled to a higher or lower frequency to match the desired analysis time frame;
- Data transformation: It involves creating new features or variables from the existing data to capture additional information or relationships. It includes trend components, variance stabilization, detrending and differencing, and decomposition [93, 114, 244];
- Sliding window (`SW`): The basic idea is to divide a time series into smaller subsequences. They provide ways to perform time series analysis or modeling on each subsequence separately [43, 201];
- Data normalization: It involves scaling the data to a common range or unit to be compared or combined with other data or to support machine learning methods. Common normalization techniques include Standard score normalization (`Z-Score`), Min-max normalization (`Min-Max`), and log transformation [118];

- Data splitting: partitioning the data into training, validation, and testing sets is important to enabling model build and evaluation [130].

2.3.1 Temporal aggregation

Temporal aggregation refers to summarizing or grouping time series data into higher frequencies as depicted in Figure 2.4. It can be applied to higher (such as minutes, hours, and parts of the day) or lower frequency time series (days, weeks, months, quarters, and years). The purpose of temporal aggregation is to reduce the level of detail in the data, which can help to simplify analysis and visualization, as well as to remove noise or fluctuations that may be present at the finer time scales.

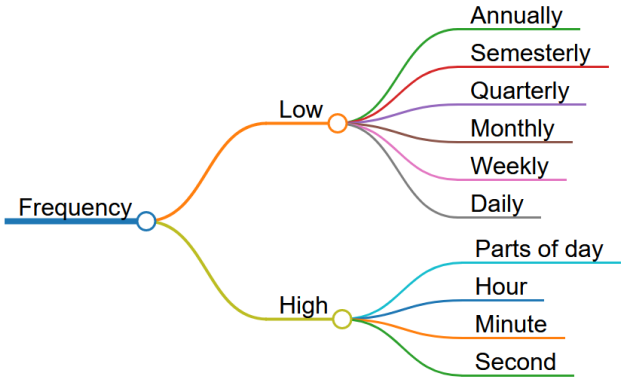


Fig. 2.4 Temporal aggregation according to frequency level.

Temporal aggregation is common in many fields, such as finance, economics, and environmental monitoring. Data is often collected at high frequencies but may need to be analyzed or reported at lower frequencies. However, it is important to be aware that temporal aggregation can also result in loss of information and may affect the accuracy of statistical models or forecasts [197].

Let X be a time series taken at equidistant time intervals, for example, the `YGT`. One may often be interested in studying not monthly (which might be interfered with by season effects) but every year, which are temporal aggregations from monthly data [263]. Taking x_i as observations of X, a non-overlapping aggregated m-period time series Y_m is defined in Equation 2.7, where y_j are observations of Y_m is the aggregated time series and m corresponds to the m-period aggregates of X, *i.e.*, the order of aggregation. Thus, in our previous example, t represents the time unit of the month, and m is 12 [279].

$$y_j = \sum_{t=m(j-1)+1}^{m \cdot j} \frac{x_t}{m} \tag{2.7}$$

Figure 2.5a presents MGT, whereas Figure 2.5b presents the YGT. As observed, YGT preserves the average behavior of MGT each year, removing seasonal components associated with monthly values.

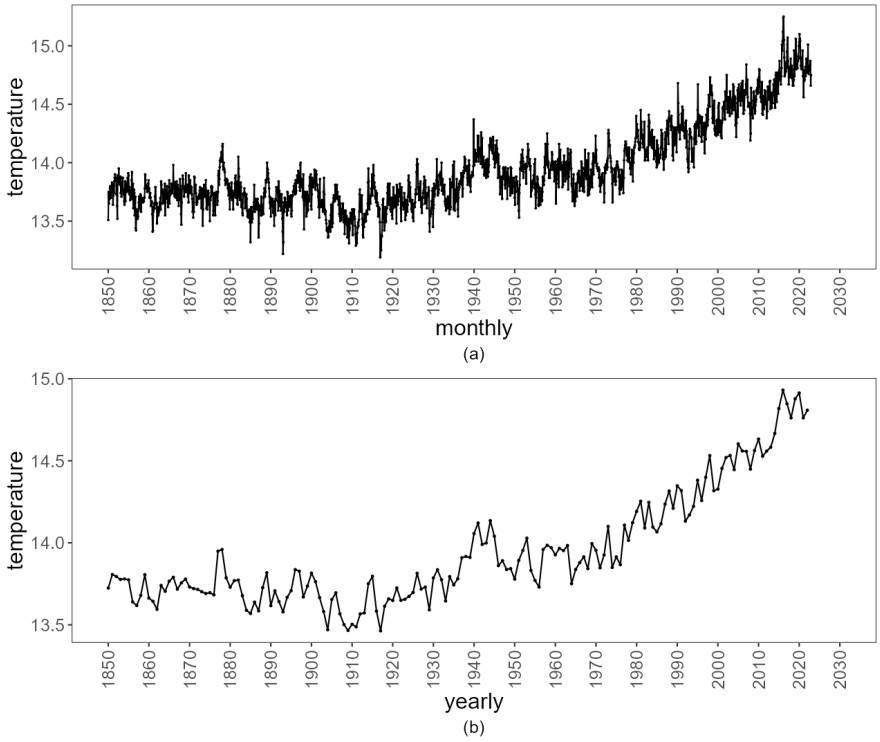


Fig. 2.5 Temporal aggregation: MGT (a); YGT computed from MGT (b).

2.3.2 Trend component

The Linear regression (LR) is the simplest and most rigid method for extracting trend components from a time series. It can be described by Equation 2.8, where $\hat{x}_t$ is the trend at time t. The α and β are obtained from mean square error adjustment, and ω_t is the residual from this LR adjustment [114, 135]. Figure 2.6a presents an example of LR applied to YGT.

$$\hat{x}_t = \alpha t + \beta + \omega_t \tag{2.8}$$

The Moving average smoother (MAS) is widely used, especially in finance and econometrics. It is useful for highlighting seasonality and long-term trends in a time series [201, 244]. MAS can detect the evolving behavior of a time series by minimizing random noise [201] and can also be used for seasonal adjustment of a time series. One of the simplest forms of the MAS is represented by Equation 2.9, where k is the order of the moving average. Figure 2.6b presents an example of MAS applied to YGT.

$$\hat{x}_t = \frac{1}{k} \sum_{i=t}^{t+k-1} x_i, \quad 1 \leq t \leq n-k+1 \tag{2.9}$$

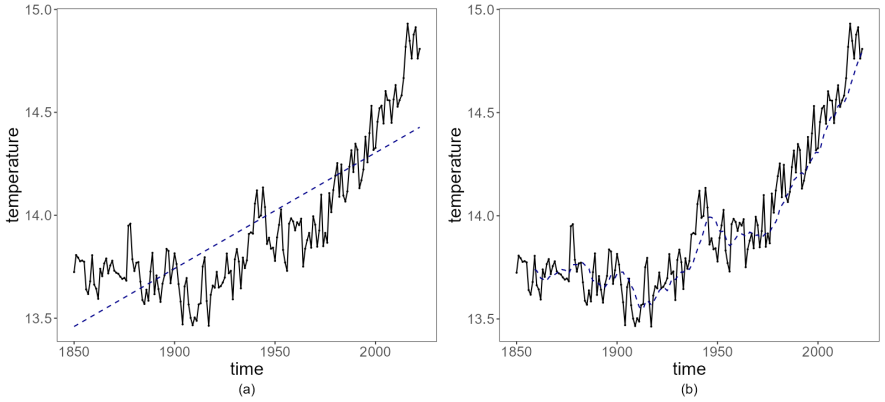


Fig. 2.6 Extracting trend components from YGT: LR (a); MAS (b).

More advanced techniques for extracting the underlying trend components of time series exist. Some of them are discussed in Section 2.3.5. Alternatives also involve segmenting the time series when changes in the trend occur. Such aspect is discussed in Chapter 4.

2.3.3 Variance stabilization

Variance stabilization is a technique used in time series analysis to transform data with non-constant variance into data with constant variance. In many time series, the variance changes, making it difficult to analyze the data using traditional statistical methods. The most common methods for addressing variance stabilization are the Logarithmic transform (LT) and Box-Cox transform (BCT) [76, 244].

The LT tends to suppress fluctuations that occur over portions of a time series which present higher values [244], and its simple formula can be seen in Equation

2.10, where $\hat{x}_t$ is the transformed version of the original time series value x_t. Often in macroeconomics, a time series is a natural log-transformed (setting b = e) to minimize effects of non-stationarity and heteroscedasticity (non-constant variability [114]), and also to induce symmetry and normality. The estimated coefficients of a logged time series can also be interpreted as elasticity. These advantages make natural logs one of the most applied `LT`. Figure 2.7a presents an example of `LT` applied to `YGT`.

$$\hat{x}_t = \log_{\mathrm{b}} x_t \tag{2.10}$$

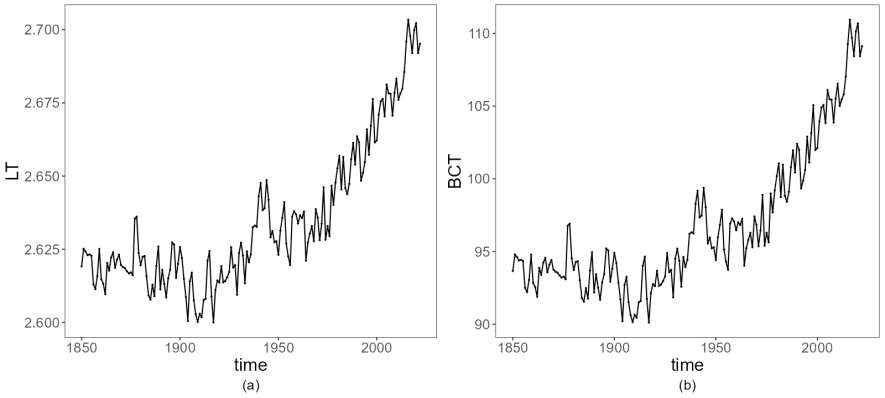


Fig. 2.7 Variance stabilization applied to `YGT`: `LT` (a); `BCT` (b)

The `BCT`, as in Equation 2.11, presents a more objective method for determining a suitable power transformation of a time series [292]. However, it demands calculating the numeric argument λ related to the data distribution function, which is not always known. Additionally, it can only be applied to positive valued data [76]. Figure 2.7b presents an example of `BCT` applied to `YGT`.

$$\hat{x}_t = \begin{cases} \left(x_t{}^{\lambda} - 1\right)/\lambda, & \lambda \neq 0, \\ \log x_t, & \lambda = 0. \end{cases} \tag{2.11}$$

2.3.4 Detrending and differencing

It is also important to mention the transformations of Detrending (`DT`) (or trend removal) and Simple differencing (`DIF`), which are widely used in combination with time series modeling techniques. The `DT` transformation involves determining and removing an inherent trend observed in a time series behavior [244]. Generally, the observed trend is estimated and defined by a deterministic functional form, which

may be represented as a fixed component in a time series model [283]. A general DT transformation can be seen in Equation 2.12, where η_t is the determined trend component and, in this case, $\hat{x}_t$ represent the variability series, that is, the residue of the time series data after trend removal. Figure 2.8a presents an example of detrending using linear regression.

$$\hat{x}_t = x_t - \eta_t \tag{2.12}$$

However, this method assumes that the deterministic trend is always appropriate. This property is frequently not the case in many applications, especially involving nonstationary processes [76]. A common approach is to use Moving average (MA) to overcome this drawback. It is a way to smooth out random fluctuations in data and identify underlying trends. Equation 2.13 computes the moving average η_t for time series X. The parameter k is the order of the average. It is the number of previous observations used to compute the MA. The parameter k introduces inertia to the MA. The higher the k, the more observations are needed to follow a change in the time series trend. The first $k-1$ points for η_t are not defined due to the absence of observations to compute the MA. An example of MA for the YGT is presented in Figure 2.8b.

$$\eta_t = \frac{\sum_{i=0}^{k-1} x_{t-i}}{k} \tag{2.13}$$

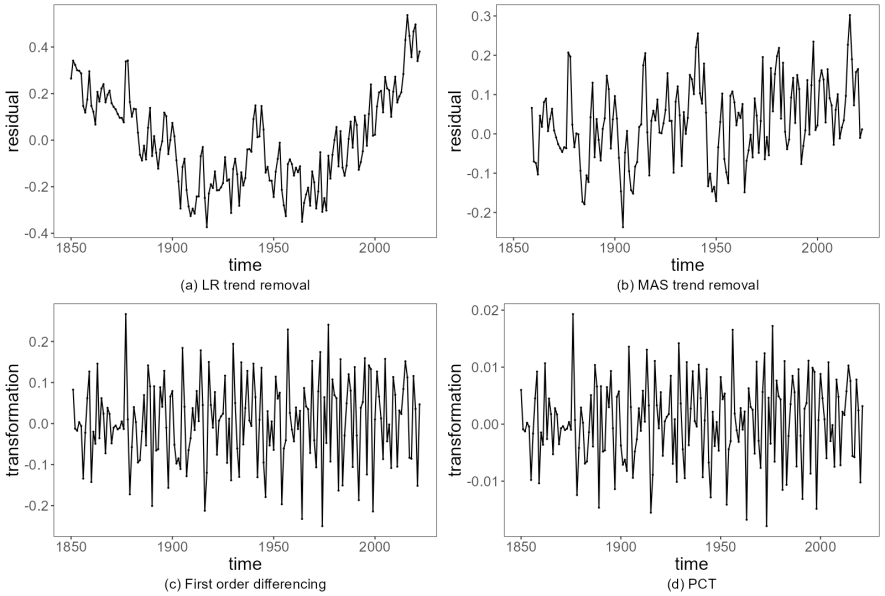


Fig. 2.8 Detrending versus differencing: DT of YGT using LR (a); DT of YGT using MAS (b); DIF of YGT (c); PCT of YGT (d).

The DIF method brings some advantages compared to DT since it does not demand a parameter estimation process and is capable of generating a stationary time series when it presents stationary behavior around a deterministic or stochastic trend [244]. A first DIF transformation, written as in Equation 2.14, can eliminate a linear trend, a second DIF eliminates a quadratic trend, and so on. For DIFs of higher-order, the backshift operator (B) is used, as seen in Equation 2.15, where ∇^d is the d-th differencing and the operator $(1 - B)^d$ is adapted for higher orders of d [244]. An example of DIF for the YGT is presented in Figure 2.8c.

$$\hat{x}_t = \nabla x_t = x_t - x_{t-1} \tag{2.14}$$

$$\nabla^d = (1 - B)^d,\ B^k x_t = x_{t-k} \tag{2.15}$$

The Percentage change transform (PCT) sometimes can be an alternative to DIF. It assumes considerable stability in the relative percentage of change between the two following observations of a time series [76]. In that case, Equation 2.16 is true if the percentage change is restricted to an interval $[-100\rho, 100\rho]$, with $0 \leq |\rho| < 1$ being a small acceptable threshold. Figure 2.8d presents an example of PCT applied to YGT.

$$\hat{x}_t \approx \log\left(\frac{x_t}{x_{t-1}}\right) \tag{2.16}$$

2.3.5 Decomposition

Most commonly, decomposition-based transformations originate in signal processing techniques such as spectral (frequency domain) analysis [173]. They decompose a time series into components (signals) having different scales (frequencies), which different physical phenomena may cause, to capture the intrinsic dynamics of a time series [156, 189, 288]. For example, a time series can be decomposed into short-term (high-frequency), seasonal, and long-term (low-frequency) components. An advantage of this decomposition process is that explaining only a few signal components is generally simpler and more physically meaningful than a collection of estimated model parameters [244]. Moreover, the derived components are more easily modeled, which can simplify the time series prediction problem [89]. Usually, decomposition may be achieved in a frequency-only or a time-frequency domain. The latter enables the preservation of information regarding localized changes. Special cases of time series decomposition are based on moving average iterations or pattern mapping (i.e., deriving patterns of a time series behavior to simplify its prediction and filter trends and long-term variations [89]). Some of the most frequently used decomposition-based transformation methods are described in the following subsections.

2.3.5.1 Frequency domain decomposition

Time series analysis in the frequency domain is often based on Fourier transform (FT) [138]. The FT creates a frequency-based representation (a spectrum) of a time series in terms of Fourier basis functions. The FT of a time series X, in this case, represented as a function of time x_t, can be formulated as in Equation 2.17, where $F(\xi)$ represents the Fourier spectrum, ξ is a frequency component, and j is the imaginary number $j = \sqrt{-1}$ [173].

$$F(\xi) = \int_{-\infty}^{+\infty} x_t e^{-j\xi t} dt, \quad e^{-j\xi t} = \cos \xi t - j \sin \xi t \tag{2.17}$$

Besides, it is common to apply Fast Fourier transform (FFT) for time series, which is an algorithm that computes the DFT of a sequence. FFT is much faster than computing the Discrete Fourier transform (DFT) using the standard FT method [75]. It is particularly useful in time series analysis, where it can be used to identify periodic patterns in the data. While using FFT, the time series is decomposed into harmonics. An example of decomposition using FFT is depicted in Figure 2.9. The decomposition is applied to YGT.

In Figure 2.9a, it is possible to observe that the YGT contains 90 harmonics. Combined harmonics can recover the entire YGT with imperceptible error. However, it is interesting to use a small number of harmonics to capture the main time series properties. In this case, a common approach is to use periodogram [49]. Common approaches include establishing a frequency threshold, for example, by visually inspecting the periodogram and identifying the frequency values where the spectral density drops off sharply or appears to level off. Another approach is to evaluate the appropriate number of harmonics by model selection criteria, such as Akaike information criterion (AIC) or Bayesian information criterion (BIC) [130, 244]. Figure 2.9b presents the periodogram for YGT. A frequency threshold is set at 0.06. It led to the adoption of the major nine harmonics.

These nine harmonics can capture major trends and seasonal components of YGT as depicted in Figure 2.9c. However, this reduced number of harmonics cannot fit the YGT in the borders. In general, when using a small number of harmonics, the FFT provides a better approximation of the time series in the middle of the series. Still, the approximation is worse near the beginning and end of the series. A common practice to avoid these border effects is applying a window function to the time series before performing the FFT. This window function reduces the weight of observations near the beginning and end of the series, which can improve the accuracy of the Fourier transform approximation. Another approach is to avoid the initial and final observations [46, 49, 244]. Dashed lines in 2.9c present that FFT is better adjusted after the first and before the last 30 observations. Finally, Figure 2.9d presents the residual of YGT and the FFT with nine harmonics. Again, the residuals are more significant at the beginning and end of the series.

A windowed variation of FT, namely the Windowed Fourier transform (WFT), was devised to minimize these border limitations. The WFT is suitable to analyze nonstationary time series with slowly changing behavior. Also, derivations of WFT are

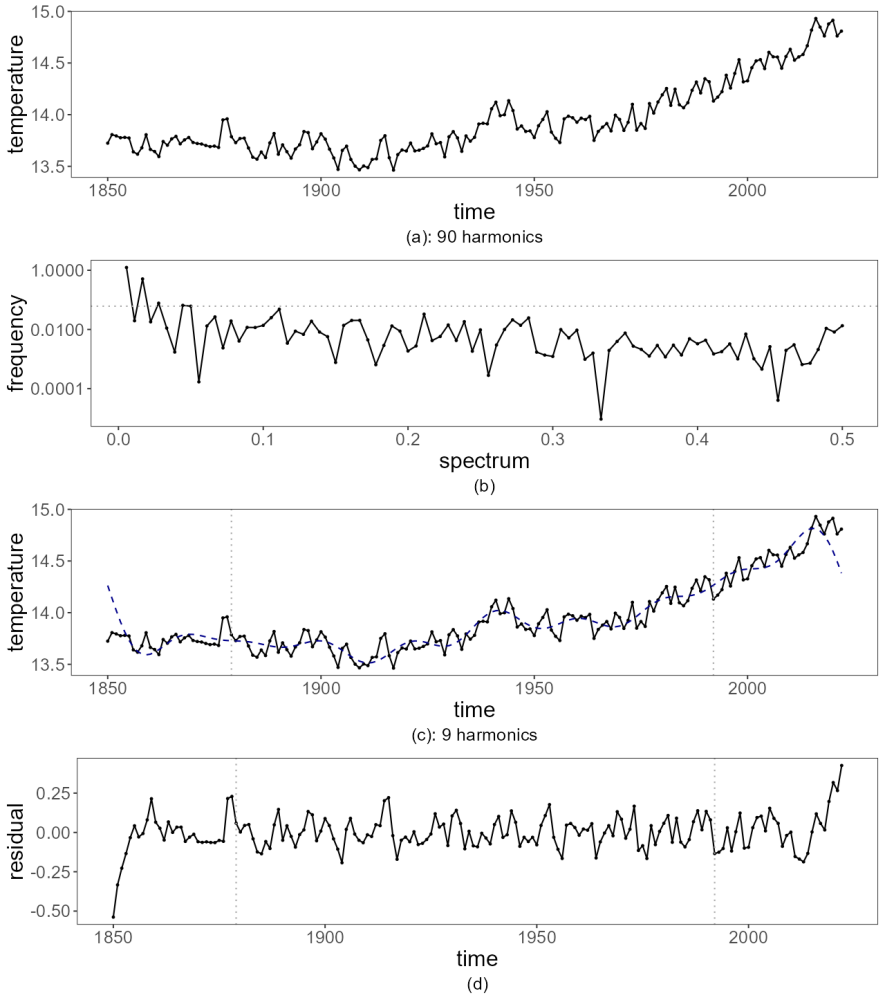


Fig. 2.9 The decomposition of `YGT` using `FFT`: FFT with all 90 harmonics (a); periodogram of `YGT` filtered frequency at 0.06 leading to 9 harmonics (b); FFT with all nine harmonics (c); Residual of `YGT` for FFT with nine harmonics (d).

possible, such as the Kolmogorov-Zurbenko Fourier transform (`KZFT`), which applies the `WFT` with a unique short-time window, being capable of providing the best possible resolution in the frequency domain [288]. Nonetheless, `WFT` and its derivations may still generate time ambiguity due to the unlimited support of the Fourier wave bases [173]. Recently, a method capable of efficiently analyzing localized changes in time series has been widely applied, based on finite wavelet bases [173].

2.3.5.2 Time-frequency decomposition

Wavelets are finite basis functions localized in both time and frequency. The Wavelet transform (WT) decomposes a time series (signal) by correlating it with a family of wavelets, providing an extremely flexible time-frequency representation [156, 173]. The WT decomposes a time series X, which is again regarded as a function of time x_t, into the wavelet series $\hat{x}_t$ observed in Equation 2.18 [138]. Here, the components ζ_t and its coefficient b represent the scale part of the wavelet series (responsible for modeling trends and seasonality). In contrast, the components ψ_t and its coefficient c represent the detail part of the wavelet series (corresponding to noise or random deviations) at scale (decomposition level) l and position k. Also, L is the defined maximum decomposition level.

$$\hat{x}_t = \sum_{k=1}^{n} b_{l,k}\zeta_{l,k_t} + \sum_{l=1}^{L}\sum_{k=1}^{n} c_{l,k}\psi_{l,k_t} \tag{2.18}$$

The WT, in Multiresolution analysis (MRA), can generally be implemented as a Continuous wavelet transform (CWT) or a Discrete wavelet transform (DWT) [23, 73]. The DWT uses a filter to decompose a signal into different frequency components. The DWT works by convolving the signal with two sets of filters: a Low-pass filter (LPF) and a High-pass filter (HPF). The LPF and HPF filters are typically designed using the scaling and wavelet functions of a given wavelet family [213].

Figure 2.10a presents the DWT for the YGT. It uses the Haar filter (HAAR). It leads to three components of ζ_t and three components of $\psi_(t)$. As it can be observed using all components, the DWT adjusts the entire time series. Figures 2.10b and 2.10c respectively presents the $\zeta_(t)$ and $\psi_(t)$ terms. Similarly to DFT, the adjustment is better in the middle of the time series.

Since finite wavelets are irregular and asymmetric [138], the WT is particularly suited for analysis of nonlinear, noisy, and nonstationary time series, with rapidly changing behavior [23, 156, 173]. If the wavelet transform is applied to a nonstationary time series, the resulting decomposed series usually presents a better behavior than the original series, being more easily and accurately predicted, even by simple linear models like ARIMA. Predictions of the original time series with increased accuracy can be obtained by applying the inverse WT to the predictions of the decomposed series [73, 138]. WT is also a tool for MRA, that is, modeling frequency behavior of a time series at multiple time resolutions, which is an improvement over WFT [173]. The MRA often makes WT a more appealing option than standard econometric models [156]. We also note the derived method of WPT, which reapplies the WT to a resulting decomposed series that does not present a required level of stationarity or decorrelation [30].

Besides the WT, there are other MRA techniques that are very recent and prominent, namely the Empirical mode decomposition (EMD) [98, 128] and the derived VMD [156]. Similarly to the WT, the EMD is an algorithm for nonlinear and nonstationary time series decomposition, generating a time-frequency representation of the series [156, 272]. The basic principles of EMD described as follows [98]:

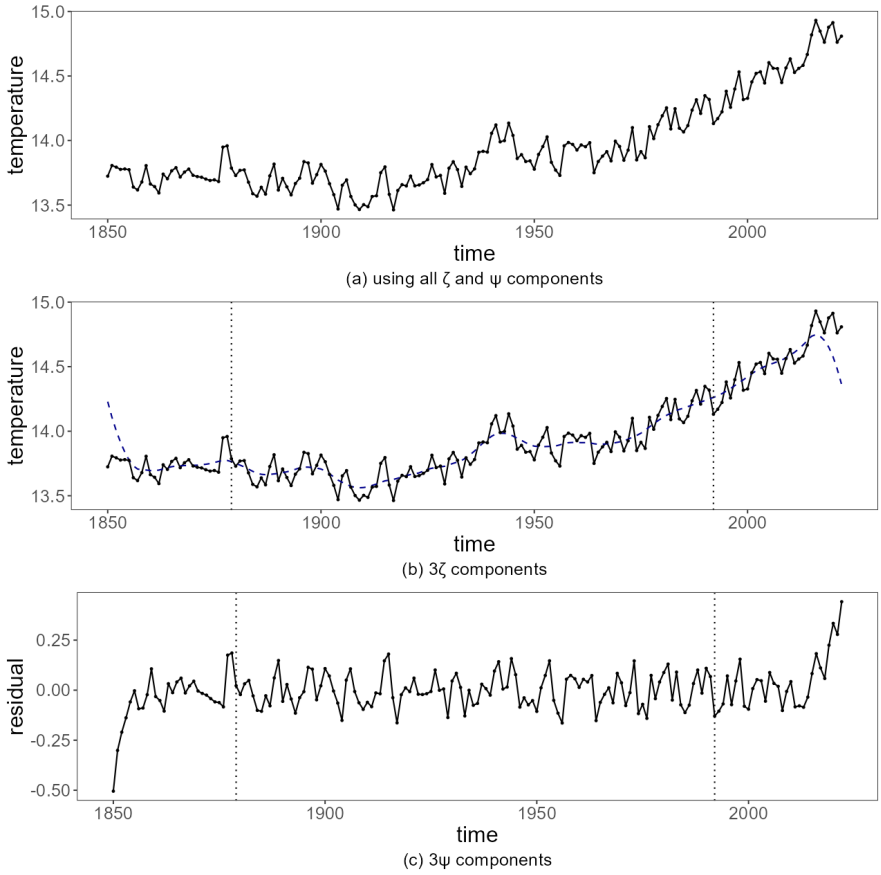


Fig. 2.10 Decomposition of `YGT` using `DWT` with `HAAR` filter: WT with all three ζ_t and three ψ_t components (a); `DWT` with only the three ζ_t components (b); `DWT` with only the three ψ_t components (c).

1. Identify all extremes of a time series X
2. Interpolate between minima (resp. maxima), ending up with two *envelopes*: $emin(X)$ and $emax_t$.
3. Compute the average time series $m(X) = \frac{emin(X)+emax(X)}{2}$.
4. Extract the detail $d(X) = x(X) - m(X)$.
5. Iterate on the residual $m(X)$.

`EMD` is also capable of decomposing a time series. When the detail time series $d(X)$ has zero mean, it can be considered an Intrinsic mode function (`IMF`). The `IMF` are more stable components that can be more easily modeled [252]. The last residual is considered a major trend component of the time series. Nevertheless, some of the last `IMF` identified in the `EMD` can also be used to model the trend components. The first `IMF` is associated with the residual components.

Figure 2.11 presents the EMD for the YGT. As it can be observed in Figure 2.11a, when using all seven EMD components, the YGT is well-adjusted. Figure 2.11b uses the last IMF component with the residual. It captures the trend of YGT. Conversely, 2.11c presents the first five IMF, which stands for the residual of the YGT.

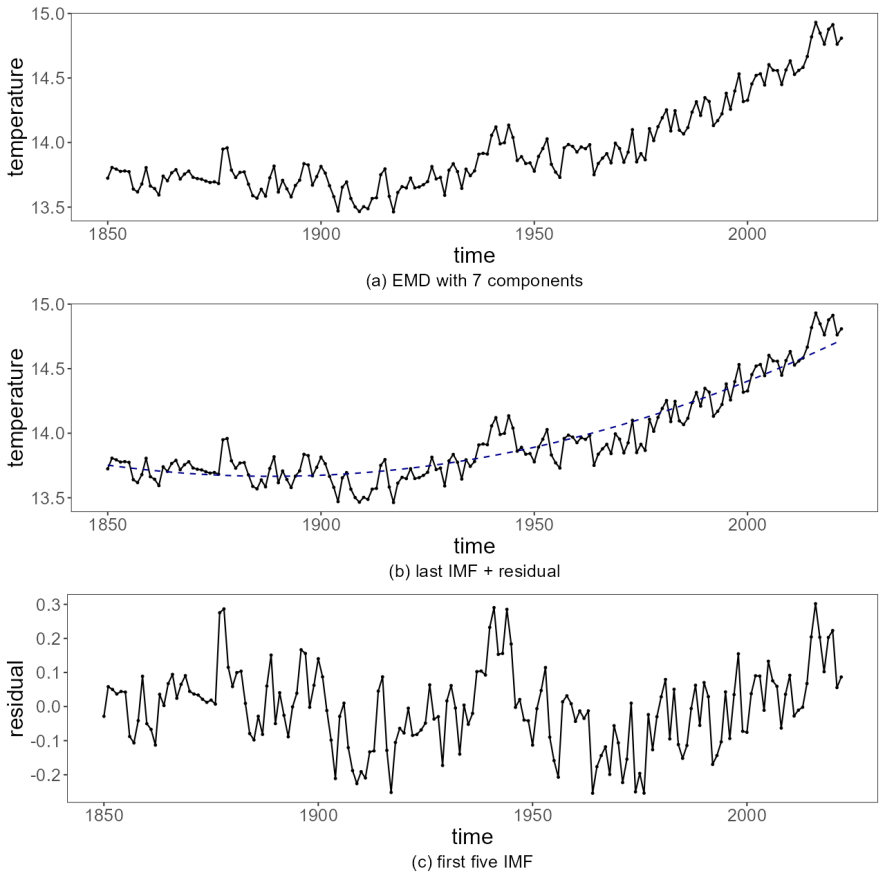


Fig. 2.11 Decomposition of YGT using EMD: EMD with all seven components (a); EMD only the last IMF and residual components (b); EMD only the first five IMF (c).

An advantage of EMD is that it does not depend on predetermined (wavelet) functions since its basis functions are derived directly from the time series, therefore being adaptive and completely data-driven [156, 272]. Conversely, EMD is recursive and does not perform well when separating time series components with similar frequencies. In order to overcome this limitation, the Variational mode decomposition (VMD) algorithm was devised, which transforms the time series decomposition into a non-recursive and variational model [156, 252]. The VMD is superior to EMD in the

sense that it decomposes a time series more precisely and is more robust to noise [252].

Another recent noteworthy approach is presented by the Hilbert-Huang transform (HHT), which combines the use of EMD with the application of the Hilbert transform for each of the resulting IMF [248]. The Hilbert transform is used for computing instantaneous frequencies, enabling a frequency-time-energy or amplitude representation of a time series. The HHT can accurately represent local time and frequency characteristics of nonlinear and nonstationary time series [248].

2.3.6 Sliding windows

Let a time series X be a sequence of n observations $< x_1, x_2, x_3, \ldots, x_n >$, where x_1 represents the value assumed by the series at the first (oldest) time point and x_n represents the value of the series at the newest time point [93, 244]. In this case, n is the length of a time series X and is represented as $|X|$. A specific time series observation is referenced as x_t indexed in time.

A subsequence of size p obtained from X that ends at the i position can be represented by $seq_{i,p}(X)$, which is a continuous sequence of values $< x_{i-(p-1)}, x_{i-(p-2)}, \ldots, x_i >$, where $|seq_{i,p}(X)| = p$ and $p \leq i \leq |X|$. Subsequences enable the analysis of data samples to evaluate local properties [69].

A Sliding window (SW) is an approach to explore all possible subsequences of a time series [144, 158]. A SW of size p for a time series X is a function $sw_p(X)$ that produces a matrix W of size $(|X| - p + 1)$ by p. Each line w_i in W is the i-th subsequence of size p from X. Given $W = sw_p(X), \forall w_i \in W, w_i = seq_{i,p}(X)$ [43].

Figure 2.12 presents the YGT from 2000 until 2023. It presents both the time series and its colored representation. Each rectangle corresponds to an observation. The colors are associated with the values. Lighter colors correspond to lower temperatures (close to 14.25), while darker colors correspond to higher temperatures (close to 15). Figure 2.12 also depicts a colored example of SW of size 3. Each triple of t, $t-1$, and $t-2$ corresponds to a separate sliding window with three rectangles aligned vertically. It starts in the third observation with its first complete SW. Figure 2.12 also presents a table with observations of the first five SW.

The SW is widely used in time series analysis [202]. It can support memory management of the time series [175, 278], *i.e.*, the ability to forget past data. It can also be used for support Machine learning methods (MLM).

2.3.7 Data normalization

Data normalization transforms data into a common scale to make it more consistent and easier to compare. Regarding time series data, normalization is particularly important because the values in the series may vary widely over time. These normal-

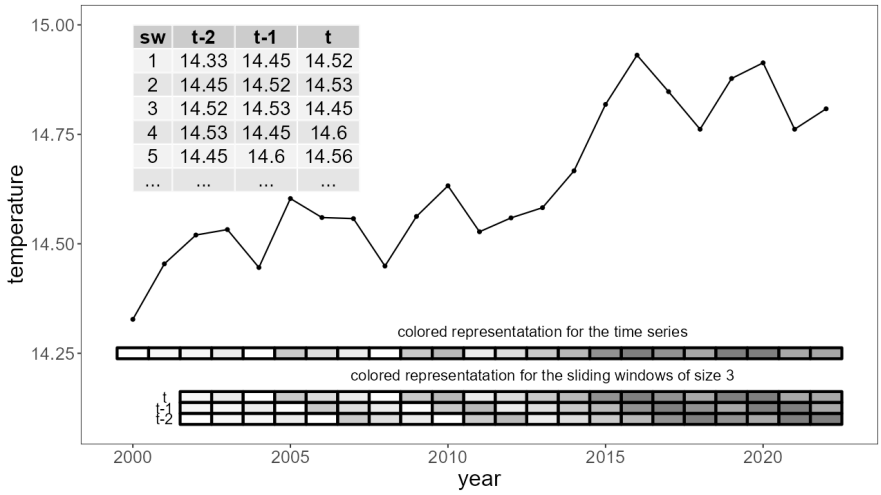


Fig. 2.12 Sub-sequences and SW of time series (adapted from Borges et al. [43])

ization techniques can be applied to the entire time series or sequences of the series. Normalization can reduce the impact of outliers, make it easier to compare trends over time, and ease the use of certain analysis techniques, such as machine learning algorithms [118, 202, 281].

The most common normalization methods used during data transformation include Min-max normalization (`Min-Max`), Standard score normalization (`Z-Score`), and decimal scaling. The `Min-Max` method normalizes the values of a time series X according to its minimum and maximum values. It converts a value x_i of X to $\hat{x_i}$ in the range $[low, high]$ as presented in Equation 2.19. It is common to set low equal to 0 and $high$ equal to 1.

$$\hat{x_i} = (high - low) \cdot \frac{x_i - min(X)}{max(X) - min(X)} + low \tag{2.19}$$

The decimal scaling normalization moves the decimal point of the values of a time series X according to its maximum absolute value. Hence, a value a of x_i is normalized to $\hat{x_i}$ as described in Equation 2.20, where d is the smallest integer such that $max(|\hat{x_i}|) < 1$.

$$\hat{x_i} = \frac{x_i}{10^d} \tag{2.20}$$

In the `Z-Score` normalization, the values of a time series X are normalized according to their mean ($\mu(X)$) and standard deviation ($\sigma(X)$). A value x_i of X is normalized to $\hat{x_i}$ as described in Equation 2.21.

$$\hat{x_i} = \frac{x_i - \mu(X)}{\sigma(X)} \tag{2.21}$$

These methods are useful in stationary environments but might be limited while facing non-stationarity. They might become effective if differentiation is applied before normalization. Besides, a traditional approach commonly used is to execute data normalization on top of SW [121, 201]. The basic idea of this approach is that, instead of considering the complete time series for normalization, it normalizes each window, considering only its statistical properties. The rationale behind this approach is that decisions are usually based on recent data. The sliding window technique can always normalize data in the desired range. However, it has the drawback of assuming that the time series volatility is uniform, which is not true in many phenomena [13, 114, 267].

To overcome this limitation of SW, Adaptive normalization (AN) was proposed to address non-stationarity, especially to support machine learning methods [201]. The general process of AN encompasses the following steps:

1. Compute the moving average for each SW;
2. Compute the residual of each SW with respect to its moving average;
3. Compute the distribution of residuals for all SW, and remove all outliers according to box-plot criteria [118];
4. Normalize all SW using this Min-Max extracted from cleaned distribution of residuals;

Figure 2.13 presents the effects of using different normalization methods. Figure 2.13a presents the YGT marking a subsequence of size five between 2008 and 2012. Figure 2.13b presents the normalization of that sequence using global Min-Max. Since observations are close to the highest values of YGT, the normalization values are close to 1. The main problem with using the global Min-Max normalization in time series prediction is that the minimum and maximum values of the out-of-sample data set might lead to values higher than one and lower than zero, which might not be supported by machine learning models [201].

Figure 2.13c presents the effect of using differentiation before data normalization. The limitation of global Min-Max is not a problem for this technique. However, the differentiated time series can differ considerably from the original one. In this example, values decrease due to a decrease in the growth rate, even though the time series increases.

Figure 2.13d presents the Min-Max applied to each sliding window. All sliding windows distribute values between 0 and 1. The drawback is that it might stretch values with small changes in the same way as values with big changes. It might not be easy to build machine learning models. Figure 2.13e presents the AN. It first applies a differentiation concerning the moving average. After that, the Min-Max normalization is done using the minimum and maximum values for all SW. Due to these actions, AN can preserve the original time series shape and spread values between 0 and 1, considering the ratio of values changes inside the SW.

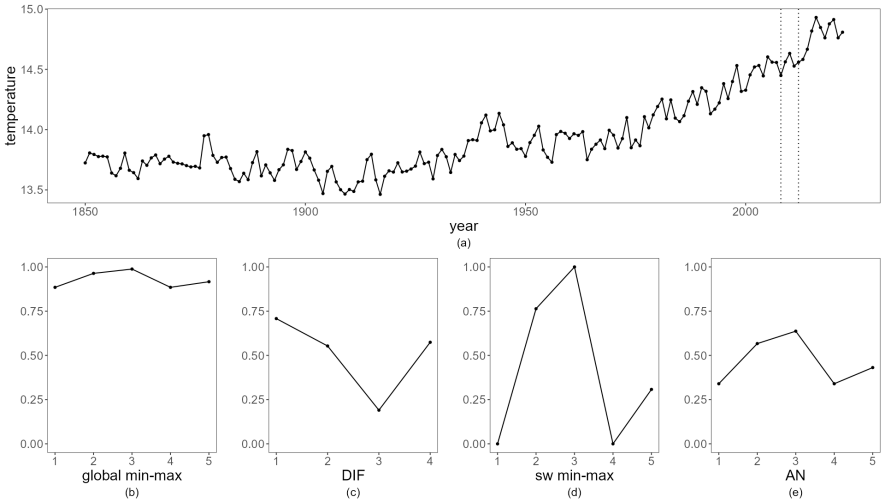


Fig. 2.13 Time series normalization

2.3.8 Data splitting

Splitting the data is an essential step in time series analysis and modeling. It involves dividing the available time series data into sets for training, validation, and testing purposes. The main reason for splitting the data is to evaluate the accuracy of the model and prevent overfitting. Overfitting occurs when the model is too complex and captures noise or random fluctuations in the training data, leading to a poor generalization of new observations [118, 130, 234].

The most common approach to splitting the data is to use `SW` technique [130]. It involves selecting a fixed interval for training the model and then using the trained model to predict subsequent observations. The process is repeated for the entire time series, with the `SW` forward by a fixed interval each time.

In addition to the training data, we also need to set aside some data for validation and testing. The validation data is used to tune the hyperparameters of the model, such as the number of lagged values to include. In contrast, the testing data is used to evaluate the final performance of the model. A common splitting ratio is $\frac{2}{3}$ for training and $\frac{1}{3}$ for testing. The optimal ratio may vary depending on the size and complexity of the time series data [118].

However, while training and validating or training and testing, a distinct difference between traditional cross-validation techniques and time series cross-validation is worth mentioning. Time series cross-validation is a technique for evaluating the performance of a time series model by testing it on a sequence of data that is distinct from the training data. It is a variation of the traditional cross-validation method commonly used in machine learning but designed specifically for time series data. In time series cross-validation, the data is divided into consecutive sequences. Each consecutive sequence is used for evaluation (validation or testing).

The remaining prior sequences are used as the training set. This process is repeated for each sequence, and the results are averaged to estimate the performance of the model. So, the key difference between time series cross-validation and traditional cross-validation is that the sequences are selected based on their temporal ordering rather than randomly. It occurs since time series data has a temporal structure, where each observation depends on previous observations, and random sampling can introduce bias and unrealistic assumptions [130].

Figure 2.14a presents the YGT from 2000 until 2022. It separates 2000 until 2018 for training. Then, the last four observations are separated for testing. Again, to simplify the visualization, we present the colored representation for the time series. As shown in Figure 2.14b, a one-step-ahead prediction uses the model built from the training set to predict the following observation. The one-step-ahead prediction with time series cross-validation uses the model built during the training set to predict the first observation. The second prediction uses the built model sliding one observation as if the first testing observation was known to predict the second. This process is repeated for all testing observations. The average error of prediction for all testing observations measures the Prediction performance (PP).

Figure 2.14c presents a three-step-ahead prediction that uses the model built from the training set to predict the following three observations simultaneously. The average prediction error for the three predictions measures the PP. Besides, the three-step-ahead prediction with time series cross-validation uses the model built during the training set to predict the first three observations. The second prediction uses the built model sliding one observation as if the first testing observation was known to predict the following three observations. This process is repeated until all testing observations are explored. The average error of all predictions (six in this example) measures the PP [130, 234].

Finally, consider the scenario where the time series model is built using SW. They correspond to the typical scenario of MLM. Figure 2.15 depicts the colored series converted to SW. The SW is size three and contains 23 sequences. The SW data is also split. The model is built using only the 17 sequences dedicated to training. In a one-step-ahead prediction, the model uses the last sequence before testing to predict the first test observation. Then, the model uses the first testing sequence to predict the second observation. This process is repeated until the last prediction is made [130, 201]. A similar process also occurs for n-step-ahead prediction using SW. In this case, the step-ahead uses predicted sequences instead of real observations.

2.4 Time series prediction

Time series prediction is an object of interest due to its increasing importance and applications in science, business, and government. The prediction context encompasses both problems of classification (prediction of discrete data) and regression (prediction of continuous data) [52, 93, 118].

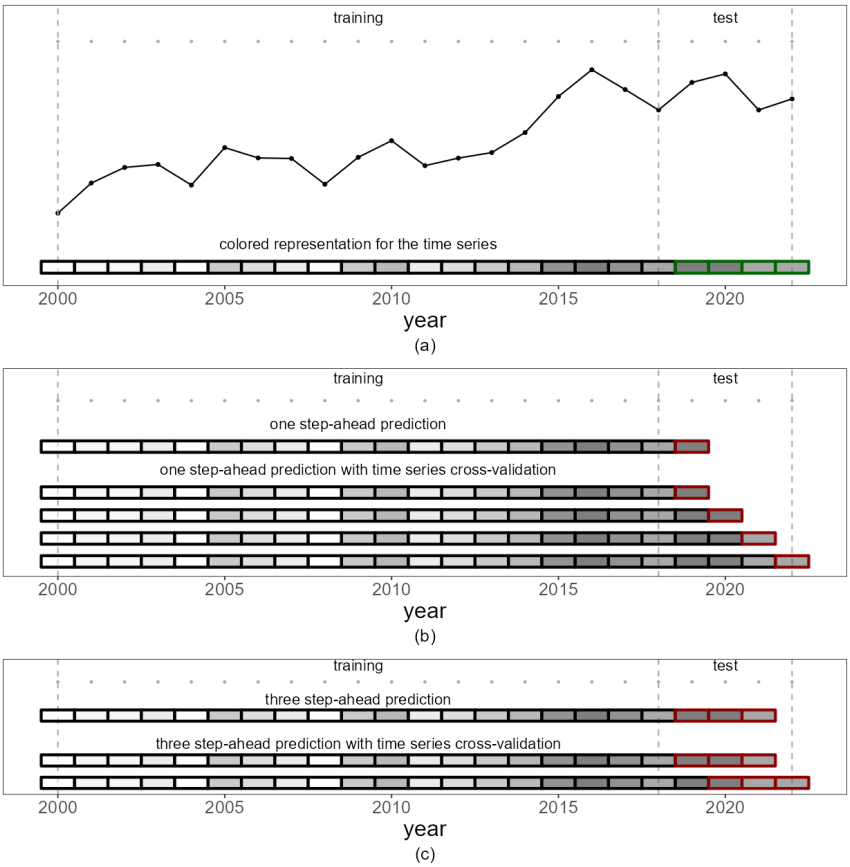


Fig. 2.14 Training and test example: splitting between training and test (a); one step-ahead prediction with time series cross-validation (b); three-step ahead with time series cross-validation (c)

This section generally focuses on the problem of prediction of time series data through regression. For simplicity, this chapter may refer to prediction and regression interchangeably. Relevant models adopted for time series prediction generally fall into the categories of statistical or machine learning models [234]. The accuracy of the predictions depends on the quality of the historical data, the appropriateness of the model, and the assumptions made about the underlying processes driving the time series [114, 130].

Figure 2.16 presents a general time series prediction process. It encompasses five main activities. It provides a general framework for predicting a time series based on a particular setup of preprocessing methods and prediction models. They are briefly described here, and some parts are detailed in the following sections.

The first activity depicted in Figure 2.16 refers to acquiring the time series and performing data preprocessing by applying time series transformation methods, such

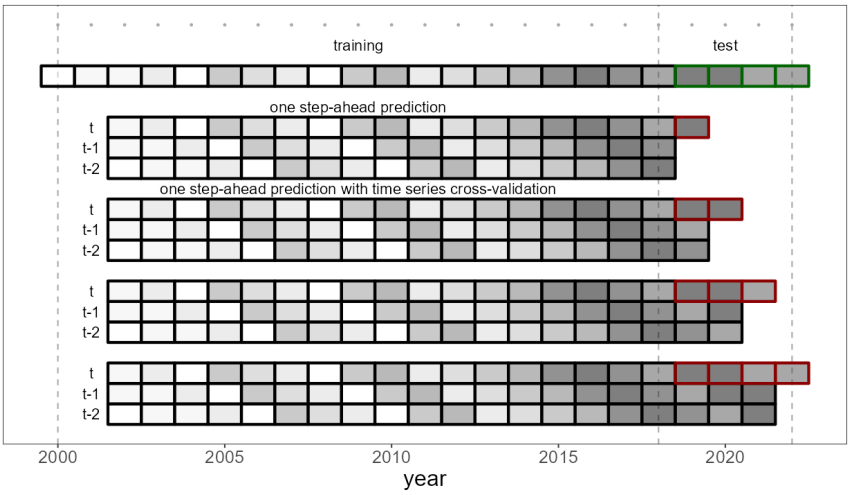


Fig. 2.15 Training test for SW of size three with one-step-ahead time series cross-validation

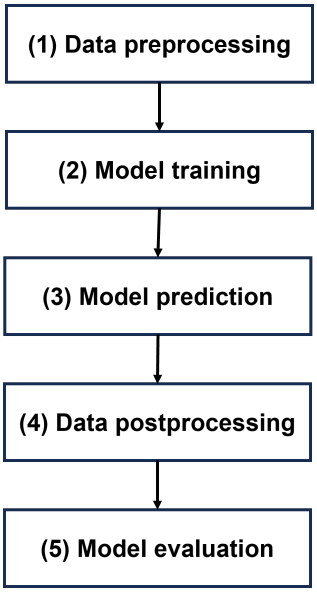


Fig. 2.16 Time series prediction process (adapted from Salles et al. [234])

as those described in the previous chapter. It also includes data normalization and SW transformation. These transformations change the time series domain values, and their parameters must be stored to support later detransformation to the original domain. For time series prediction, splitting the time series into a training and test set is also important during Activity 1. All data preprocessing parameters should be

computed during training and reapplied from the tune-values of training during the test. The model is built using the training slice and evaluated using the unseen test set. However, when the goal is to adjust a model for the time series, the model does not need to be partitioned into a training and test set.

Activity 2 addresses model training. The prediction methods very often require hyperparameter optimization. In such a case, the training slice is again split into a novel training and validation set. Alternative models exploring hyperparameter values are built using the novel training and evaluated using the validation set. Once hyperparameters are fixed, a single model is built using the entire training dataset. From this moment, the model is available for use.

Activity 3 refers to the model prediction. It is worth mentioning that the predicted values are not in the time series domain. In this sense, they can not be directly evaluated. Data postprocessing is needed in this case. Activity 4 corresponds to the postprocessing of predictions, reversing transformations applied to the time series data in Activity 1.

Finally, Activity 5 is the evaluation of prediction errors yielded by the model, as well as model fitness metrics. If the results are inadequate, this process can be revised to refine models. This entire process may be repeated if the PP needs to be improved. This process iteratively improves the quality of predictions (for time series prediction) or model adjustment (for time series modeling).

Evaluating the performance of a predictive model can be done in several ways. The most common measures are Mean square error (MSE) and Symmetric mean absolute percentage error (sMAPE). The MSE for n predictions is described by Equation 2.22. The sMAPE is an accuracy measure based on percentage (or relative) errors. For n predictions, it is described in Equation 2.23 [130], where the absolute difference between actual x_i and predicted $\hat{x}_i$ values are divided by half the sum of absolute values of the actual x_i and the predicted $\hat{x}_i$ values. Finally, R-squared (R^2), also known as the coefficient of determination, is the proportion of the variance in the dependent variable that is predictable by a model [68]. It is described in Equation 2.24.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{x}_i - x_i)^2, \tag{2.22}$$

$$\text{SMAPE} = \frac{100}{n}\sum_{i=1}^{n}\frac{2|x_i - \hat{x}_i|}{(|x_i| + |\hat{x}_i|)} \tag{2.23}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(\hat{x}_i - x_i)^2}{\sum_{i=1}^{n}(\overline{x} - x_i)^2} \tag{2.24}$$

For both MSE and sMAPE, the minimum values are 0, and the maximum is infinite (∞). The lower the values, the better the prediction. Regarding R^2, the minimum value is minus infinite ($-\infty$), and the maximum is 1. The higher the value of R^2, the better the prediction. The R^2 might be more informative than MSE and sMAPE for regression analysis [68].

2.4.1 Statistical models

The DIF method can be used together with the linear Autoregressive moving average (ARMA) models, producing one of the most important time series linear prediction models, the ARIMA models [44], which can model stochastic trends [76]. An ARIMA(p, d, q) model is composed of an Autoregressive (AR) and a Moving average (MA) modeling processes (represented by p and q, respectively) with the application of a preliminary DIF process (I) (represented by the order d) to handle non-stationarity in time series [114]. ARIMA models assume that an observation of a time series, x_t, can be described as a function of its p past values and its q past white noise values [244]. The latter is represented as ω_t, a Gaussian white noise series with mean zero and variance σ^2_ω. Let $\theta(B)$ and $\phi(B)$ be, respectively, the AR and the MA operators. The ARIMA model is denoted in Equation 2.25 [44, 244]. Variations of the ARIMA models may arise by introducing AR and MA polynomials that identify dependence on some underlying seasonal lag, generating the Seasonal autoregressive integrated moving average (SARIMA) models [244].

$$\theta(B)(1-B)^d x_t = \phi(B)\omega_t. \tag{2.25}$$

Figure 2.17 presents an ARIMA model used for both prediction (Figure 2.17a) and for model adjustment (Figure Figure 2.17b). The time series is split into a training and test set in the former. The test size equals four. The ARIMA model is adjusted using the training size. It leads to an ARIMA(1, 1, 3). The adjusted model is plotted as blue dashed lines. The real observations are plotted as black dots. The last four observations are plotted as red dashed lines corresponding to the four-step-ahead prediction horizon (Figure 2.17a). The latter uses the entire time series to build an ARIMA model (Figure 2.17b).

Statistical models are designed for modeling a time series by describing it as statistical terms. Some terms directly model behavior, such as an inherent trend or seasonality. These models may also be combined with transformations such as detrending or differencing to achieve stationarity. Examples of models falling into this category are set by the Structural model (SM) [71, 244], Linear mixed model (LMM) [135], Holt-Winter's exponential smoothing (HW) [89], Taylor's double seasonal Holt-Winter (THW), Exponential smoothing state space model (ETS), Theta forecasting (TF), and the Generalized autoregressive conditional heteroscedasticity model (GARCH).

Firstly, the SM is a representative model in this category. SM consists of terms such as trend and seasonal. These terms may provide a straightforward interpretation of a time series behavior. A general SM is presented in Equation 2.26, where η_t is the trend term, χ_t is the cycling term, and ω_t is the error term. Trend terms may be stochastic and can be estimated using an ARIMA process.

$$x_t = \eta_t + \chi_t + \omega_t \tag{2.26}$$

SM models may also be considered special cases of state space models and can be represented as so [71]. Their parameters can, therefore, be estimated by a Kalman

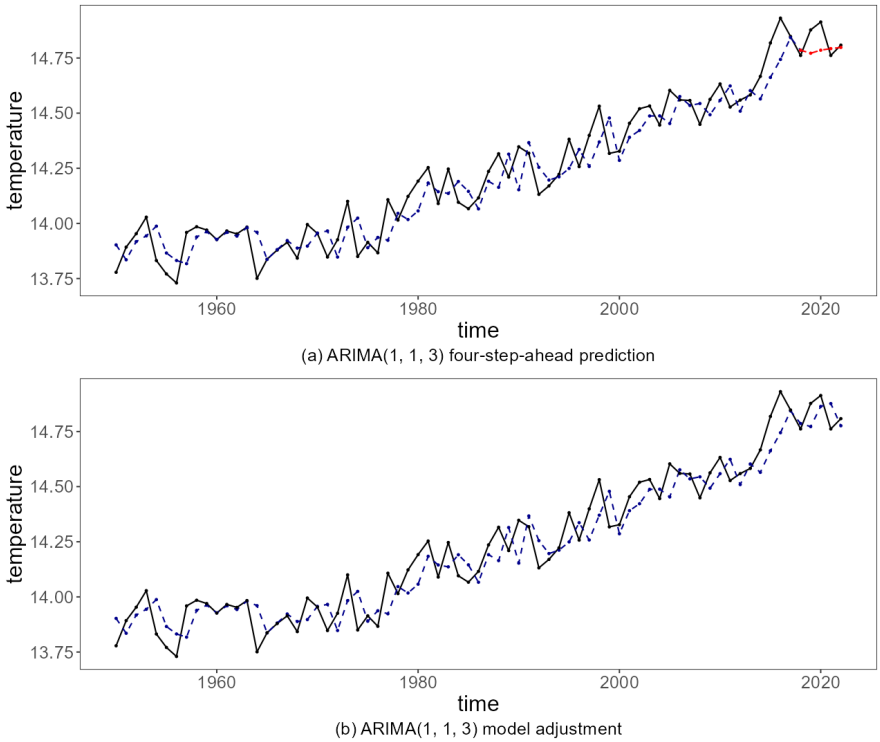


Fig. 2.17 `ARIMA` prediction model for the training slice of `YGT` (a); `ARIMA` model for the entire `YGT` (b)

filter [71, 288]. A basic example of a state space model is given in Equation 2.27, which consists of two parts: a so-called state equation (Equation 2.27a) and an observation equation (Equation 2.27b). The latter is added since $\dot{x}_t$ (considered a state vector) is assumed not directly observable. Instead, one can only observe x_t, a linear transformation of $\dot{x}_t$ with the addition of noise. Here A_t is a measurement matrix, ϵ is a coefficient matrix, and ω_t and υ_t represent noise [244].

$$\dot{x}_t = \epsilon \dot{x}_{t-1} + \omega_t, \tag{2.27a}$$

$$x_t = A_t \dot{x}_t + \upsilon_t \tag{2.27b}$$

Time series data can present dependencies and serially correlated errors due to clustering when observations can be divided into related subgroups (clusters). The `LMM` models belong to the category of mixed-effects models. They are designed to address such dependencies among observations, which would typically make standard linear models inappropriate [135]. It considers both fixed (deterministic) and random (stochastic) effects of time in a time series. In this case, fixed effects are

often modeled using a deterministic trend model, and random effects are modeled by stochastic AR or ARMA structures [288].

There are several forecasting frameworks based on the classical model of exponential smoothing. A general exponential smoothing model is expressed as in Equation 2.28, where each time series value x_t, here obtained by the function of time x_t, is based on the fitting function vector for the time series f_t, the coefficient vector κ_t (T is the transpose operator) and a white noise ω_t. The function f_t depends on its past values and the transition matrix M [186].

$$x_t = {\kappa_t}^T f_t + \omega_t, \quad f_t = M f_{t-1} \tag{2.28}$$

Methods based on exponential smoothing are applied in numerous applications. Among them, it is noted the HW method, which can be used for nonlinear modeling of heteroscedastic time series [89]. It estimates level, slope, and seasonal terms at each time point [89, 272]. Despite taking a long processing time to determine a few parameters, it can represent trend, seasonality, and randomness effectively [12].

Another model commonly applied in finance for estimating heteroscedastic time series is the GARCH. A time series may be explained as a GARCH(p, q) model by Equation 2.29a, where μ_t is a mean function, σ_t is the conditional standard deviation, and ω_t is a noise series. The ω_t is considered an Independent and identically distributed (IID) variable that follows a normal distribution N(0, 1) [54, 182]. The conditional variance σ_t^2 is defined by Equation 2.29b with α and β as coefficients [76]. Although GARCH models present a useful tool for treating non-constant variability in time series, they are known for not being able to capture long memory properties and highly irregular behavior [250].

$$x_t = \mu_t + \sigma_t \omega_t \tag{2.29a}$$

$$\sigma_t^2 = \alpha_0 + \sum_{j=1}^{p} \alpha_j \sigma_{t-j}^2 + \sum_{j=1}^{q} \beta_j \tilde{x}_{t-j}^2 \tag{2.29b}$$

2.4.2 Machine learning models

MLM have been used for nonlinear time series prediction in manufacturing systems, finance, health informatics, and energy grids, among other fields [66]. The models generated by MLM are universal approximators. They can approximate any continuous function to arbitrary precision. They can be beneficial to the problem of modeling time series properties.

Figure 2.18 depicts the general architecture adopted to build MLM. It takes a SW as input a set of lagged observations. The number of lagged values is a hyperparameter for MLM. Each MLM has a specific set of hyperparameters. The output is the next observation for that SW. So, it uses the present time series value and its $sw - 1$ predecessors $<x_{i-sw+1}, \cdots, x_{i-1}, x_i>$, to approximate the next value x_{i+1}. The model is built using the training set. If hyperparameters need to be adjusted, candidate MLM

is built by splitting the training set into a novel training and validation set. Once the hyperparameters are established, MLM can be built using the entire training set. The built model can be used later for testing.

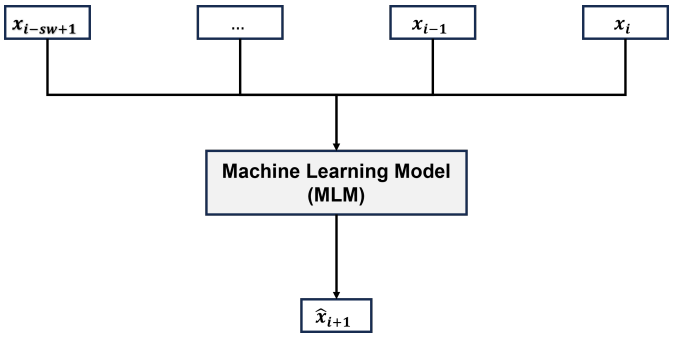


Fig. 2.18 Typical MLM for time series prediction (adapted from Ogasawara et al. [202])

Some of the most relevant MLM are the Neural network (NNET) [121], Multilayer perceptron network (MLP) [177], Extreme learning machines network (ELM) [127], Radial basis function network (RBF) [121], Support vector machine (SVM) [66], Random forest regression (RFR) [48], and the deep learning models Convolutional neural network for time series (Conv1D) and Long short-term memory neural network (LSTM) [163].

A NNET is a bioinspired computational approach for recognizing structural data patterns through neurons connected through synapses. The synapses have associated weights representing the relevance of the connection [121]. Usually, NNET presents a feed-forward architecture [121]. During a training process, approximation errors are backpropagated in order to adjust synaptic weights. A practical time series prediction process would involve setting neural network parameters, such as the number of input entries, hidden layers, and neurons in hidden layers [202]. NNET with error backpropagation has been employed for nonlinear time series prediction, outperforming traditional statistical methods such as ARIMA in functional approximation. Perhaps a disadvantage of NNET is that it still assumes that the dynamics underlying a time series are time-invariant [66]. A fully connected NNET is referenced as MLP, and it is probably the most common network architecture currently in use [177].

The slow learning speed of networks such as NNET and MLP has been a bottleneck in their applications. It is due to slow gradient-based learning algorithms and iterative tuning of network parameters [127]. Unlike traditional implementations, the ELM network adopts a learning algorithm that randomly chooses hidden neuron nodes and analytically determines output weights. Thus, there is no need for any iterative tuning or setting of parameters like learning rate, momentum, or epochs, making learning time very fast [127].

An RBF network is similar to a MLP. However, it performs a linear combination of basis functions radially symmetric around a center [121]. Several types of RBF

networks were able to capture different forms of trends and volatility in time series [66].

The SVM can recognize patterns in both linear and nonlinear data. Usually, in a regression supported by SVM, a linear learning machine approximates a nonlinear function in a kernel-induced feature space. The capacity of the system is controlled by a parameter that does not depend on the dimensionality of the space [121]. SVM can have higher PP than NNET and employ fewer parameters for chaotic time series prediction [66].

RFR is based on the combination of decision tree classifiers, or the ensemble of base models, that acts as a "forest". After the formation of the forest, the model may combine the predictions of each tree additively or by average. The results are returned as the estimated time series prediction values [48]. The generalization error for a forest converges if there is a sufficiently large number of trees, which makes over-fitting not a problem.

The idea behind deep learning is to discover multiple levels of representation, expecting high-level resources may represent a more abstract semantics of the data [115]. A Conv1D is an architecture composed of three distinct layers: an input layer, a convolutional layer, and a pooling layer, which reduces the size of the input data. The convolution layers in the datasets can be applied as an extractor of characteristics implicit in the data. The Equation 2.30 presents a convolution process, where g is the input layer, h is one of the k filters that a Conv1D optimizes for an aim function during the learning process at time t, $*$ is the convolution operator, and n is a hidden layer in the neural network.

$$(g * h)[n] = \sum_{t=0}^{k} g[k-t]h[t] \tag{2.30}$$

LSTM networks have the same properties as conventional recurring networks. However, they can store information for long periods when processing a time sequence. The memory points of an LSTM network are called cells. Cells can carry information until the end of a sequence or identify information the network should forget after some processing step [110].

Figure 2.19 presents a LSTM with four input neurons model for the YGT. Figure 2.19a presents the LSTM for four-step-ahead prediction. Figure 2.19b presents the LSTM model adjustment for the entire time series.

It is important to compare MLM with baseline statistical models such as ARIMA to infer the adequacy of the prediction model. Additionally, MLM is also affected by time series preprocessing activities. Thus, during such comparison, given an input time series, values predicted by a particular setup of preprocessing methods and MLM (and their parameters) are compared to the ones found by statistical models. Such benchmarking provides a way of raising the relative quality of predictions and inferring adequate preprocessing-model setups for a particular time series.

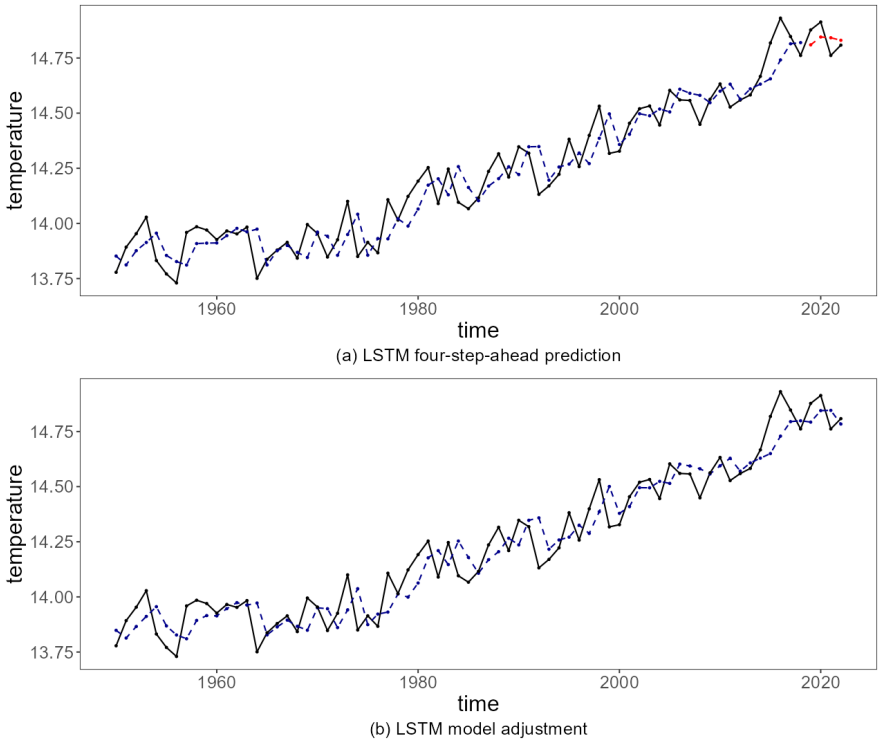


Fig. 2.19 `LSTM` with four input neurons for `YGT`: four-step-ahead prediction (a); model adjustment (b)

Chapter 3
Anomaly detection

Anomalies are unexpected or abnormal patterns or events in a time series. This chapter provides an overview of anomaly detection. Anomalies principles are described with basic definitions, characterizing point and sequence anomalies in Section 3.1. The main anomaly detection methods are grouped into six categories: regression, classification, clustering, statistical, spectral, and information theory. They are covered in Section 3.2. Finally, Section 3.3 presents the advanced topics in this research area.

3.1 Basics of anomalies

Anomaly detection has been drawing attention for many years due to its impact in many domains, such as fault diagnosis in industry, water quality, outbreaks in health, and fraud detection [58, 174, 214]. Anomalies are observations that do not fit the typical time series observations. They seem to be generated by a different process. However, the actual definition and characterization of anomalies might differ among various works. They are also commonly known as outliers and discords [125, 167, 198].

There is a slight difference between anomalies and outliers. Commonly, outliers are treated in data mining as something undesired. Usually, they are associated with noise, error, or unwanted data [5]. They are generally removed before the main data mining function during data preprocessing [118]. Conversely, anomalies are usually those unusual data (outliers) we are interested in studying. Fraud and fault detections are examples of this case since the goal is exactly to detect and analyze these outliers [5, 38, 208].

Figure 3.1 shows the taxonomy of anomalies. Anomalies can be organized according to their type, circumstances in which they occur, their dimension, and interpretation. Additional anomaly detection is driven according to how detection models learn from data.

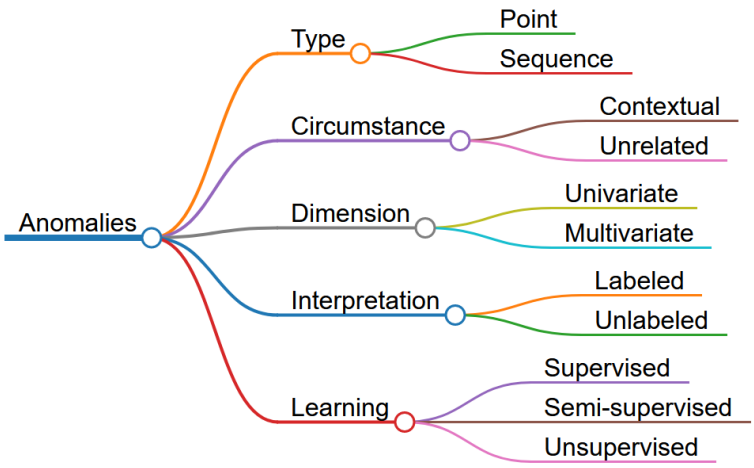


Fig. 3.1 Anomaly taxonomy. It is organized according to the type, circumstance, time series dimension, event interpretation, and model learning

3.1.1 Point and sequence anomalies

Anomalies can be classified by their type: point or sequence [39]. A point anomaly is a single observation that significantly differs from the remainder of the time series. Figure 3.2a presents a point anomaly (either in green or red) of the temperature in Celsius for each week in Seattle during 2019. The observations in green correspond to a labeled anomaly identified by specialists. It was correctly detected by Forward and backward inertial anomaly detector (`FBIAD`) using a sliding window size of 12 [164]. The observations marked in red correspond to false positives.

A sequence anomaly corresponds to a sequence of observations significantly different from the remainder of the time series [38]. Figure 3.2b presents an example of sequence anomaly of size three (marked in red) for a synthetic time series. It was the same temperature for Seattle during 2019, but the temperature of weeks 25 and 26 were modified to 16.6^oC and 36.6^oC, respectively. These changes lead to a sequence anomaly from weeks 24 to 26, with an atypical amplitude change for the maximum temperature.

3.1.2 Contextual and out-of-context anomalies

Anomalies can be classified according to the circumstances in which they occur. Consider an anomaly that occurs at time t. It is called contextual anomaly when the observation at the time t is not anomalous [195]. Still, it is anomalous considering

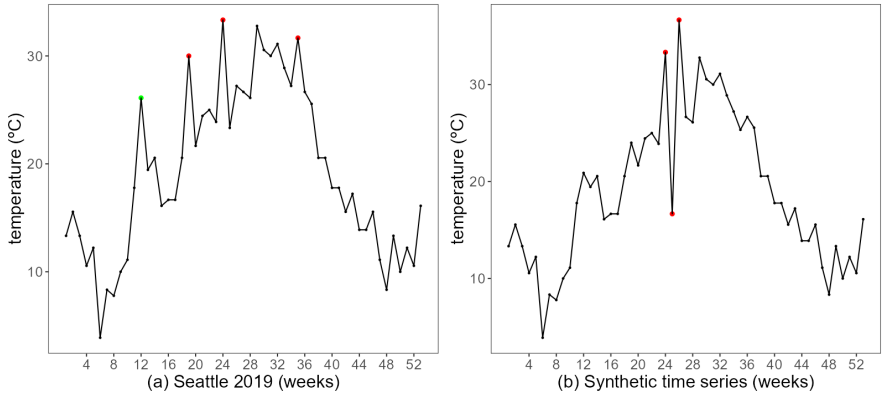


Fig. 3.2 Point and sequence anomalies examples: maximum temperature of each week in Seattle in 2019 (a), synthetic time series (b)

the nearby observations. So, for example, 26^oC in a day is a common temperature in Seattle during summer. Such temperature is anomalous in winter as reported by local newspapers[1]. An example of a contextual anomaly is depicted in Figure 3.2a. The data was extracted from Weather Underground[2]. Conversely, an out-of-context anomaly is an anomaly that is not related to the temporal occurrence of the observation [59], as the red observations detected by `FBIAD` in Figure 3.2a.

3.1.3 Univariate and multivariate anomaly detection

Anomalies can be classified according to dimension. They can occur in univariate or multivariate time series [299]. In univariate time series, an anomaly is an unusual observation that occurs in time t. An anomaly in a multivariate time series can be an unusual combination of observations in a given time t. Figures 3.3a and 3.3b exemplify a multivariate time series scenario. They depicted two variables generated using the standard normal distribution for the sake of an example. Both series can be addressed using univariate methods for anomaly detection. Observations in Figures 3.3a and 3.3b marked as red or green correspond to detections obtained using `ARIMA` model.

The extreme values of Figures 3.3a and 3.3b are usually anomalies. However, certain combinations of observations can be anomalous from a statistical point of view but are not captured by individual time series analysis. This situation opens room for multivariate anomaly detection methods. It is the case of Figure 3.3c where a `PCA`-based anomaly detection method is applied to the multivariate time series. The `PCA` for these two variables is computed, and the residual between observations and

[1] `https://eic.cefet-rj.br/~eogasawara/en/seattle-temperatures`

[2] `https://www.wunderground.com/weather/us/wa/seattle`

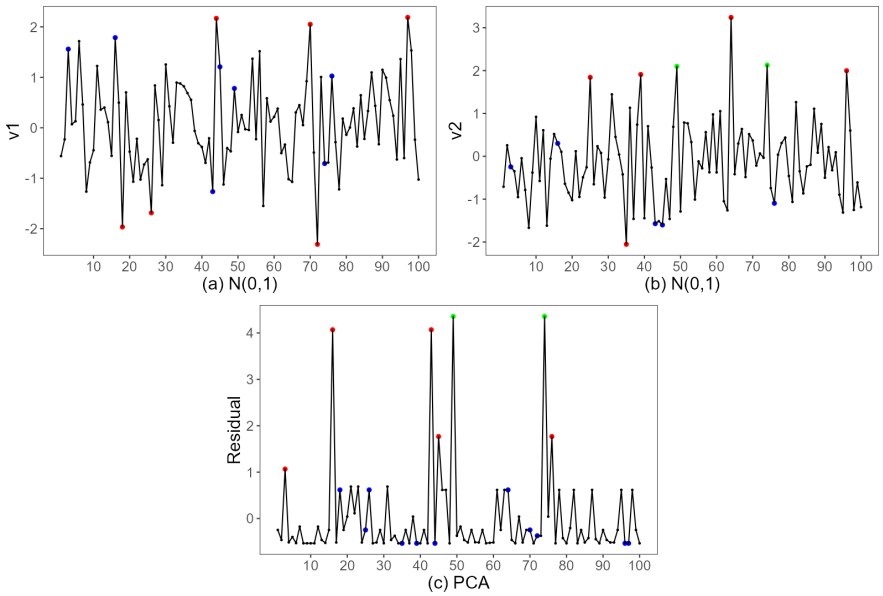


Fig. 3.3 Multi variate example: two standard normal distribution variables (a) and (b), residual of Principle component analysis (PCA) multivariate anomaly detection method (c)

the PCA is checked for atypical values. They are marked in green and red in Figure 3.3c. The blue points in Figure 3.3a and Figure 3.3b correspond to the observations that were atypical in the combination of the two series. The observations in green in Figure 3.3 matched the ones that both univariate and multivariate methods detected. The observations marked as blue in Figures 3.3a and 3.3b were detected by the multivariate method but not by the univariate method. Conversely, observations marked as blue in Figure 3.3c are detected by univariate methods but not using the multivariate methods. It becomes clear that combining univariate methods is not enough to capture certain anomalies obtained in multivariate analysis. Nevertheless, multivariate time series anomaly detection does not replace univariate anomaly detection. These methods should be used in cooperation to address the problem.

3.1.4 Labeled and unlabeled time series

A time series might have information about each observation, indicating whether it is an anomaly. It is called labeled time series. Conversely, when such a label does not exist, it is an unlabeled time series. In this case, the anomalies might be evaluated using statistical analysis, but they are subjected to interpretation [133].

When the dataset is labeled, such labels are commonly marked by humans (experts). However, there are two challenges to providing these labels. First, some events

might be missed during labeling. Second, the indication of the event might be registered imprecisely (at an uncorrected time), leading to a lag between the event and the actual moment it occurred [280].

3.1.5 Supervised, semi-supervised, and unsupervised

Since detection models are targeted to discover anomalies, it is possible to classify the methods that build these models according to the presence or absence of these labels. Stating in other words, according to how they learn from data [59]: (i) supervised; (ii) semi-supervised; (iii) unsupervised.

In supervised learning, the time series has a label associated with each observation, indicating if it is an anomaly or typical data. Once trained, this model is expected to determine the class label of a novel observation. Thus, supervised learning separates the time series into training and testing sets. This approach makes sense in online event detection, where the entire time series is used for training and new upcoming data is tested [198]. This process is rather simple and is traditionally used in general data mining tasks [118].

However, there are major challenges in supervised learning for anomaly detection. First, anomalies are, by definition, rare in the time series. This rarity raises the traditional imbalanced class distribution problem [118]. Approaches such as data augmentation inject artificial anomalies into a time series to obtain a labeled training data set [299] as a way to address this issue. The second challenge concerns obtaining time series with labels, which brings up the discussion previously mentioned [198].

In semi-supervised learning, it is assumed that the training data has only labeled instances for the typical data. Since it does not require anomaly class labels, they are more used than supervised techniques [253]. Finally, in unsupervised learning, no training data is required. The approach assumes that typical data is much more frequent than anomalies. Since it is a less constrained approach, it is the most adopted one [8, 309].

3.2 Methods for anomaly detection

This section introduces the main anomaly detection methods. They are grouped according to the principles on which they are based. It includes regression, classification, clustering, statistical, spectral, and information theory.

While presenting the methods, Figure 3.4 describes a synthetic time series that compares methods under a controlled situation. It also separates the time series into training (between 1 and 75) and test (between 76 and 101).

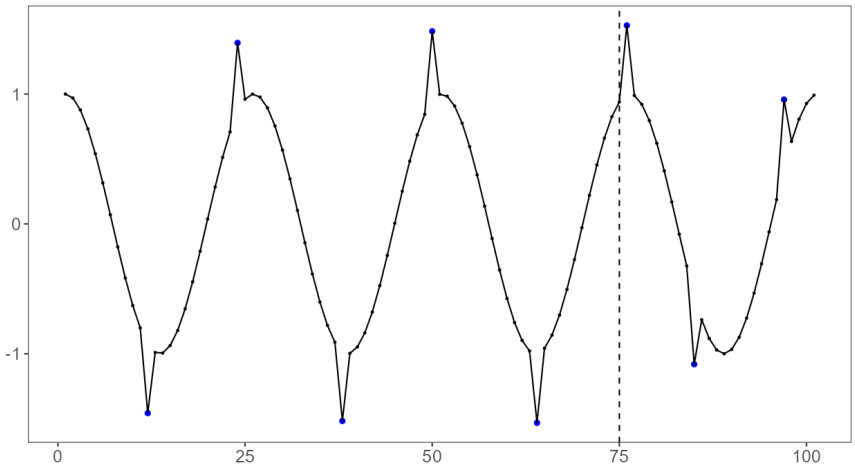


Fig. 3.4 Synthetic time series for comparison of methods (Example 18 available at Harbinger [236])

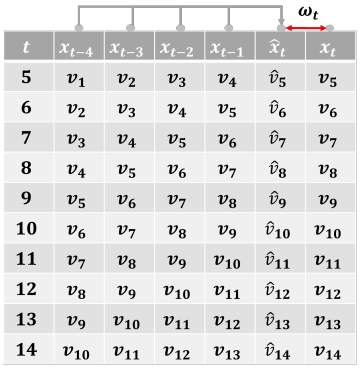
3.2.1 Regression

One of the most commonly adopted strategies is based on model deviation detection. Model deviation event detection refers to identifying events where there is a deviation between the actual behavior of the time series and the expected behavior as predicted by a model. First, a statistical or machine learning model is fitted to the available data. Then, events are identified as the observations that most deviate from the fitted model [118].

Figure 3.5 presents an example of a time series organized as `SW` of size 5. The four lagged terms of the time series x_{t-4} to x_{t-1} are used to predict x_t. The prediction is expressed as $\hat{x}_t$. In this case, ω_t corresponds to the difference between $\hat{x}_t$ and x_t. The distribution of ω_t is analyzed for noise anomalies.

This strategy is one of the most widely used for anomaly detection [38, 59, 205]. This strategy was adopted by Liu et al. [171], presenting a multivariate event detection method based on the deviation from an ARIMA model above a threshold. Other applications include machine learning models, generally obtained by unsupervised learning [302]. This strategy is also the basis for many change point detection approaches.

Consider a time series X whose observations are represented as x_t $(1 \leq t \leq |X|)$. A regression model can be built, providing a prediction (or estimation) for each observation $\hat{x}_t$. The distance between observations and estimations can be modeled as a residual time series ω_t, such that $\omega_t = distance(x_t, \hat{x}_t)$. An anomaly is an observation that significantly deviates from the estimation, *i.e.*, $\omega_t > \gamma$, where γ stands for a threshold to model deviations. This type of method is commonly known as the regression method. It is the most common approach adopted in the literature



(a)

Fig. 3.5 Example of model deviation detection using SW

for anomaly detection. This approach is sensible to the method adopted to model the time series and the function used to measure the distance between the predicted and observed values [38, 155].

The simplest methods used to model the series include trend models (or followers). It includes FFT, WT, EMD, MAS, and splines. The first four are relatively rigid because their modeling assumption provides inertia [114, 164]. The last one is flexible, adjusting smoothly to the data [135]. Considering machine learning methods and statistical methods, as described in Chapter 2.1, there are various approaches to model time series, including statistical and machine learning methods [47, 227]. The former includes LR and ARIMA. The latter includes MLP, RFR, RBF, SVM, MLP, ELM, Conv1D, LSTM. These methods were covered in Chapter 2.

As expected, how distance is computed interferes with whether the anomalies are detected. A typical approach is to compute the absolute difference between observations and estimations. The error is relatively easy to interpret and has the same scale as the data. The squared difference between observations and estimations is also well-known and commonly adopted. Since the differences are squared, large errors are highlighted.

In regression methods, outliers in the distance distribution are commonly registered as anomalies. The simplest way to identify if an observation is an outlier is by assuming a Gaussian distribution for the distance [270]. It is a parametric evaluation that establishes that observations that are distant from the mean ($\overline{\omega} \pm 2.698\sigma_\omega$) are outliers, where $\overline{\omega}$ and σ_ω stand for the mean and the standard deviation of distance, respectively [135].

An alternative in the parametric analysis is to use Grubb's test [111]. This test is known as the maximum normed residual test. It is a statistical test used to detect outliers in a dataset by identifying data points significantly different. It compares the suspected outlier to the sample mean and standard deviation. Suppose the difference between the suspected outlier and the mean is significantly larger than expected

based on the standard deviation. In that case, the data point is considered an outlier [261].

The box plot rule is the simplest non-parametric statistical technique to detect anomalies, considering that distant observations are relative to the first quantile (Q1) and third quantile (Q3). The Inter Quartile Range (IQR) equals Q3 - Q1. Generally, outliers are observations that are not between the interval defined by Q1 - 1.5IQR and Q3 + 1.5IQR. This interval contains 99.3% of observations [118]. The lower constraint is not applicable in anomaly detection, which could stand for very precise models concerning the observations. So, only distances greater than Q3 + 1.5IQR are registered as anomalies.

Figure 3.6 presents an example of regression methods. `ARIMA` is adjusted to the time series (represented in dashed blue). The residuals at anomalies are significantly different from the remainder of the residuals. Surrounding anomalies, the `ARIMA` model is slightly influenced, and the distance between the adjusted model and the time series is higher than usual. As the influence of the anomaly decreases, the model smoothly adjusted correctly to the time series.

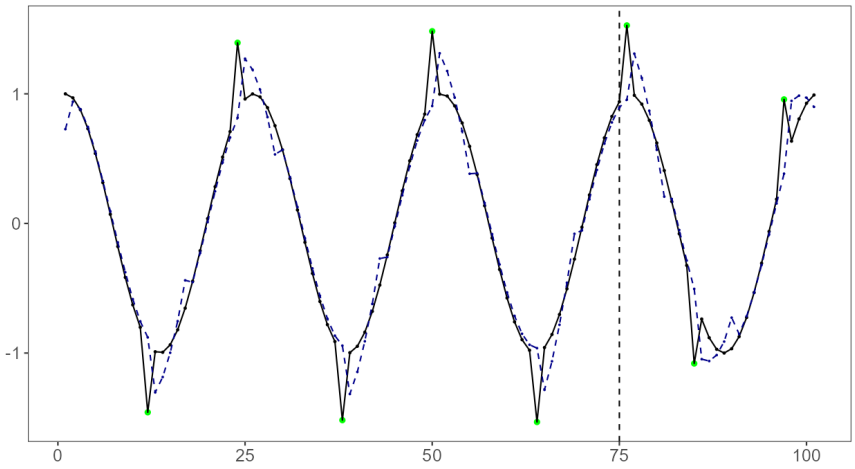


Fig. 3.6 Anomaly detection using `ARIMA`

Finally, more advanced approaches include using extreme value theory. It computes a typical time series behavior boundary. It adopts a density-based comparison to detect any significant changes in the distribution of the observations [137, 256]. These approaches are generally adopted in online event detection. The advantages include not having assumptions on the distance distribution [245].

3.2.2 Classification

Classification-based event detection involves identifying and classifying events based on time series. In this process, a set of features or lagged observations are extracted from the time series, and machine learning algorithms are used to classify the events into different categories. The approach involves training a classification model on a labeled dataset.

Methods in this category commonly perform binary classification of time series observations as falling into a normal or abnormal class based on previously learned patterns. Figure 3.7a presents an example of a time series organized as SW of size 5. It is worth mentioning that during the model building, the labels for the presence of events should be known and used during training. All lagged terms of the time series x_{t-4} to x_t are used to predict the class label for the event e_t. The prediction of the model is expressed as $\hat{e}_t$. Some metrics for the evaluation of events, such as f_1, are used to measure the accuracy of the model during training, leading to the confidence of the model prediction during testing. As it is depicted in Figure 3.7b, no labeled data is provided during testing.

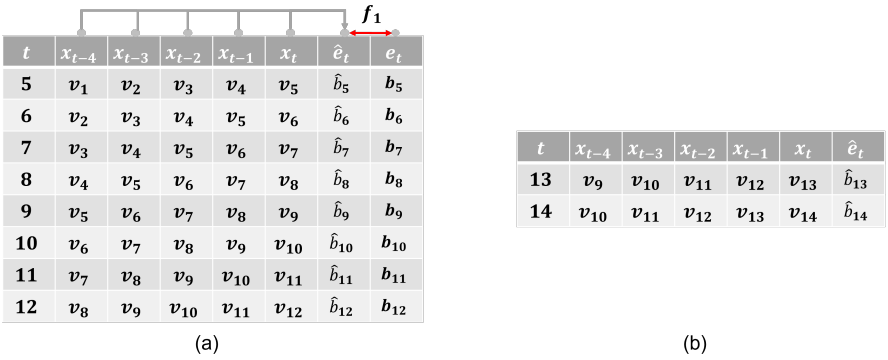


Fig. 3.7 Example of classification-based detection using SW: (a) during training labeled data is available; (b) during testing no labeled data is available

For example, Provotar et al. [222] perform unsupervised anomaly detection in time series using LSTM-based autoencoders. Park et al. [209] also combined autoencoder and LSTM networks for fault detection and system diagnosis. Additionally, Polz et al. [219] and Kiskin et al. [154] use convolutional neural networks to detect events. Polz et al. [219] detect rain events in commercial microwave link attenuation data. Kiskin et al. [154] conduct bioacoustic detection with wavelet-conditioned convolutional neural networks.

Anomaly detection using classification models is the counterpart of supervised learning. It uses time series and labels to build a model. The simplest scenario is a binary classifier that predicts typical and anomaly observations. In this way, Logistic

regression (LogReg), Naive Bayes (NB), Decision trees (DTree), Random forest (RF), SVM, NNET, LSTM, and Conv1D are examples of classifiers [59, 118].

As previously mentioned in supervised learning, the classification is more appropriate for online event detection since it requires training and testing phases. The training phase uses observations and labels to build the model. The testing phase uses the model with new upcoming observations [198, 260]. This methods class is discussed more deeply in Chapter 7.

Figure 3.8 presents anomaly detection using an SVM classifier. The model uses SVM with a radial kernel with $\epsilon = 0$ and $cost = 80$. During training, all anomalies were confirmed, but during testing, only the first anomaly (observation 80) was detected. Other anomalies were false negatives.

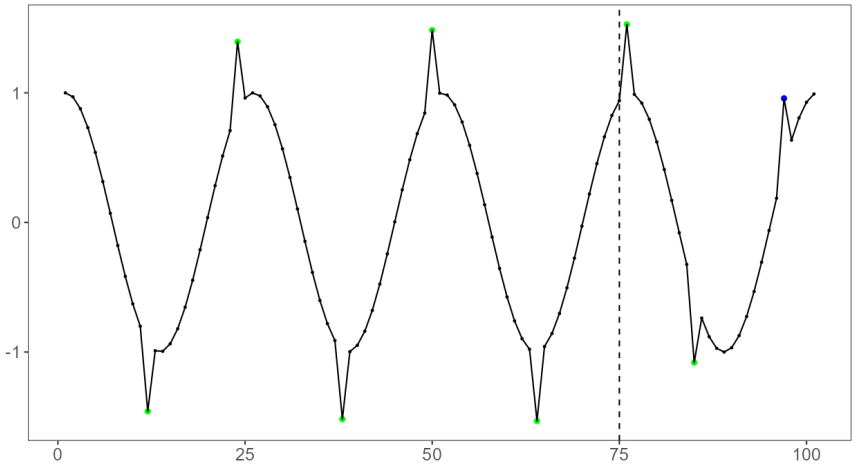


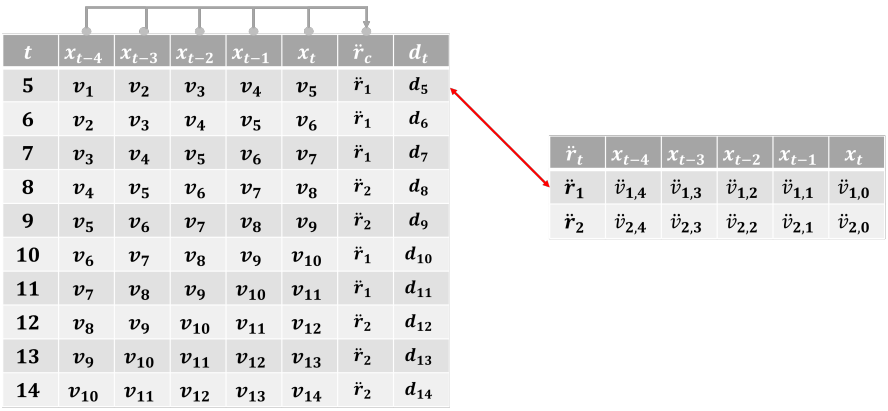
Fig. 3.8 Anomaly detection using SVM classifier

3.2.3 Clustering

Clustering-based event detection refers to identifying and grouping events that exhibit similar patterns or behaviors. In this process, a set of features or attributes is extracted from the data, and clustering algorithms are used to group data according to their similarity. In this case, abnormal data samples can also be identified through a clustering-based strategy by comparing them against normal sample clusters. The approach assumes that events are related to abnormal data [181].

Figure 3.9 presents an example of clustering-based event detection using SW. Each window is assigned to a cluster $\ddot{r}_c$. Each cluster has a representative sequence for distance evaluation, such as centroid. All windows are evaluated according to

a distance metric (such as Euclidean or DTW [47]) concerning the representative sequence. The distance distribution analysis characterizes the presence or absence of an anomaly. As it computes the distance of the entire sliding window, this technique can be used for both point and sequence anomaly detection.



t	x_{t-4}	x_{t-3}	x_{t-2}	x_{t-1}	x_t	$\ddot{r}_c$	d_t
5	v_1	v_2	v_3	v_4	v_5	$\ddot{r}_1$	d_5
6	v_2	v_3	v_4	v_5	v_6	$\ddot{r}_1$	d_6
7	v_3	v_4	v_5	v_6	v_7	$\ddot{r}_1$	d_7
8	v_4	v_5	v_6	v_7	v_8	$\ddot{r}_2$	d_8
9	v_5	v_6	v_7	v_8	v_9	$\ddot{r}_2$	d_9
10	v_6	v_7	v_8	v_9	v_{10}	$\ddot{r}_1$	d_{10}
11	v_7	v_8	v_9	v_{10}	v_{11}	$\ddot{r}_1$	d_{11}
12	v_8	v_9	v_{10}	v_{11}	v_{12}	$\ddot{r}_2$	d_{12}
13	v_9	v_{10}	v_{11}	v_{12}	v_{13}	$\ddot{r}_2$	d_{13}
14	v_{10}	v_{11}	v_{12}	v_{13}	v_{14}	$\ddot{r}_2$	d_{14}

$\ddot{r}_t$	x_{t-4}	x_{t-3}	x_{t-2}	x_{t-1}	x_t
$\ddot{r}_1$	$\ddot{v}_{1,4}$	$\ddot{v}_{1,3}$	$\ddot{v}_{1,2}$	$\ddot{v}_{1,1}$	$\ddot{v}_{1,0}$
$\ddot{r}_2$	$\ddot{v}_{2,4}$	$\ddot{v}_{2,3}$	$\ddot{v}_{2,2}$	$\ddot{v}_{2,1}$	$\ddot{v}_{2,0}$

Fig. 3.9 Example of clustering-based detection using SW

A common approach to detect anomalies is to analyze clustering sequences in a time series, such as using K-means. The centroid of each cluster is defined by the algorithm iteratively by averaging sequences within each cluster [38]. Afzalan et al. [4] adopted this strategy, identifying events through automated clustering. They propose self-configuring event detection in electricity monitoring for human-building interaction. Zhao et al. [304], on the other hand, perform spectral analysis for identifying anomalies in network traffic time series. Other measures for distance, such as Dynamic time warping (DTW) [82], are used to improve the detection of anomalies. Clustering is a data mining function that groups similar data instances into clusters [162]. It is an unsupervised learning technique [8]. Several clustering-based anomaly detection methods have been developed. They can be classified into two major categories: (i) density-based and (ii) centroid-based [77, 239].

The density-based category assumes that typical observations belong to a cluster. Conversely, anomalies do not belong to any cluster [77]. This assumption relies on clustering methods that are not obliged to assign a cluster to each observation. These methods are optimized to find similar data. The ones that do not fit such similarities are outliers.

More formally, techniques within this category consider that a sequence w (of size l) with less than κ close neighbors are outliers. Close neighbors are the ones with a distance smaller or equal to γ. In this way, w_t is an outlier in a time series X if and only if $|\{\forall w \in sw(X,l), d(w,w_t) \leq \gamma\}| < \kappa$ [38].

Density-based methods include Density-based spatial clustering of applications with noise (DBScan) [94] and ROCK [113]. This category also includes approaches that separate typical data from anomalies based on the size of the clusters. Typical data

belongs to large clusters, whereas anomalies belong to smaller clusters. Techniques based on this assumption include [5, 59, 310].

The centroid-based category assumes that typical data are closed to clustering representatives. A common approach considers centroids as the clustering representatives but could also be medoids [142]. Conversely, anomalies are distant from their closest cluster centroid. So, the method consists of clustering data and computing the distance between observations and their respective centroids. K-mean is a representative algorithm for this category [134].

This category also supports semi-supervised learning. In this case, typical data are clustered using Self organizing maps (`SOM`) and Expectation Maximization (EM) [63, 162]. This approach characterizes clusters of typical data. Then, new observations are compared against the clusters. An anomaly score is obtained for them. It is the reason why the training dataset should not have anomalies. Otherwise, these methods would not be able to detect anomalies [38].

Figure 3.10 presents an example of anomaly detecting using clustering. It applies a `K-means` using k and sequence size equals three and one, respectively. Since it is an unsupervised learning method, it computes the centroids using the entire time series. Sequences that are far away from centroids are marked as anomalies. It correctly found five out of the eight anomalies presented in the example.

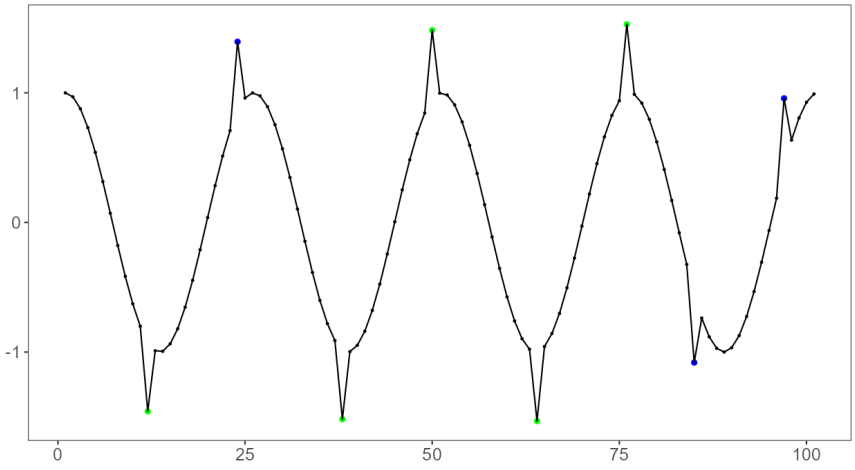


Fig. 3.10 Anomaly detection using `K-means` with k equals 3 and sequence size equals 1

3.2.4 Statistical

Statistical-based event detection methods refer to statistical techniques for identifying events or situations deviating from the expected behavior. These methods involve analyzing the data and applying statistical tests to detect changes, anomalies, or patterns indicative of potential problems or failures. It may include time series analysis, hypothesis testing, and regression analysis.

For example, some works adopted a method that involves the analysis of divergence in the distribution of time-varying poison processes [59, 217]. Other probabilistic approaches involve a density estimation of the normal class [17]. McKenna et al. [179] present a method that checks whether outliers in a water quality time series can be marked as events when their probability in a data window differs from the expected binomial distribution. Statistical approaches are also reviewed by Souto and Liebig [249] in the context of anomaly detection for urban traffic applications. They also present frameworks for anomaly detection on streaming data.

The main principle of the statistical anomaly detection technique is that typical observations occur in high-probability regions of a statistical model. Conversely, anomalies occur in the low probability regions [6, 135]. Models can be either parametric or non-parametric. Both `LR` and `ARIMA` fall under the parametric model's category. These models consist of two steps. In the first step, the statistical model is fitted to the data. There are many approaches to evaluate model fitting, including `MSE`, `sMAPE` R^2, and Akaike information criterion (`AIC`) [45, 270]. In the second step, for each test instance, the residual for the test instance is used to determine the anomaly. It has the same principles presented in model adjustment, but models are based on statistical assumptions [47].

Under the non-parametric model analysis, the histogram-based methods fall into the simplest approaches. It uses histograms to maintain a profile of typical observations. This technique is also known as frequency-based or counting-based. Like classification models, they are more appropriate for online event detection. The histogram is built using the training set. Then, new observations are tested to fall in any one of the bins of the histogram. If it falls in a frequent bin, the observation is typical. Otherwise, it is anomalous [59]. It becomes clear that the bin intervals used to build the histogram are key for anomaly detection. If the bin intervals are too short, many typical observations fall in empty or rare bins. This results in a high false alarm rate. If the bin intervals are too large, many anomalous observations might fall in frequent bins, resulting in a high false negative rate [38].

The histogram-based approach has limitations, especially when dealing with high-dimensional or multi-modal data. Creating histograms becomes challenging in such cases, as it may require dividing the data into many bins, leading to sparse histograms and loss of detail. They also might find difficulty in capturing the relationship among dimensions [59].

The Parzen window estimation is an example of kernel density estimation [211]. It is a non-parametric statistical technique to estimate the `PDF` of a continuous random variable. Inspired by this principle, Desforges et al. [80] proposed a semi-supervised statistical technique to detect anomalies. It estimates the Probability density function

(PDF) for the typical observations. A new observation in this low probability area of the PDF is declared anomalous [131].

An advantage of statistical techniques is that such methods adopt statistically justifiable solutions for anomaly detection. An additional benefit of such an approach is the ability to define a confidence interval that separates anomalies from typical observations. Conversely, the disadvantage of statistical techniques is the implicit assumption that the data is generated from a particular distribution. This assumption often does not hold, especially for non-stationary or high-dimensional real-world time series [47].

Figure 3.11 presents anomalies detected using the histogram method. Anomalies are observations that fall outside bin intervals, or observations present in bins with low frequency. In the former criteria, anomalies are values less than -2 or greater than 1.5. In the latter, anomalies are values in bins 1, 2, and 7 (between -2 and -1 or between 1 and 1.5).

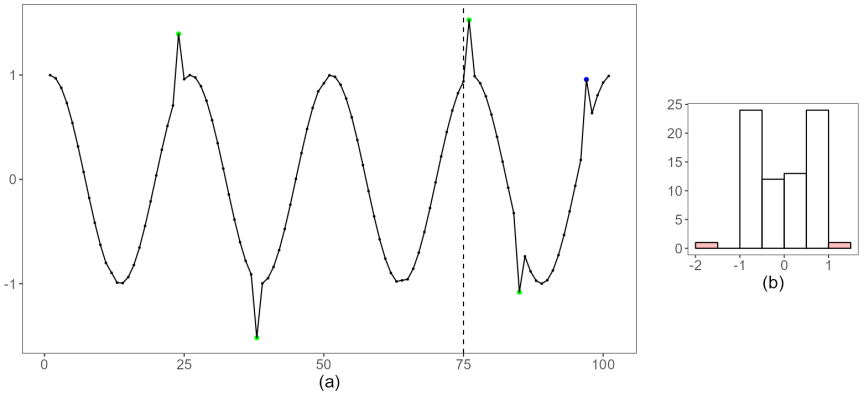


Fig. 3.11 Anomaly detection using histogram (a). Computed histogram in the training set to support anomaly detection (b)

It is worth mentioning that some of the anomalies in the training set were removed from the example of Figure 3.11 since they would be too frequent. The histogram would fail to find them in such a case due to their high frequency.

3.2.5 Spectral

Generally, spectral techniques try to find an approximation of a dataset using a combination of attributes that capture the main variability of the data. This category of methods is based on the premise that data can be embedded into a lower dimensional space. In this reduced subspace, typical observations and anomalies appear significantly different. Thus, the general approach adopted by spectral anomaly detection

techniques is determining subspaces (such as embeddings and projections) where the anomalous instances can be easily identified [90]. Such techniques can work in an unsupervised and semisupervised setting [59].

Intuitively, this approach is suitable for analyzing multivariate time series. In this case, the embedding naturally reduces the number of dimensions. Nevertheless, the approach is also applicable to univariate time series. Considering the sliding windows sw for a time series of size p, each lagged term is expressed as an individual dimension. Thus, lagged terms could be embedded similarly to multidimensional datasets.

One of the simplest spectral approaches to project data in lower dimensional spaces is `PCA` [2]. It projects data into a lower dimensional space. The goal is to analyze the projection of each data instance along the principal components. A typical instance satisfies the correlation structure of the data and the projection. In contrast, an anomalous instance significantly deviates from the PCA representation [38, 210]. An example of anomaly detection using `PCA` was depicted in Figure 3.3c.

Recently, autoencoders have become one of the most commonly used methods in spectral methods [181, 209, 222]. Autoencoders are a type of neural networks that learn the most significant features of a training set [7]. Since Anomalies often correspond to non-representative features, autoencoders fail to reconstruct them, providing distances between encoded and original sequences [3]. Sakurada and Yairi [231] use this method, where the input of the autoencoder is a single multivariate point of the time series. This approximation does not consider the temporal correlation between the observations within each variable.

Figure 3.12 presents an example of an autoencoder anomaly detection method. The example has a neural network with five inputs. The output layer produces five observations. These values should be the same as the input. The key point of the autoencoder neural network is the hidden layer structure having an hourglass shape. This property is the main principle of this spectral approach, as the encoder layer with fewer neurons than the input has enough information to decode the output values. Anomalies are observations outputted from the autoencoder significantly different from the input. Due to that, it is a practical approach to train the neural network using semi-supervised learning, *i.e.*, the training dataset has only typical observations. Each layer might be fully connected to the following. However, the example of Figure 3.12 is not fully connected for visibility.

Figure 3.13 presents an example of anomaly detection using autoencoder. It uses an input size of three neurons and an encoder layer of size 1, and the output size has three neurons. Even in this simple architecture, all anomalies in this example were correctly detected.

Kieu et al. [150] propose extracting features within overlapping sliding windows (e.g., statistical features) before applying the autoencoder to account for temporal dependencies. Su et al. [251] suggest a more complex approach based on a Variational autoencoder (`VAE`) with a Gated recurrent unit (`GRU`). The input of the model is a sequence of observations containing x_t and p preceding observations ($< x_{t-p+1}, \cdots, x_t >$). The output is the reconstructed ($< \hat{x}_{t-p+1}, \cdots, \hat{x}_t >$). Additionally, they apply extreme value theory [245] to detect the anomalies.

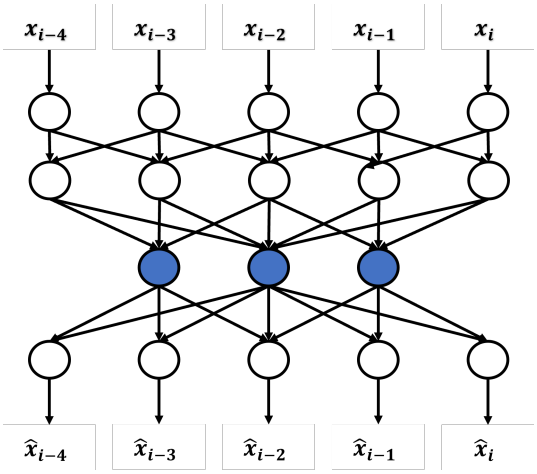


Fig. 3.12 Auto encoder neural network

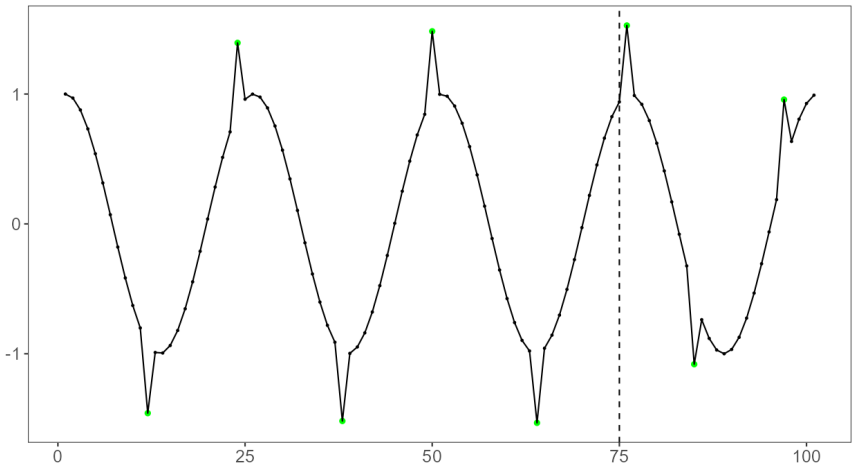


Fig. 3.13 Anomaly detection using autoencoder with input size equals 3, and encoder layer size equals 1.

3.2.6 Information theory

Methods based on this category analyze the time series context based on information theory concepts, such as Kolmogorov Complexity and entropy [35, 59]. The general assumption is that anomalies are rare, and typical observations provide little information. Conversely, anomalous observations introduce more information and entropy to the sample analyzed.

Let S be a set of all sequences of size p of a time series X, *i.e.*, $S = sw(X, p)$. The information of these sequences can be represented as $I(S)$. Anomaly detection methods based on information theory aim to find a subset A of S such that $|I(S) - I(S - A)|$ is maximized. The sequences present in A are considered anomalies. To compute the information I, Keogh et al. [147] use the size of the compressed data file (using any standard compression algorithm) to measure the Kolmogorov Complexity.

Some anomaly detection methods are publicly available at Harbinger (see Section 8.3).

3.3 Advanced topics in anomaly detection

Due to the importance of this subject and its numerous applications, several surveys on event detection have been published, especially in the last decade [203, 302]. Most focus on reviewing and comparing methods for detecting anomalies in time series [9, 116, 271]. Some works target specific data structures, such as sequences [60] and graphs [11, 228]. Some works present their analysis in a particular context, such as Ahmed et al. [9] that surveyed anomaly detection techniques in the financial domain, Dixit et al. [88] study anomalies on autonomous electric vehicles, and Aggarwal [6] provides comprehensive coverage of outlier analysis from a computer science point of view. Other works focus on methods that can be applied to data presenting specific characteristics. For example, Taha and Hadi [253] focus on categorical data, Thudumu et al. [261] focus on high dimensional big data, and Chandola et al. [61] focus on sequence data events. Alevizos et al. [17] and Pang et al. [205] concentrate their reviews on probabilistic methods and deep learning methods, respectively. Alamr and Artoli [16] apply transforms to improve anomaly detection more efficiently than other deep learning methods. Wang et al. [271] chose to analyze the progress of anomaly detection methods from 2000 to 2019.

Several anomaly detection survey papers are available in the literature [74, 253, 261]. Recently, there have been specific surveys on deep learning [77, 205]. Some provide taxonomies that introduce a categorization of detection techniques [41]. Zhou et al. [306] studied non-parametric event detection in multiple time series for power distribution networks. They categorize event detection methods as Domain-based, Signal Processing and Information Theory Based, and Machine Learning Based. Besides the taxonomy, Sebestyen et al. [240] propose a platform that allows a developer to find and tune an optimal anomaly detection method for an application. Additionally, some initiatives combine the benefits of many different methods using ensembles [50, 161, 226, 307]. Finally, some surveys focus on benchmarking a broad range of anomaly detection methods [133, 194, 239].

3.3.1 Detection methods for volatility anomalies

As explained in Chapter 1, volatility anomalies are unexpected or abnormal fluctuations in the volatility of a time series. In this context, capturing the instantaneous volatility estimates for each time becomes important. For this goal, Generalized autoregressive conditional heteroscedasticity model (GARCH), described in Section 2.4.1 is commonly adopted.

The difference between observations and the computed model is subjected to outlier analysis to detect anomalies. This approach was evaluated by several fields, including finance and ecology [109, 241, 303]. Figure 3.14 presents a synthetic time series example with a sequence of observations with increased volatility. The example allows exploring a GARCH anomaly detector. The model found some anomalous observations within the sequence limited by the two dashed lines. However, a false positive is also detected at the beginning of the time series.

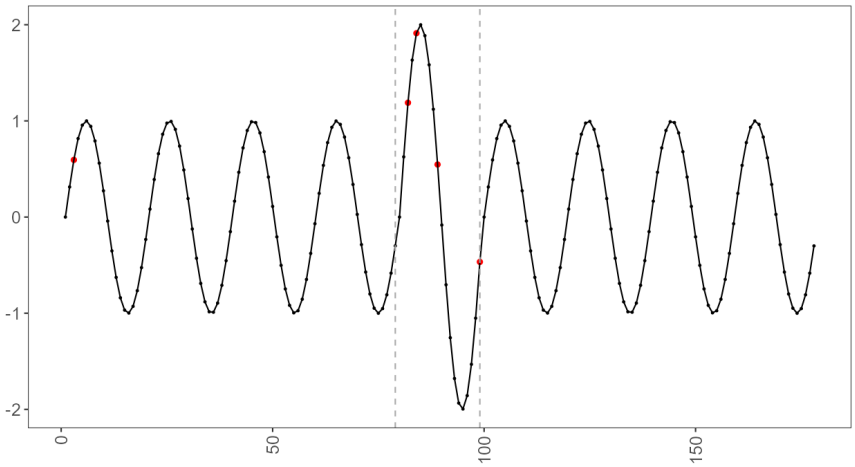


Fig. 3.14 Synthetic time series with increased volatility between observations 79 and 99

3.3.2 Detection methods for sequence anomalies

The first and most straightforward idea consists of detecting the most unusual subsequence in a time series (denominated time series discord) by comparing each subsequence with all other subsequences in the time series. In other words, w is a discord of size p in a time series X when there does not exist another sequence $v \in sw(X, p)$ such that $w \approx v$. Typically, discord discovery techniques require the user to specify the length of the discord [60, 144].

A simple approach to finding discords is using brute force, which requires a quadratic pairwise comparison. Even so, the process can be sped up by reordering the search using heuristics and pruning off some fruitless calculations [146, 166] using indexing techniques such as SAX discrete representation [38, 145]. Recently, initiatives such as matrix profile are driven to discover motifs and discords efficiently [167].

3.3.3 Detection methods for multidimensional time series

There are many cases where data might come from different sources or contain different attributes. Both cases fit the scenario of multidimensional time series. Strictly speaking, multivariate anomaly detection goes beyond univariate methods. It considers the relationships and dependencies among multiple variables [59, 96, 162, 299]. However, many insights from univariate time series analyses can aid the multivariate analysis.

The first direct approach uses univariate methods to detect anomalies in each time series. The observation at time t is considered typical only if all individual time series do not detect an anomaly at time t. The observation is considered anomalous if any time series contains an anomaly at time t. This approach directly benefits many of the methods described so far.

The second approach directly tries to detect events using the multivariate time series. Thus, multivariate anomaly detection methods aim to model the typical observations [251, 269]. Many different methods, including clustering, KNN or autoencoders, can be used to detect anomalies.

Using clustering, methods like K-means or DBScan can cluster observations. Anomalies are then detected as observations that do not belong to any cluster or belong to small clusters [162]. The KNN identifies anomalies based on the distance of observations to their nearest neighbors in the dimensional space [28, 96]. Autoencoders use a neural network for dimensionality reduction and feature learning. In anomaly detection, an autoencoder is trained to reconstruct normal data, and anomalies are identified by measuring the reconstruction error concerning the original data [209, 299]. It is important to observe that clustering techniques should be evaluated carefully since some situations might not lead to meaningful groups [144].

Finally, the last approach reduces the dimensionality of the time series. This approach commonly uses spectral methods. An example of this approach is to apply PCA. When the dimensions are reduced to one, it directly enables univariate time series analysis. When the reduced dimensions are still greater than one, both the first and second approaches can be applied.

Some multidimensional time series can be especially challenging to analyze, demanding more advanced approaches. It is the case for spatial-time series, encompassing data that evolve in space and time [96]. The challenge is due to the fact that one or more dimensions are, in fact, related to space. It introduces semantics on how data is arranged and might constrain the pattern and anomalies. Andrienko

et al. [25] and Souto and Liebig [249] designed their methods to extract anomaly events from spatial time series. Zhang et al. [300] derived a model for body posture spatio-temporal evolution. There are also initiatives on trajectory analysis, where the goal is to discover the most anomalous trajectories [160, 294].

3.3.4 Extreme Values

A particular type of anomaly occurs in a time series where some values are considered above or below a threshold. Nevertheless, they refer to significant observations of the monitored phenomenon. Observations falling into this category are named after Extreme Values (EV). They are considered to be generated by some Extreme Event (EE) process. Examples of EEs include extreme meteorological conditions, such as heatwaves and strong rainfall, and Tsunamis, leading to extreme wave heights. From a probabilistic point of view, EEs are naturally scarce, low frequency, and denoted by a few point values in the historical curve of the monitored phenomenon. When events are independent of each other, or at least they happen so far apart to keep any dependency, they are considered Independent and identically distributed (IID). As an example of an application, a city hall may plan to build a bikeway at height along its beach coast. In order to avoid being hit by the waves, it would be interesting to estimate the value for the highest waves that could reach the coast so that bikeway riders are secured.

Given the low frequency of extreme events, a distribution function modeling the underlying phenomenon places extreme values on both sides of the distribution tails. Thus, estimating extreme values from a history of measurements suffers from the effects of the central limit theorem and tends to be biased by more frequent normal measurements. In this context, dealing with anomalies of extreme values requires a special approach. A straightforward approach is to extract extreme values from the set of historical observations computed as a max function described in Equation 3.1, where n is the number of observations in a block, and x_i is an observation, for instance, the daily maximum wave height at day i. Moreover, x_i is univariate, so we can easily define an ordering among its values.

$$M_n = max\{x_1, x_2, \ldots, x_n\}^3 \tag{3.1}$$

The probability of an extreme event to occur, given 3.1 and a bound on the normal values z, can be exactly computed as Equation 3.2 where F(z) is a distribution function, which is, unfortunately, unknown. An alternative is to compute an approximation of $F(z)$ given by the extreme data only.

$$Pr(M_n \leq z) = Pr\{x_1 \leq z\} \times \cdots \times Pr\{x_n \leq x\} = F(z)^n \tag{3.2}$$

[3] The model can equally represent minimum values by adapting the aggregating function.

Coles [72] discusses three extreme distributions functions known as **Gumbel, Fréchet, and Weibull** distributions, that can be generalized using the Generalized extreme value (`GEV`) family of distributions, as in Equation 3.3. The three variations of `GEV` are obtained by considering different shapes for the tails of the distribution, given by the ζ shape parameter. While G decays exponentially for the Gumbel distribution, it follows a polynomial shape for the Fréchel one.

$$G(z) = exp\left\{-\left[1+\zeta\left(\frac{z-\mu}{\sigma}\right)\right]^{\frac{-1}{\zeta}}\right\} \tag{3.3}$$

In addition to the ζ shape parameter, the model includes a location parameter μ and a scale parameter σ to adapt to the three different distributions. The implementation of GEV considers a set of m blocks of size n, such that a set $M_m = \{M_{n,1}, M_{n,2}, \cdots, M_{n,m}\}$ can fit the GEV model parameters in Equation 3.3.

Chapter 4
Change points and concept drift detection

In science, enterprise, and government, there is a demand for time series analysis to support decision-making. Predictions, identified patterns, and models are valid as long as the observations in which these artifacts were built resemble the current analyzed observations. Detecting that changes occur is important for adaptability [33]. Besides, detecting the precise moment the change occurs is important for control systems to enable reaction at the right moment [21]. The general definition of change point is described in Section 4.1. Change point detectors are covered in Section 4.2. Then, the theme of concept drift is discussed in Section 4.3, whereas the main drift detector methods are detailed in Section 4.4. Finally, advanced topics regarding change points and concept drift are presented in Section 4.5.

4.1 Basics of change points

The change in a time series process is commonly observed as a change in its dynamics or distribution. Detecting change points becomes an important monitoring procedure. When changes are abrupt, the observation that separates each time series process is considered the change point. Identifying the exact observation in which change occurs might not be clear or feasible when changes are gradual. Change point detection is the problem of finding these observations that characterize the change between the time series process [21].

Figure 4.1 presents a time series with four change points (cp_{AB}, cp_{BC}, cp_{CD}, cp_{DE}). It separates the time series into five segments (from A to E), corresponding to different time series processes. The cp_{AB} characterizes a change in volatility. The cp_{BC} characterizes an abrupt change. The last two change points correspond to gradual changes.

For the moment, it is important to characterize the difference between offline and online change point analysis. In an offline scenario, the goal is to detect when the different time series processes occurred. In online analysis, the goal is to identify the change in the time series process as fast or as early as possible. This difference is

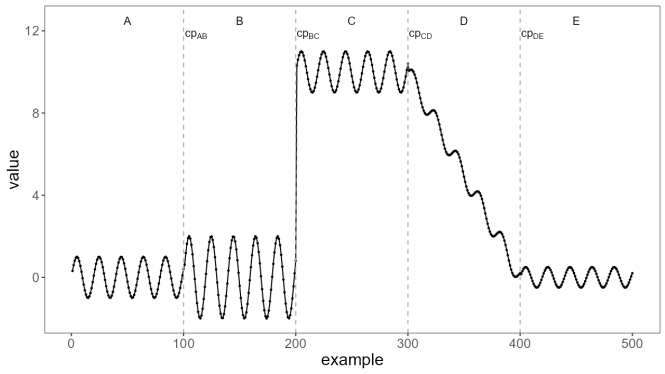


Fig. 4.1 A synthetic example of a time series with four change points

important to guide the reaction to an undesirable phenomenon. Additionally, labels that characterize anomalous observations or change points might not be available in online analysis immediately after the data is produced [254, 265].

Another aspect that drives most change point detection methods is related to the prior knowledge of the number of change points presented in the time series. If the number of change points is known, the problem becomes identifying when they occurred. In this case, we are studying an offline scenario. An easy-to-understand example is the analysis of hospital admissions of patients due to breath problems during the pandemic. Considering that a health monitoring government system knew about new COVID-19 waves, the goal would be to detect when these outbreaks affected each hospital. When the number of change points is unknown, the problem becomes identifying the right number of detected time series segments. Considering the previous example, it would discover if and when the new COVID-19 waves appear. If the change point detector finds too many small segments [265], there would be some false positives. If the detector finds too few segments, some false negatives might occur. Correctly addressing this dichotomy becomes an optimization problem, targeting to separate strong change points from weaker or even spurious ones.

Figure 4.2 presents a taxonomy adapted from Truong et al. [265] for the major change point detection methods. It is organized based on the search principles for change point methods and representative detectors. The search principles can be organized according to segmentation, regression, and sliding windows.

The segmentation approach aims to minimize the distances among observations in each segment or maximize the distances among segments. When the number of change points (m) is established, a naive solution is to explore all possible segment partitions by brute force. If a segment is representative, the distances among their observations should be low. This property is similar to clustering analysis. When the number of segments is not established beforehand, exploring different m values is needed. Otherwise, the solution would face the same challenges as observed in clustering algorithms with clustering, leading to an extreme scenario of having an

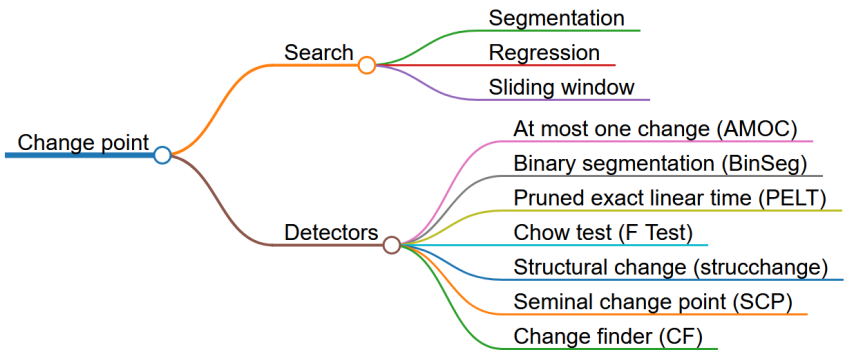


Fig. 4.2 Change point taxonomy (adapted from Truong et al. [265])

individual cluster for each observation. A penalty for each increase in the number of segments is commonly adopted to address this issue. Methods in this category include At most one change (`AMOC`) [151], and Pruned exact linear time (`PELT`) [152]. They are detailed in Section 4.2.

The search using regression is based on exploring breakpoints in the time series. For each candidate segment, a regression model is built. The quality of each segment is measured using Residual sum squared (`RSS`), for example. The lower the `RSS`, the better the established breakpoint. This property leads to an optimization problem of finding the right breakpoints that minimize the `RSS`. Dynamic programming is usually adopted to support this optimization process. Methods in this category include Chow test (F test) (`Chow test`) [24, 139], and Generalized fluctuation test (`GFT`) [32, 84, 298]. They are detailed in Section 4.2.

Finally, the search based on sliding windows explores the time series by computing all possible subsequences of a specified size p to detect change points. Inside each sequence, the goal is to check if a single change point exists. Many methods reside in this category, including, but not limited to, the two previous ones (segmentation and regression) [92]. The most common sliding window methods search for a breakpoint and a test to evaluate if the possible two segments inside the window are significantly different. Methods in this category include Seminal change point (`SCP`) [117] and Change finder (`CF`). They are detailed in Section 4.2.

4.2 Change point methods

This section explores some representative methods, including ones based on `AMOC` [123], `PELT` [152], `Chow test` [24, 139], `GFT` [32, 84, 298], `SCP` [117], `CF` [255].

4.2.1 At most one change (AMOC)

The AMOC is based on finding a single change point for a time series X. This method is well-presented in Killick and Eckley [151]. Consider a time series X of size n. We can segment it into two parts at a time k $(X_{1:k}, X_{k+1:n})$, such that $1 < k < n-1$. If a change point exists at time k, the distributions of both segments are different. They are also different concerning the entire time series X.

Each candidate segment is modeled using Maximum likelihood estimation (MLE) using a particular PDF, such as Gaussian. The evaluation can be stated as a hypothesis test. The null hypothesis (H_0) corresponds to the scenario of not having a change point. The alternative hypothesis (H_1) is related to having a single change point at time k. Hinkley [123] proposed a statistical likelihood test (assuming a normal distribution) for a change in the mean. The likelihood ratio method computes the maximum log-likelihood (ML) under the null (Equation 4.1) and alternative (Equation 4.2) hypotheses. In both equations, the $p(\cdot)$ function stands for the probability density function associated with the distribution of the sequences. For Equation 4.1, θ_0 is the maximum likelihood of the parameters. Besides, Equation 4.2 has the maximum likelihood from both sequences separated in time k. Each sequence has estimated parameters (θ_1 and θ_2, respectively) [243].

$$\texttt{ML}(X) = log\left(p(X_{1:n}|\hat{\theta_0})\right) \tag{4.1}$$

$$\texttt{ML}(X,k) = log\left(p(X_{1:k}|\hat{\theta_1})\right) + log\left(p(X_{k+1:n}|\hat{\theta_2})\right) \tag{4.2}$$

From both Equations 4.1 and 4.2, it is possible to compute λ as described in Equation 4.3, which presents the optimal k from all possible change point locations. Assuming a threshold cp, the null hypothesis is rejected when $\lambda > cp$. In this case, k is a change point for X. The challenge is to establish the value for the cp threshold.

$$\lambda = 2\left(\left(\underset{k\in\{2,\cdots,n-1\}}{\arg\max}\ \texttt{ML}(X,k)\right) - \texttt{ML}(X)\right) \tag{4.3}$$

There are also alternatives regarding possible distribution assumptions and studied parameters. Thus, the test proposed by Hinkley [123] was later expanded to support changes in variance. Also, a more general approach can combine mean and variance tests [151].

Figure 4.3 presents the AMOC for changes in mean or variance method applied to the synthetic example of Figure 4.1. The identified change point is close to cp_{DE}.

4.2.2 Pruned exact linear time (PELT)

The PELT method is proposed by Killick et al. [152]. It is an exact approach for segmentation. It applies a dynamic programming approach and adopts some as-

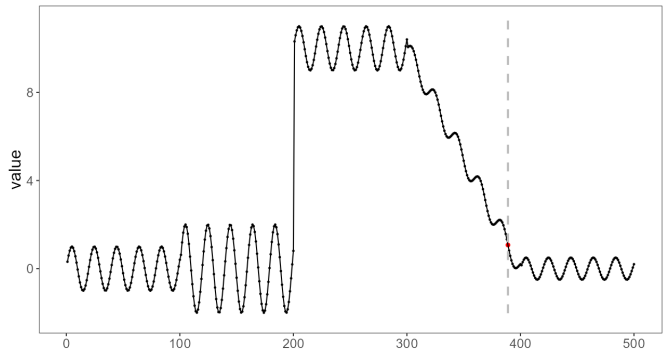


Fig. 4.3 `AMOC` applied to the synthetic time series for changes in mean and variance

sumptions to become efficient. The first assumption is that the number of change points increases linearly as the time series increases. The second is that the change points are spread through the time series rather than appearing close to one another. These innitatives lead to a complexity of $O(n)$ [151, 243].

Figure 4.4 presents an example of `PELT` applied to the synthetic time series of Figure 4.1 searching for change points, considering changes in mean and variance and Modified Bayesian information criterion (`MBIC`) as adopted penalty. The method detects 11 change points. It detects change points close to the actual labels in addition to seven points, which are false positives (between observations 300 and 400).

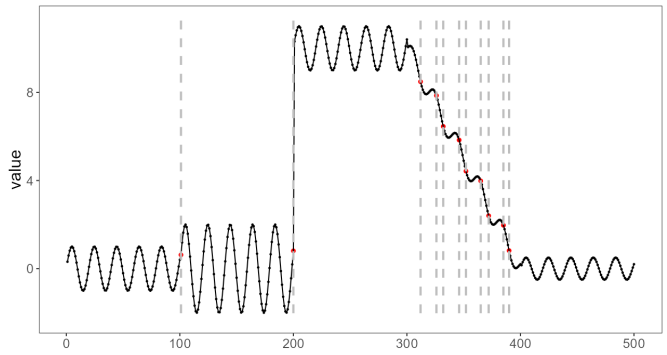


Fig. 4.4 `PELT` applied to the synthetic time series for changes in mean and variance

4.2.3 Chow test

Consider a time series X that can be modeled using linear regression, described in Equation 4.4, where y_t is the observation of the dependent variable, x_t is the independent variables, and ω_t is white noise. The only major difference is that β can vary according to time. The `Chow test` is concerned with testing the null hypothesis of no structural change, *i.e.*, $H0 : \beta_t = \beta_0$ against the alternative that the coefficient β_t indeed varies over time [297, 298].

$$y_t = \beta_t x_t + \omega_t \tag{4.4}$$

Several tests have been created to evaluate the hypothesis. One of the main classes is based on F statistics [296]. These F-statistics-based tests primarily target the alternative hypothesis of a single unknown breakpoint. Due to their ease of interpretation, these tests are widely used, with popular examples being the `Chow test` and $supF$.

When the potential change point is known, the original idea for a structural change test was introduced by Chow [70]. The approach involved fitting separate regressions for two subsamples defined by the change point t_k and rejecting the null hypothesis when the F statistic exceeded a certain threshold [298]. Assume n is the number of time series observations, and r is the number of regressors in the linear model. An Ordinary least squares (`OLS`) model is fitted for the observations before and after the potential change point t_k ($2r$ parameters are estimated) for computing the F statistic. The error sum of squares is computed (ESS). Another `OLS` model for all observations with a restricted sum of squares (RSS) is computed (r parameters are estimated). The F statistic is given by Equation 4.5 [298].

$$F = \frac{(RSS - ESS)}{ESS/(n - 2r)} \tag{4.5}$$

The drawback of the `Chow test` is that the change point must be known in advance, which is often not the case. On the other hand, the $supF$ test, also based on F (Chow) test statistics, does not require a specification of a particular change point. It is based on the extension of the `Chow test`, where F statistics F_t are calculated for all potential change points in an interval, for $t_i < t_a \leq t \leq t_b < t_f$, where t_i and t_f represent the first and last available time points, and t_a and t_b define the evaluated interval, respectively. Next, the series of F statistics can be aggregated into a single test statistic using the $supF$ statistic suggested by Andrews [24].

In that case, the null hypothesis is rejected when the maximal F statistic gets too large. F statistics boundaries can be computed such that the asymptotic probability that the supremum of the statistics F_t (for $t_a \leq t \leq t_b$) exceeds this boundary is α. As the F statistics cross their boundary, there is evidence for a structural change (at the level $\alpha = 0.05$). The actual breakpoint (change point event) estimate is given by minimizing the `RSS`. Alternatively, the point that maximizes the sequence of F statistics can also serve as an estimate for the breakpoint, as it is equivalent to the minimum `OLS` estimator of the breakpoint in a 2-segment partition [298].

Figure 4.5 presents an example of `Chow test` applied to the synthetic time series of Figure 4.1. As expected, it detected a single change point. It detected the change point close to 200, which differs from the change point obtained by `AMOC`.

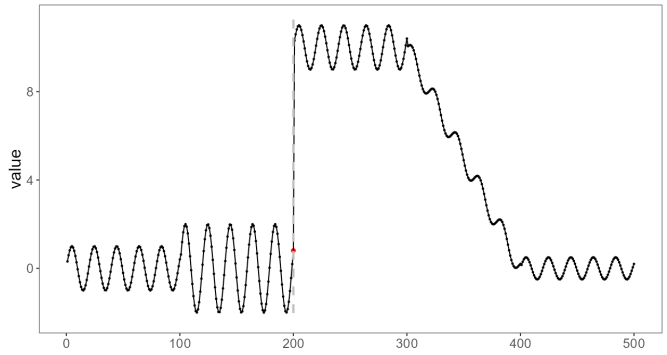


Fig. 4.5 `Chow test` applied to the synthetic time series

4.2.4 Generalized fluctuation test

`GFT` represents an alternative approach for detecting change points based on regression, which is defined by Equation 4.4. What sets `GFT` apart is its ability to identify multiple change points simultaneously, as it does not assume a specific pattern of deviation from the null hypothesis. This method involves adjusting a model to the data and extracting an empirical process that captures variations in residuals or parameter estimates. When the null hypothesis holds, these variables are subject to the constraints of central limit theorems, leading to their boundaries being defined with a fixed probability denoted as α. However, under the alternative hypothesis, the fluctuations in the process generally increase [297, 298].

As an example, consider an `OLS`-based Cumulative sum control chart (`CUSUM`) test, as introduced by Ploberger and Krämer [218]. This test relies on cumulated sums of standard `OLS` residuals (ω_i), as defined by Equation 4.6. In this case, $W(\cdot)$ represents standard Brownian motion. When observing the limiting process of $W_n^0(u)$, it resembles the standard Brownian bridge $W^0(u) = W(u) - uW(1)$. In the context of a single-shift alternative, this process is expected to peak around the change point [297, 298].

$$W_n^0(u) = \frac{1}{\hat{\sigma}\sqrt{n}} \sum_{i=1}^{\lfloor nu \rfloor} \hat{\omega}_i, (0 \leq u \leq 1) \tag{4.6}$$

Figure 4.6 presents an example of GFT applied to the synthetic time series of Figure 4.1. It can detect some labeled change points. Since changes in the mean drive the method, the labeled change point related to variance change was not captured.

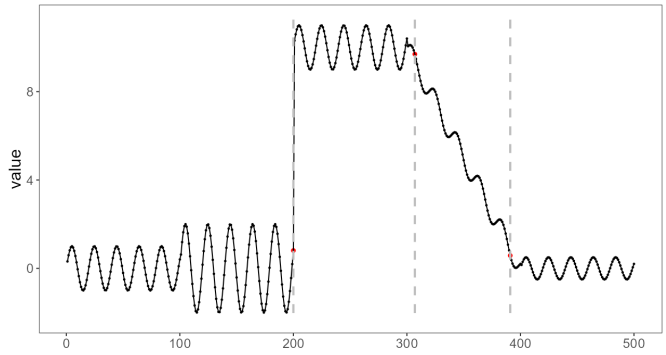


Fig. 4.6 GFT applied to the synthetic time series

4.2.5 Seminal change point

The seminal regression method (SCP) for detecting change points has become a reference in the literature [117] for SW based methods. It adopts the following strategy: for a time point in a sliding window, models are adjusted to segments before and after the point. Then, it is determined that there is a change point if the total fitting errors are significantly reduced compared to the case where there is no change point. The method adopts linear regression as a model for adjusting the data in its simplest version. The adjustment error is measured in terms of square errors [255].

Figure 4.7 illustrates the strategy adopted by the method. The blue line presents the regression for the entire window. The candidate observation is marked in gray. The regression for the observations before and after the candidate observation are marked in green and red, respectively. Suppose the sum of residuals for all observations concerning blue regression is significantly higher than that of residuals from before and after regressions. In that case, the candidate observation is a change point.

Figure 4.8 presents an example of SCP applied to the synthetic time series of Figure 4.1. The SW size is equals 30. The zone of volatility change influenced the method. However, only the first and last detections were close to the labeled events.

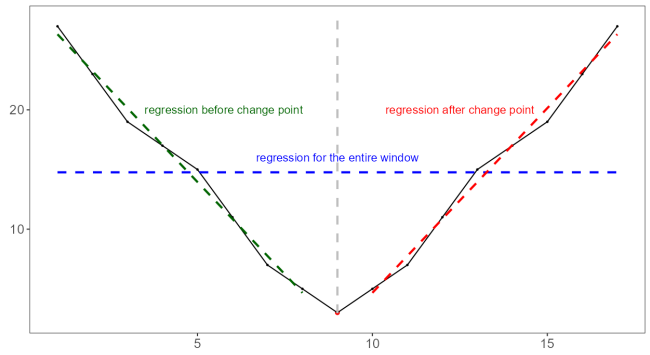


Fig. 4.7 Illustration of the strategy adopted by the method proposed by Guralnik and Srivastava [117].

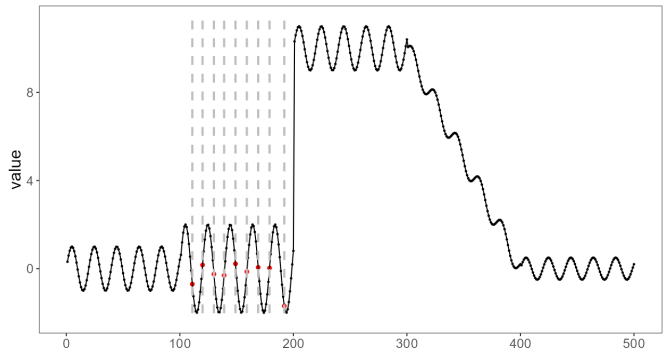


Fig. 4.8 SCP with SW size equals 30 applied to the synthetic time series

4.2.6 Change finder

The creation of SCP enabled the development of many other SW based methods, such as the CF [255]. CF is composed of two phases. In the first phase, given a time series X, a model α is adjusted using an autoregressive model, such as AR. Let $\omega_i = (x_i - \hat{x}_i)^2$, which is the squared residual from the model. Outliers in this residual series W are directly marked as anomalies.

The general insight of CF is to compute a moving average for the residual series W, represented as Z time series. This procedure provides an inertia for the residuals. Z is modeled again using an autoregressive model, such as AR. It leads to a $\hat{Z}$ series. In this case, the residual series R, such that, $r_i = \zeta_i - \hat{\zeta}_i$ is produced. Outliers in R are considered as change points.

Figure 4.9 presents an example of CF using ARIMA applied to the synthetic time series of Figure 4.1. The SW size is equals 30. It can detect some labeled change points.

Also, points marked in red were considered anomalies. Not all labeled change points were detected. Some were misjudged as anomalies.

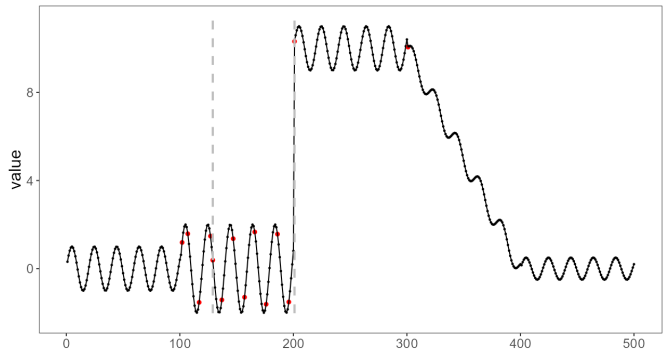


Fig. 4.9 `CF` using `ARIMA` with `SW` size equals 30 applied to the synthetic time series

Some change point detection methods are publicly available at Harbinger (see Section 8.3).

4.3 Concept drift

Change point detection is related to monitoring concept drifts in the time series context. Both problems focus on observing considerable changes in time series behavior. The latter focuses on discovering if a change occurred, whereas the former focuses on finding when it occurred.

Consider a multivariate time series D organized into batches (non-overlapping segments) with a varying distribution χ[1]. A concept for D at a batch i (SD_i) is defined as the probability of the distribution χ for D. It is described in Equation 4.7.

$$concept(SD_i) = p(\chi_i) \tag{4.7}$$

A concept drift between batches i and $i+1$ is defined as a difference (with statistical significance) between the probabilities $p(\chi_i)$ and $p(\chi_{i+1})$. It is described in Equation 4.8 [278].

$$p(\chi_i) \neq p(\chi_{i+1}) \tag{4.8}$$

[1] Although concept drift is a broad area, the formalization and examples are contextualized to time series analysis.

Figure 4.10 presents a taxonomy for concept drift. It is organized according to the type of drift. Detectors are organized according to their type and method. The taxonomy also presents the main drift-handling strategies commonly adopted.

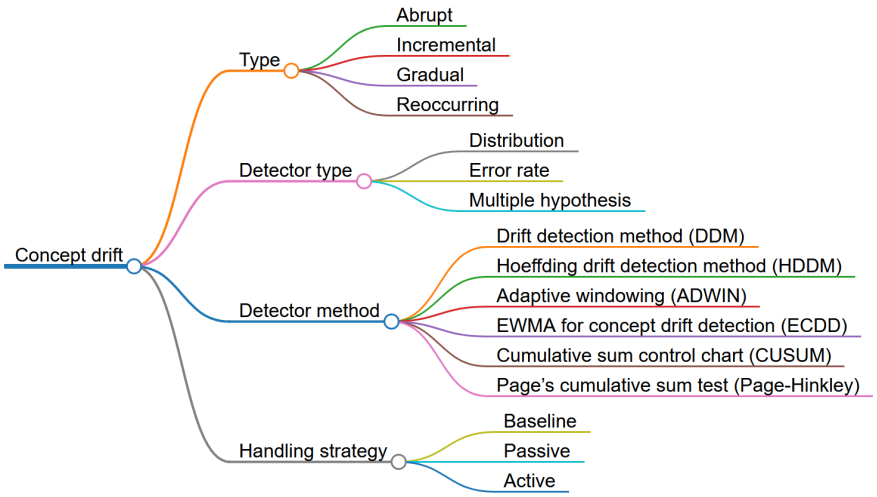


Fig. 4.10 Taxonomy of concept drift. It includes the type of drift, drift detector type, detector method, and drift handling strategy

The type of concept drift is classified as abrupt, incremental, gradual, and reoccurring. Figure 4.11 depicts an example of these four types of concept drift. The time series is organized into four batches ($SD_1, \ldots, SD_4$) of size eight. Observations marked in red correspond to change points, whereas dashed black lines correspond to a change in the concept between consecutive batches.

The concept becomes significantly different in the following batches in the abrupt scenario. Figure 4.11a presents an example of abrupt drift from batch SD_2 to SD_3. In the incremental scenario, the drift occurs, but observations smoothly change from one concept to another [132]. Figure 4.11b presents an incremental drift from batch SD_2 to SD_4.

Gradual drift, represented in Figure 4.11c, is the scenario where the concept does not directly change from one concept to another. It may go back and forth until it stabilizes into the new concept. The concept starts to change at batch SD_2 but only stabilizes at the new concept at batch SD_4. Finally, Figure 4.11d presents an example of reoccurring drift. In this case, the time series shows a periodic behavior that switches from one concept back to the previous one. This occurs between batches SD_1 to SD_2, SD_2 to SD_3 and later from SD_3 to SD_4 [33].

Due to changes in time series distribution, it is not uncommon to have more events in the time series when the concept drifts occur. Besides, event detection models might be vulnerable to the presence of concept drifts. Detecting and handling drifts is a relevant subject associated with event detection. It enables a specific action

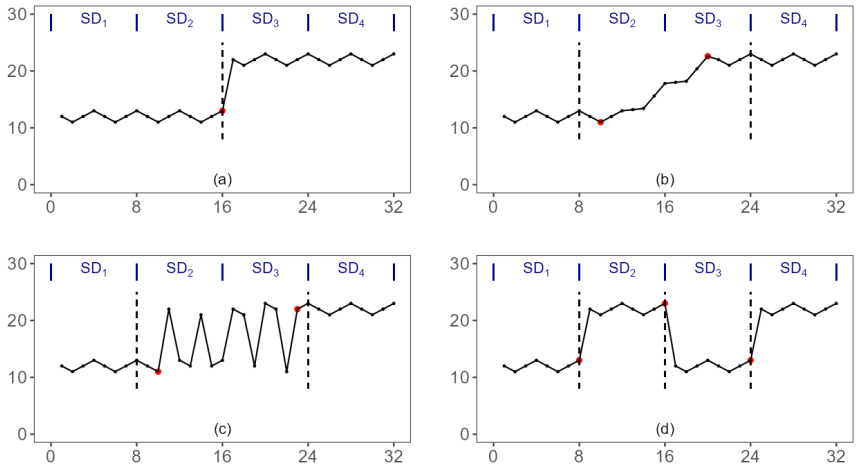


Fig. 4.11 Different types of concept drift: abrupt (a), incremental (b), gradual (c), reoccurring (d) (adapted from Bayram et al. [33]).

to avoid increasing errors in online learning systems after drift is observed [278]. There are two main categories of drift detection [33]: data distribution, error rate, and multiple hypothesis tests.

Data distribution-based methods use statistical inference and analysis of feature distribution to detect significant output class proportion changes concerning input variables. Once a new batch arrives, the distribution of the batch is compared with the distribution of the data used to train the models [132]. They are connected to change point detection methods based on distribution.

Error rate methods use a learning algorithm (statistical or machine learning [135]) and indicate a drift based on the error rate of prediction results. Once a new batch arrives, an inference is made. The predictions are used and later evaluated if they are correct or not. A drift is assumed to exist once the error obtained from this batch is significantly higher than the previously observed errors [103].

Finally, the multiple hypothesis tests are based on a combination of methods. They can be assessed according to the data distribution, error rate, or both. They are built as an ensemble of methods. In other words, they can be parallel-checked, and the most voted result indicates the presence of a drift. Alternatively, the test can be structured as a stack. A fast detector with more chance of having false positives triggers another detector, which is more time-consuming but potentially more accurate [175].

Consider a multivariate time series (D) partitioned into batches of size b. SD_1 and SD_n correspond to the first and last batches of D, respectively. Yet, a batch of size b at time i is formally defined as $SD_i =< D_{(i-1)\cdot b+1}, \cdots, D_{i\cdot b} >$, such that $|SD_i| = b$. It can target the detection and handling of concept drifts [132].

From these concepts, it is possible to define three strategies to handle concept drift: (i) *baseline*; (ii) *passive*; (iii) *active*. In the *baseline* strategy, a model is

built using the first batch. The trained model is continuously used. No action is taken when drift occurs, and the trained model might increase its error when inferencing newer batches. It corresponds to Figure 4.12a, where the first batch (SD_1) is used for training a model (indicated as a red square) for inferencing all other batches (SD_2 to SD_n).

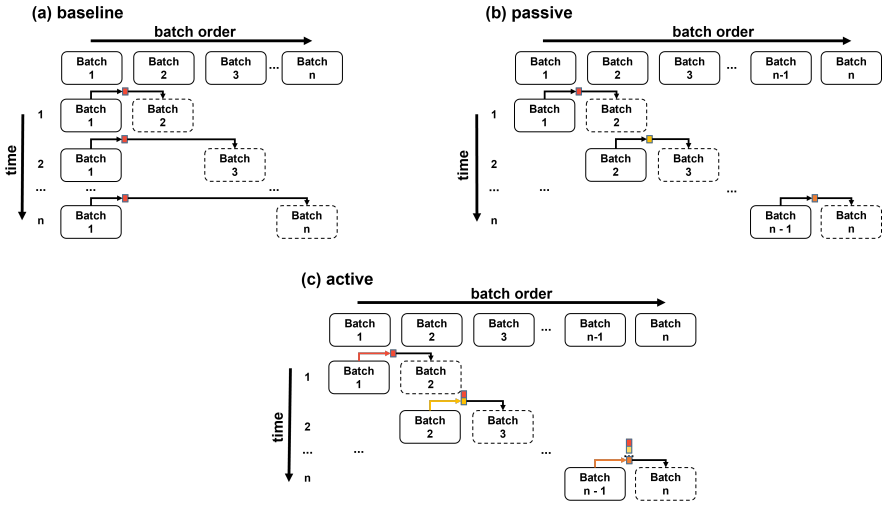


Fig. 4.12 Drift handling strategies. (a): *baseline*; (b): *passive*; (c): *active*

In a *passive* strategy, it is assumed that drift always occurs. Thus, batch SD_i is used for training the model to inference batch SD_{i+1}. This scenario corresponds to Figure 4.12b. Models are constantly updated (they are presented in different colors). The drawback of this approach is that it might retrain models, even if no drift occurred in the dataset [104].

Finally, the *active* strategy applies drift detection whenever introducing a new batch. If no drift is detected, the previously trained model is still used. However, a new model is built using previous batches if drift is detected. Figure 4.12c depicts this scenario. A new model (in orange) is used if drift occurs between batches two and three. Otherwise, the previous model (presented in red) is preserved. In this strategy, two extreme scenarios may occur. The same model can be used from the first batch to the last one, resembling the baseline strategy. The difference is that a model is only kept if no drift has been detected. Conversely, continuous retraining may occur between batches, resembling the *passive* strategy. Again, such a decision is based on whether drift is observed whenever a new batch is introduced [132]. Moreover, some techniques may change the size of the batches using soft computing (fuzzy systems) to identify drifts and adapt accordingly [27, 176, 305].

4.4 Concept drift detectors

There is a broad range of concept drift detectors. This section covers representative methods from which one can understand the principles of other novel methods [119, 175]. Figure 4.13 presents the overall framework for drift detection. It encompasses four steps: data retrieval, data modeling, dissimilarity measure, and hypothesis test. The data retrieval step aims to establish how data is collected from streaming and passed to drift detection. The data modeling targets creating a model to support drift detection. It might demand extracting features and conducting transformations that might increase the performance or accuracy of drift detectors. The dissimilarity measures aim to measure the difference between previous and new batches. It quantifies the severity of the drift to support the hypothesis test. The last step evaluates the statistical significance of the change observed in the dissimilarity step to characterize a drift.

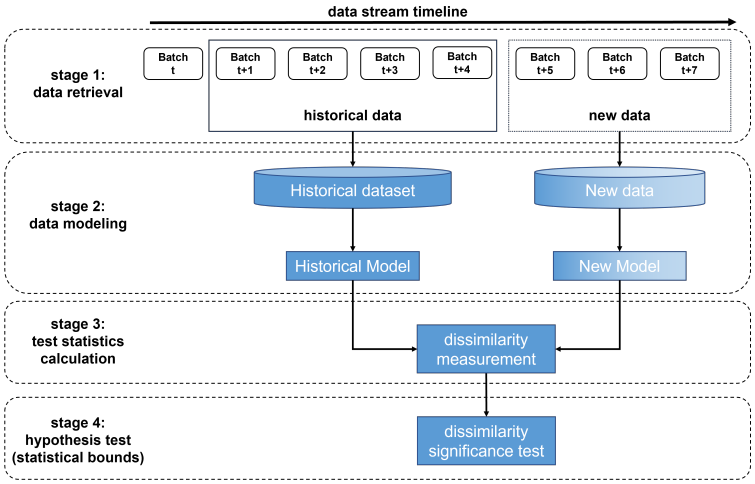


Fig. 4.13 General framework for drift detection (adapted from Lu et al. [175])

4.4.1 DDM

The Drift detection method (`DDM`) (Drift Detection Method) uses a binomial distribution to model the behavior of a random variable that provides the number of classification errors in a batch of size b. `DDM` calculates, for each instance i in the data stream, the probability of misclassification p_i and its standard deviation σ_i. If the distribution of samples is stationary, the model maintains its performance [108]. Suppose the error rate of the learning algorithm increases significantly. In that case,

it suggests changes in the class distribution, indicating that the current model is inconsistent with the current data, thus providing a signal of concept drift.

While monitoring a time series, DDM calculates the values of p_i for each instance. When $p_i + \sigma_i$ reaches its minimum value, it stores $p_{\min}$ and $\sigma_{\min}$. Then, DDM checks two conditions to detect whether the system is in the alert or concept drift level: the alert level is triggered when $p_i + \sigma_i \geq p_{\min} + 2\sigma_{\min}$. Beyond this level, it stores instances anticipating a possible concept drift. The concept drift level is triggered when $p_i + \sigma_i \geq p_{\min} + 3\sigma_{\min}$. At this level, DDM resets the variables ($p_{\min}$ and $\sigma_{\min}$) and the induced model. Subsequently, a new model is built using the instances stored since the alert level has been triggered.

DDM performs well in detecting gradual and abrupt changes but struggles to detect drifts in incremental scenarios. In this case, storing samples can be accumulated over time before the concept drift level is triggered [108].

4.4.2 HDDM

The Hoeffding drift detection method (HDDM) [100], like DDM, Fast Hoeffding drift detection method (FHDDM) [215] and McDiarmid Drift Detection Method (MDDM) [216], belongs to a group of methods that include drift detection methods based on prediction error analysis. The mechanics of HDDM involve monitoring an estimated performance based on classifier errors using Hoeffding's inequality. It adapts step four of the general framework for drift detection (depicted in Figure 4.13) to introduce Hoeffding's inequality.

Consider a batch SD of size b of observations x_i, such that $x_i \in [l_i, h_i]$, with $i \in \{1, \cdots, |SD|\}$. Consider also $\overline{X}$ as the computed mean of observations. For any ϵ threshold ($\epsilon > 0$), Equation 4.9 indicates a boundary for the probability of the computed mean being different from the expected mean ($E[\overline{X}]$, computed from previous batches) by a ϵ threshold.

$$Pr\{\overline{X} - E[\overline{X}] \geq \epsilon\} \leq e^{\frac{-2b^2\epsilon^2}{\sum_{i=1}^{n}(h_i - l_i)^2}} \tag{4.9}$$

Equation 4.10 enables estimating the error ϵ for a significance level of δ. HDDM establishes bounds to check the distance or inequality between the error distributions during predictions. In other words, the difference between the number of false positives or false negatives at batch i is compared to those in a measurement taken at $i + 1$ to indicate concept drift.

$$\epsilon_\delta = \sqrt{\frac{1}{2n} ln(\frac{1}{\delta})} \tag{4.10}$$

4.4.3 ADWIN

Adaptive windowing (ADWIN) [37] uses variable-sized sliding windows, recalculated online based on the observed rate of change in the data within these windows. The window (W) dynamically expands when there is no clear change in the context and contracts when a change is detected. If a change is identified, it eliminates an outdated portion of the window.

ADWIN relies on the use of Hoeffding's bound [36] to detect drift. It compares the average difference between two sub-windows ($W0$ for older instances and $W1$ for recent instances) within W. If the average difference is statistically significant, then ADWIN removes all instances from $W0$ that are considered to represent the old concept and only retains $W1$ for the next test.

4.4.4 ECDD

EWMA for concept drift detection (ECDD) applies an inertia-based drift detection. The Exponentially weighted moving average (EWMA) is an inertial follower adopted for monitoring time series. The exponential property progressively gives less weight to older data points as they move back in time. A parameter λ determines the rate at which older data enters the statistic. The EWMA is represented in Equation 4.11 as z_t. From the properties of EWMA, the mean and standard deviation for the inertial time series Z at a time t is characterized in Equation 4.12. The μ_{z_t} corresponds to the time series mean (μ_{X_t}) at time t [230].

$$z_t = \begin{cases} \mu_t, & t = 1 \\ \lambda \cdot \mu_t + (1 - \lambda) \cdot \mu_{t-1}, & t > 1 \end{cases} \tag{4.11}$$

$$\mu_{z_t} = \mu_{X_t}, \sigma_{z_t} = \sqrt{\frac{\lambda}{2 - \lambda}(1 - (1 - \lambda)^{2t})}\sigma_{X_t} \tag{4.12}$$

The EWMA is a preprocessing step for the general framework depicted in Figure 4.13. For the sake of ECDD, before a change point $\mu_t = \mu_a$, the time series varies around the follower z_t. When a change occurs, the value of μ_t changes to μ_b and z_t gradually moves towards μ_b. In this case, it is possible to establish a control limit L to determine how much z_t must diverge from μ_a before characterizing a concept drift. Such limit is established by Equation 4.13.

$$z_t > \mu_a + L \cdot \sigma_{z_t} \tag{4.13}$$

4.4.5 CUSUM

Cumulative sum control chart (`CUSUM`) is a sequential analysis technique [103]. `CUSUM` is designed to detect changes in distribution and propose a criterion for deciding when to take corrective action. As the name suggests, the `CUSUM` involves the calculation of a cumulative sum (hence the term sequential) time series Z. Considering the general framework, the hypothesis test is verified when Z significantly deviates from zero.

The time series Z uses an additional quality time series (ω). An example of ω is the residual, *i.e.*, difference between the model and time series. In this case, observations from a time series z_t are assigned from previous values z_{t-1}, ω_t, and a magnitude parameter δ. It is described in Equation 4.14. When the value of z_t exceeds a certain predefined threshold, it is identified as a change point, characterizing a concept drift.

$$z_t = \begin{cases} 0, & t = 1 \\ \max\left(0, z_{t-1} + \omega_t - \delta\right), & t > 1 \end{cases} \tag{4.14}$$

4.4.6 Page-Hinkley

The Page's cumulative sum test (`Page-Hinkley`) test is a variation of `CUSUM` [103]. It considers a cumulative time series Z expressed as z_t. It accumulates the difference between the values x_t and the mean of X up to time t ($\overline{x}_t$). Equation 4.15 characterizes the `Page-Hinkley` test (PH), where $\overline{x}_t = \frac{\sum_{i=1}^{t} x_i}{t}$ and δ correspond to the magnitude of the allowable changes.

$$z_t = \sum_{i=1}^{t} \left(x_i - \bar{x}_i - \delta\right) \tag{4.15}$$

This difference signals a drift when it exceeds the threshold δ. The threshold δ depends on the acceptable false alarm rate. Increasing δ results in fewer false alarms (false positives) but may lead to missed (false negatives) or delayed change detection.

Figure 4.14 presents the concept drift detection of the example depicted in Figure 4.7 using `Page-Hinkley`. As expected, the method could only detect abrupt changes in mean. It also took some time to find it. The drift is discovered a little bit after observation 200.

Some concept drift detection methods are publicly available at Harbinger (see Section 8.3).

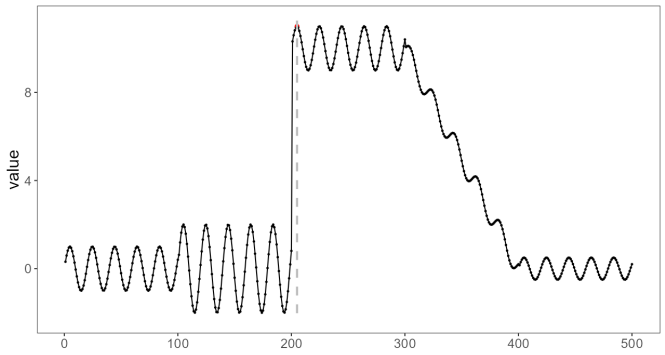


Fig. 4.14 Concept drift detection using `Page-Hinkley`

4.5 Advanced topics in change points

There are some surveys and books regarding change points. Aminikhanghahi and Cook [21] present a survey for change point detection method. Also, Truong et al. [265] review algorithms of change points for multivariate time series. Besides, Chen and Gupta [64] present the most parametric methods for detecting change points. Sayed-Mouchaweh and Lughofer [238] present a comprehensive review of recent developments and important methodologies associated with building models in a non-stationary scenario.

Chib [67] provides a Bayesian detection approach for multiple change points. It adopts a discrete state Markov process. The transition probabilities are constrained so that the state variable is used to trigger the detection of change points. Bai and Perron [32] apply dynamic programming principles to compute multiple change points. Similarly, Alippi and Roveri [19] adopt an approach of detecting changes in a non-stationary process to support change point detection.

There are some initiatives for detecting change points in multivariate time series [220]. Hlávka et al. [124] apply empirical analysis to support detecting change points. Very similar to multivariate time series are the spatial-temporal ones. Chen et al. [65] proposed a change point incorporating multinomial logistic regression models into a multidimensional latent Dirichlet allocation. Such grouping enabled distinguishing meaningful activity categories, such as commuting publicly available bike sharing in New York. Another spatial-temporal change point method was proposed by Kim et al. [153], who could distinguish tactically intended formation and role changes from temporary changes in soccer matches.

Change points and anomalies are relevant for control systems. Kammammettu and Li [140] propose a novel algorithm for change point and fault detection using Kantorovich Distance. Flórez et al. [99] proposed a method called CatSight. It can detect change points in a spatial-temporal multivariate time series. CatSight was designed to support industrial monitoring.

Gedda et al. [106] provide a change point method to support other data analytics activities. It works as a preprocessing step for other data mining functions. It is worth mentioning that most existing methods are not driven to distinguish between anomaly and change point. Like Takeuchi and Yamanishi [255], Liu et al. [170] proposed an unsupervised method to simultaneously detect anomalies and change points for time series with concept drift.

As mentioned earlier, online change point detection is relevant since it requires detecting the changes in time series as soon as possible to enable timely reaction to these changes [276]. Desobry et al. [81] proposed a kernel change detection method to support online scenarios based on soft margin single-class SVM. Belcaid and Belkbir [34] also proposed a change point method that can be applied in both online and offline scenarios. It is based on a globally constrained energy function.

Babüroğlu et al. [31] provide a bibliometric review of drift from 1980 until nowadays, enabling a comprehensive view of the evolution of the area. Hoens et al. [126] discuss the scenario of handling concept drift considering an imbalanced classification problem. Gama et al. [104] and Khamassi et al. [149] review adaptive learning processes on data stream scenarios. Pratama et al. [221] present incremental learning techniques using recurrent fuzzy neural networks. Bayram et al. [33] review model degradation under concept drift. Han et al. [119] compare the passive versus active strategies to tackle the concept drift problem.

Benchmarking drift detector methods is an important subject and has been given attention recently. There are initiatives to explore various methods and different types of drifts (gradual, abrupt, and incremental) [108, 232]. Other works focus on domains, such as flight [107] and security [112].

Concept drift is a data-evolving problem. Certainly, time is a relevant dimension associated with drift. Another very important dimension is space. The combination of time and space provides spatial-temporal data. Some works focus on studying drifts in this category of problems [20, 168]. Flórez et al. [99] adopt PCA to reduce the dimension of multivariate time series to apply spatial principals to detect concept drifts.

Despite the high number of applications of deep learning to build prediction models, when it comes to addressing concept drift, there are many challenges to address, such as computational cost and the absence of new data to adapt deep learning models. Xiang et al. [284] summarizes concept drift adaptation methods under the deep learning framework. Particularly considering time series, there are worth mentioning works [185, 293]. Herbert et al. [122] discuss how to orchestrate the training and update of recurrent models under scenarios of concept drift. Yuan et al. [295] discuss concept drift adaptation, particularly considering the data required to rebuild deep learning models under abrupt, gradual, and recurrent drift scenarios. Some initiatives apply transfer learning as a way to support the adaptability of deep learning to the concept drift scenarios [1, 86, 274].

Finally, some works combine studies of anomalies, change points, and concept drift. Park [208] provides an integrated review of anomalies and concept drift. Tian et al. [262] provide an anomaly detector method using machine learning that can address non-stationary scenarios through drift adaptation. Abdel Wahab [1] applies

PCA method to study the change in the variance of the features across the intrusion detection data streams. It adopts an online outlier detector capable of observing divergence between historical and recent observations. Liu et al. [170] built an unsupervised method to simultaneously detect anomalies and change points for time series with concept drift.

Chapter 5
Motif discovery

Motif discovery refers to identifying recurring sequences in a time series. An important property regarding motifs is that the repeated subsequence is unknown and is discovered when scanning the time series. Section 5.1 provides the basic concepts related to motifs. Motif discovery methods are influenced by how time series are prepared and how sequences are considered similar. These concepts are respectively addressed in Sections 5.2 and 5.3. In addition, state-of-the-art motif discovery techniques are explored in Section 5.4. Finally, Section 5.5 describes advanced topics related to motif discovery.

5.1 Basics of motifs

In this chapter, we introduce the general concepts related to motif discovery. An intuitive definition of motif was provided in Section 1.2.5. This section formalizes the definition, presenting the concept of sequence occurrences in a time series. From this definition, the definition of a motif can be derived.

Let q be a sequence of size p. Consider W be the set of sliding windows of size p obtained from a time series X. As described in Equation 5.1, the set of occurrences of q in X is the set of windows w_i of W that are similar to q [43].

$$occurs(q, X) = \{w_i\} \mid \forall w_i \in W, w_i \approx q \tag{5.1}$$

The sequence q is a *motif* in X with support σ, if and only if q occurs in X at least σ times, described in Equation 5.2. The length p of a motif q is also known as word size [43].

$$motif(q, X) \iff |occurs(q, X)| \geq \sigma \approx q \tag{5.2}$$

Some definitions of motifs require that their occurrences should not be trivial. No sequence presented in $occurs(q, X)$ should be overlapped regarding the time they occur [191]. The most significant motif (1-motif) in X is the most occurred subse-

quence, disregarding the trivial matches formed by overlapping the subsequences. Consequently, the k-motif is the k-th most occurred motif in the time series [166].

In this context, another aspect that should be considered is the level of importance of the motif in a time series. If the occurrences of a motif q are too high, it loses its expected property of being previously unknown. Initially, the subsequences are characterized as motifs. However, they can be classified as false motifs because they have little relevance and produce little or no information of interest [69].

Figure 5.1 organizes the main concepts interfering with motif discovery. The taxonomy includes time series indexing, similarity functions, and discovery methods. They are described in the following sections.

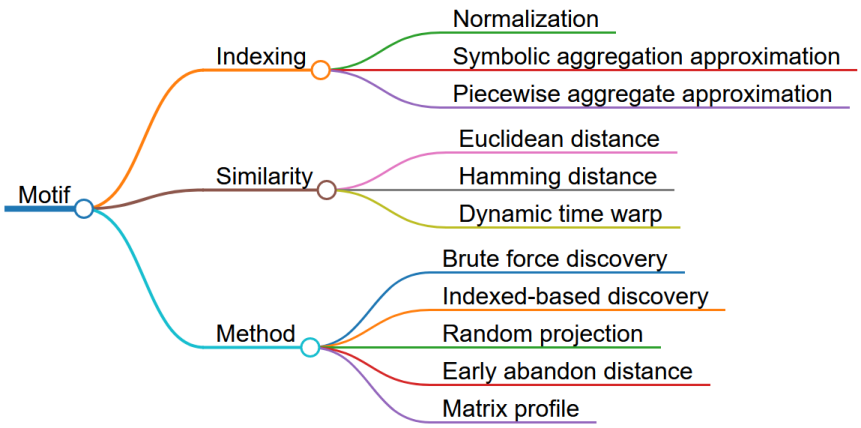


Fig. 5.1 Basic taxonomy for motif discovery

5.2 Time series indexing

Many methods proposed in the literature to discover motifs in time series are computationally intensive [212]. Due to that, many works aim to improve the effectiveness of motif discovery and reduce the computational resources needed. Such a process requires data preprocessing, such as normalization and encoding, before running the motif discovery algorithms to increase the performance and precision of results [191].

5.2.1 Normalization

Different methods exist to compare time series, such as morphological comparison [172]. It is necessary to verify some properties, such as scale, to apply such methods to compare them. In this way, normalization becomes a fundamental step in data preprocessing to enable the effectiveness of time series comparison methods [148].

One of the most common normalization methods adopted in motif discovery is the `Z-Score`. Equation 2.21 (Section 2.3.7) describes `Z-Score` normalization, where x_i is an observation of the time series X, μ_X is the mean, σ_X is the standard deviation of X, and $\hat{x}$ is the normalized observation [202]. The normalized time series has mean and variance, respectively, equal to *zero* and *one*.

5.2.2 Symbolic aggregation approximation

Time series observations are usually real values. For motif discovery, processing directly numerical representation might not be an efficient way to adopt [78]. Some discretization methods, such as `SAX`, have been proposed. It consists of representing a range of values as symbols.

`SAX` partitioning the time series into ranges such that each range is associated with a particular symbol [166]. The `SAX` alphabet size defines the number of partitions for the domain. All values are replaced by their respective associated symbol. Give a time series X with n observations. Consider an alphabet $(a_1, \cdots, a_m)$ of size m, the values of time series X are divided into m ranges $([-\infty, \beta_1], \cdots, [\beta_{m-1}, \infty])$ according to Gaussian distribution function (with different sizes, but same probability), such that each value x_i is mapped to an alphabet value a_k, where $1 \leq k \leq m$ [165]. Equation 5.3 describes the transformation from a time series X to a SAX-based time series Y using an alphabet of size m. The overall complexity of `SAX` is $O(n + m \cdot log(m))$.

$$y_t = \begin{cases} a_1, & x_t \in [-\infty, \beta_1], \\ a_2, & x_t \in [\beta_1, \beta_2], \\ \cdots & \\ a_m, & x_t \in [\beta_{n-1}, \infty]. \end{cases} \tag{5.3}$$

Figure 5.2a presents the normalized `MGT`. It also presents three intervals for converting it into the SAX alphabet of size 3.

5.2.3 Piecewise aggregate approximation

The Piecewise aggregate approximation (`PAA`) is a type of temporal aggregation. Given a time series X $(< x_1, x_2, x_3, \ldots, x_n >)$ with n observations. The number of observations is reduced to n' such that X' $(< x'_1, x'_2, x'_3, \ldots, x'_{n'} >)$. Commonly,

each observation in X' is the mean of equally time observations concerning X. In this case, it is possible to assume that X is partitioned in time into k segments, such that $k = \frac{n}{n'}$. Equation 5.4 characterizes such transformation [285]. The overall complexity of `PAA` is $O(n + k)$.

$$x'_j = \sum_{t=n'(j-1)+1}^{n' \cdot j} \frac{x_t}{n'} \tag{5.4}$$

Figure 5.2b is an example of a `PAA` applied to `MGT`. Another method is Maximum entropy partitioning (`MEP`) [78], which is similar to the previous method, but instead of splitting time uniformly, the division considers some criteria to address important intervals. Besides, it is common to apply `SAX` [165] after applying `PAA`.

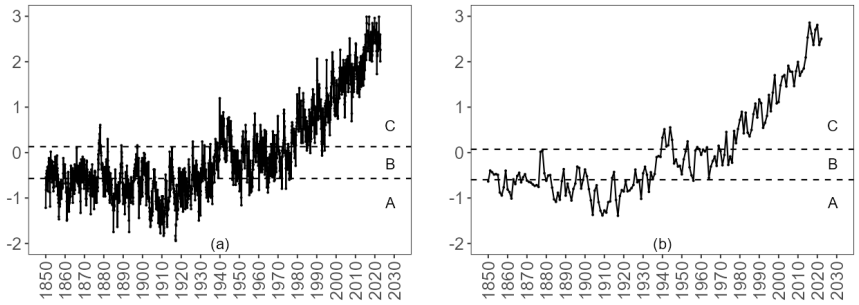


Fig. 5.2 SAX encode in `MGT` (a), SAX applied to PAA on top of `MGT` (b)

5.3 Similarity measures

In motif identification, several authors have proposed methods to discover motifs efficiently [191, 264]. The identification of motifs is based on evaluating the similarity between the subsequences of the time series, identifying, thus, the sets of candidates for motifs. Based on this, several algorithms have been proposed to solve this problem [69, 157]. Similarity measures are techniques based on the distance between time series and sequences. They can be divided into Euclidean distance (`EDist`), Hamming distance (`HDist`), and Dynamic time warping (`DTW`). These metrics are presented as follows.

5.3.1 Euclidean distance

The most widely used distance measure in time series analysis is the `EDist` [143], which measures the distance between points from different sequences. Given two sequences q and q', both of size p, a Euclidean distance $EDist(q, q')$ is obtained by the Equation 5.5.

$$EDist(q, q') = \sqrt{\sum_{i=1}^{p} (q_i - q'_i)^2} \tag{5.5}$$

5.3.2 Hamming distance

The `HDist` between two sequences of equal length is the number of positions at which the corresponding observations differ. In other words, it measures the minimum number of substitutions required to change one sequence into the other. A $\texttt{HDist}(q, q')$ is obtained by the Equation 5.6 from the sum of distances between each Hamming item (`HItem`). The `HItem` is described in Equation 5.7. The distance between two items (x and y) is zero when equal and one otherwise.

$$HDist(q, q') = \sum_{i=1}^{p} HItem(q_i, q'_i) \tag{5.6}$$

$$HItem(x, y) = \begin{cases} 1, & x \neq y, \\ 0, & x = y. \end{cases} \tag{5.7}$$

5.3.3 Dynamic time warp

When observing the same phenomenon several times, it might be expected to occur a difference in elapsed time [52]. `EDist` is not robust for lag differences in time series [83]. A widespread and widely used method to address this issue is the `DTW`. This method uses a dynamic programming approach to align time-phased time series. To compare two different time series using `DTW`, it is necessary to build a matrix composed of the quadratic distances between two points. A path through the matrix that minimizes the total cumulative distance is traversed to find the best match between the two series.

Given two sequences q and q', respectively, with sizes p and p', the `DTW` measures the editing needed to transform q and q'. The editing operations for each pair of observations between q and q' are: (i) replacing the next element of q to the next element of q'; (ii) elongating an element, *i.e.*, the next element of q is matched to the last element of the already-matched prefix of q' or vice versa. The dynamic pro-

gramming in DTW computes the minimal transformation cost while filling the entries of an $p \times p'$ matrix ($dtw^{q,q'}$). Assuming no elongation operation and the univariate time series scenario, Equation 5.8 describes the DTW between two sequences q and q'. Since it uses dynamic programming, further lookup on the pre-computed matrix is used and expressed in Equation 5.9.

$$DTWDist(q, q') = dtw^{q,q'}_{p,p'} \tag{5.8}$$

$$dtw^{q,q'}_{i,j} = min(|q_i - q'_j| + dtw^{q,q'}_{i,j-1}, |q_i - q'_j| + dtw^{q,q'}_{i-1,j}, 2|q_i - q'_j| + dtw^{q,q'}_{i-1,j-1}) \tag{5.9}$$

The formalization for the general scenario can be found at Buza [52]. Figure 5.3 presents an example of DTW between two sequences: $T_1 =< 1, 3, 2, 4, 3 >$ and $T_2 =< 1, 2, 3, 3, 4 >$. The DTW between the two sequences is 3. Figure 5.3a provides the dtw matrix, which enables the follow-up of computed distance. The solution for the computation of $dtw^{T_1,T_2}_{3,4}$ is expressed in Figure 5.3b, which is equal 2.

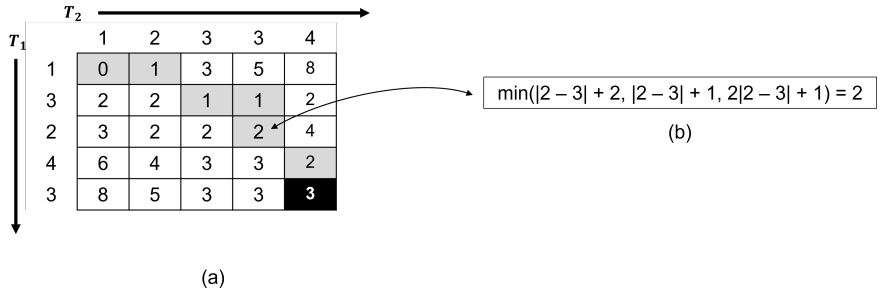


Fig. 5.3 DTW between two sequences: T_1 ($< 1, 3, 2, 4, 3 >$) and T_2 ($< 1, 2, 3, 3, 4 >$) (adapted from Buza [52]).

5.4 Motif discovery methods

There are some main approaches to discovering motifs, such as brute force, indexed-based, random projection, early-abandon, and matrix profile. They are described as follows. Methods that are based on indexing are also called approximate methods. Methods that compute the distance between sequences to discover motifs are called exact methods [192].

5.4.1 Brute force discovery

The brute force approach is the simplest method. It has a high computational cost, especially when used to discover sequences of greater size in large datasets [192]. It is indicated for discovering sequences of smaller size. The coverage and accuracy for this method are complete since it makes all possible comparisons between all subsequences of a time series [264].

Algorithm 1 presents a brute force approach for motif discovery. The first step normalizes the time series X using `Z-Score`. It extracts all pairs (p, p') of sequences of size q in a time series and computes their distance. If their distance is below ϵ, their occurrences are associated with p. When all occurrences of a particular sequence p are greater than σ, p is included as a motif with all associated occurrences. Assuming n as the length of the time series, it has a complexity of $O(n^2)$ times the complexity of the distance function.

```
1:  procedure BruteForce(X, q, σ, ϵ)
2:      X' ← zscore(X)
3:      motifs ← ∅
4:      for i ← 1 to |X'| − q do
5:          p ← subseq(X', i, q)
6:          occurences ← i
7:          for j ← i + 1 to |X'| − q do
8:              p' ← subseq(X', j, q)
9:              if dist(p, p') < ϵ then
10:                 occurences ← occurrences ∪ j
11:         if |occurrences| ≥ σ then
12:             motifs ← motifs ∪ < p, occurrences >
13:     return motifs
```

Algorithm 1: Brute force principle

5.4.2 Indexed based discovery

The indexed-based approach assumes a general tolerance for motif discovery. Instead of computing a distance between subsequences, the time series are indexed using `PAA`, `SAX`, or both. Such indexes incorporate a tolerance for search motifs. The `EMMA` is a representative algorithm for this category [212]. Some improvements introduce hash principles [285] while doing motif discovery.

Algorithm 2 provides an indexing approach for motif discovery. It receives as input a time series X, the alphabet size a for `SAX`, the number of segments for `PAA`, the motif length q, and the minimum support for the number of occurrences σ. Line 2 first normalizes the time series. Lines 3 and 4 compute the `PAA` and `SAX`. Line 5 computes the `SW` for the indexed time series Y'. Line 6 efficiently computes a group

by filtering the ones with the number of occurrences greater or equal to σ. The complexity of this operation is $O(n \cdot log(n))$.

1: **procedure** $IndexBased(X, a, k, q, \sigma)$
2: $X' \leftarrow zscore(X)$
3: $Y \leftarrow PAA(X', k)$
4: $Y' \leftarrow SAX(Y, a)$
5: $W \leftarrow sw(Y', q)$
6: $motifs \leftarrow group_by_having(W, \sigma)$
7: **return** $motifs$

Algorithm 2: Indexed based discovery

5.4.3 Random projection

The random projections approach was proposed to handle large datasets, reduce dimensionality, and take data samples aleatory. It optimizes the execution time and reduces the computational consumption in identifying motifs [51, 69]. In random projection, the search for similar subsequences is speedup using hash-based methods for matching certain random elements of the sequence. It uses `SW` and generally includes indexing (`SAX`). The main principle is to add a certain `HDist` tolerance for indexed time series. This principle was inspired by genetics and is related to DNA chains with a certain degree of mutation allowed [51].

Algorithm 3 presents the principles of random projection. It assumes an indexed time series Y, the length for checking a motif q, assuming a maximum `HDist` of ext, and the minimum support for the number of occurrences σ. Line 2 provides initial time series indexing (such as `Z-Score`, `PAA`, `SAX`). Line 3 creates a `SW` of size $q+ext$. Line 4 randomly projects q columns from W. In this case, motifs can be checked by group W' filtering the observations with the number of occurrences greater or equal to σ.

1: **procedure** $RandomProjection(X, q, ext, \sigma)$
2: $Y \leftarrow index(X)$ ▷ Assuming `Z-Score` and SAX
3: $W \leftarrow sw(Y, q + ext)$
4: $W' \leftarrow project(W, q)$
5: $motifs \leftarrow group_by_having(W', \sigma)$
6: **return** $motifs$

Algorithm 3: Random projection discovery

Applying one interaction is insufficient for large motifs to guarantee that the subsequences mapped are motifs since they cover only a small part of the subsequence. The random projection process is applied more times to address this problem. A

collision matrix is built for the different executions. The motif identification is done by selecting the indexes with the highest values in the collision matrix. Such indexes may represent a motif, but this property is not guaranteed since it is a probabilistic method [69]. Random projection can also be used directly in numeric times series. The goal is to speed up the cases where sequences are not motifs [285].

5.4.4 Early abandon distance

The early abandon distance-based methods modify the structure of the brute force algorithm to early abandon the computation of distances whenever the partial computation of the distance leads to a distance higher than the accepted error ϵ. For example, consider the scenario of computing Euclidean distance (`EDist`). Commonly, the square root is applied when all distance components are computed. However, instead of checking the distance error, it is possible to square the accepted error (ϵ^2) and partially check if the sum of each distance component is higher than ϵ^2 [192].

The approach has two advantages: it speeds up distance checking. Besides, some assumptions of neighbor observations that could lead to trivial motifs can be avoided, which decreases the search space. In the general case, the complexity of the algorithm is the same as brute force. However, in empirical evaluation using real data, much computation is avoided [192].

5.4.5 Matrix profile

The Matrix profile (`MP`) is a distance-based data structure representing the distance of a sequence of size q with the most similar subsequence present in the time series. It can use numeric data directly and uses a matrix for support comparisons of subsequences, which can be used as a pair of similar sequences (motifs) [290]. It is called `MP` since a naive implementation of this technique is to compute all pairwise distances for all sequences present in the time series, storing them in a matrix. It also contains an additional data structure named Matrix Profile Index, which contains the location of the closest matching subsequence for each subsequence [79, 291].

It is worth mentioning that `MP` enables all-pairs-similarity-search on two different time series. In this case, the `MP` builds a vector containing the z-normalized Euclidean distances between each subsequence from the first series and its closest matching subsequence from the second time series. Formally, given two normalized time series of n observations, Y and Y' and a subsequence length q, the `MP` M is a matrix in $\mathbb{R}^{n-q+1}$ and a Matrix Profile Index is a vector I in $\mathbb{R}^{n-q+1}$. I_i contains the index of the start of the subsequence of Y' with length q that best matches $subseq(Y, i, q)$. M_i contains the corresponding distance. In case a *self-join* is performed where $Y = Y'$, an additional constraint is added to prevent *trivial matches*, where subsequences match themselves or nearby subsequences, called *exclusion zone* [290].

A natural product of the MP is its ability to detect discords. Discords are anomalous or unusual subsequences in the time series. By finding the highest values in the MP, it is possible to identify subsequences dissimilar to their nearest neighbors, indicating potential discords [79].

The MP was originally published together with the STAMP [290]. It is an algorithm to calculate the MP over a time series (depicted in Algorithm 4). Internally, Scalable time series anytime matrix profile (STAMP) uses a similarity search algorithm called MASS that, under z-normalized Euclidean, iteratively calculates the distance of each subsequence of a time series X to every subsequence of another time series X'. It uses FFT distance function to enable fast computation [291].

```
1:  procedure STAMP(X, X' = nil, q)
2:      Y' ← Y ← zscore(X)
3:      if X' ≠ nil then
4:          Y' ← zscore(X')
5:      M ← infs, I ← zeros
6:      for i ← 1 to |Y'| − q do
7:          seq ← subseq(Y', i, q)
8:          D_i ← MASS(seq, Y)
9:          M_i, I_i ← eleMin(M, I, D_i, i : (i + q − 1))
10:     return {P, I}
```

Algorithm 4: STAMP Algorithm

The STAMP is outlined in Algorithm 4. In line 5, the matrix profile M and matrix profile index I are initialized. From lines 6 to 9, the distance profiles D are computed using each subsequence seq from Y'. The pairwise minimum for each element in D is performed with the paired element in M (*i.e.*, $\min(D_i, M_i)$ for $i = 1$ to $|Y'|$. Then, as the minimum pair operations are performed, I_i is updated with seq (when $D_i \leq M_i$). Finally, the result M and I are returned in line 10.

When $Y = Y'$, STAMP computes the MP for the general similarity join. In this case, trivial matches in D should be ignored when performing element-wise minimum ($eleMin$) in line 9. The overall complexity of the algorithm is $O(n^2\ logn)$, where n is the length of the time series. Since all subsequences are compared using the MASS algorithm, the $n \log n$ factor comes from the FFT distance subroutine invoked inside $MASS$.

Figure 5.4 presents motif discovery using MP with STAMP for the YGT. It searched for motifs of size six. The algorithm was able to find six pairs of similar motifs. Figure 5.4 presents the most similar motif in this configuration.

Some motif discovery methods are publicly available at Harbinger (see Section 8.3).

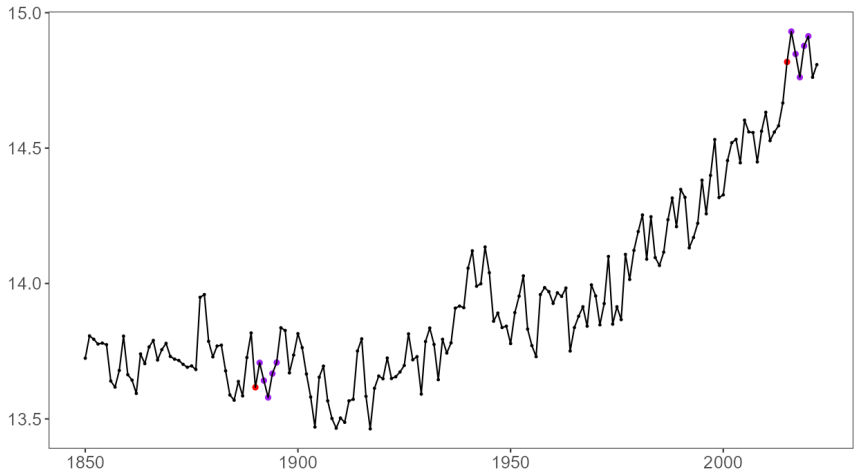


Fig. 5.4 Most similar motif of size six detected using `MP` for `YGT`

5.5 Advanced topics in motif discovery

This section covers some advanced topics in motif discovery. It includes discovering motifs in multidimensional time series, variable motif size (*i.e.*, when the size of the motif is not specified beforehand), ranking of discovered motifs, discovering motifs in Big Data, and novel data structures.

5.5.1 Multidimensional motif discovery

Many variables are collected through time in many real-world scenarios, such as sensor monitoring. It leads to a multidimensional time series. When it comes to multivariate scenarios, there are many initiatives adopted [169, 178, 273, 289]: (i) reduction of dimensionality; (ii) combination of discovery; (iii) indexed-based; (iv) exact match. These initiatives can be used separately or jointly.

Reduction of dimensionality. The first one is straightforward in the sense that dimensional reduction techniques are applied (such as `PCA`) to transform a multidimensional problem into a uni-dimensional one [258]. Sometimes, due to many dimensions, some techniques first provide subspaces. These methods project the time series onto a lower-dimensional subspace and then apply dimensionality reduction techniques for other univariate motif discovery techniques.

Combination of discovery. In combination, univariate motif discovery methods are applied to each dimension of the multivariate time series independently as a separate univariate time series. Subsequently, the identified motifs in each dimension

are combined using logical operations (AND or OR) to form multidimensional motifs [268].

Indexed-based. Indexed-based techniques [247] discretize the data and provide temporal aggregation for time series. It uses `SAX` and `DFT` for discretization and `PAA` for temporal aggregation. Such approaches speed up the motif detection process in multidimensional time series.

Exact match. Exact match techniques are based on distance measures. They capture the dissimilarity between multivariate time series using a particular distance (Euclidean, `DTW`, and correlation). These distance measures are then used within clustering or pattern-matching algorithms to identify motifs [14, 290].

Finally, some of these initiatives can be used together. Tanaka et al. [258] applies Minimum description length (`MDL`) to address multidimensional indexed time series. It combines both indexed-based with dimensional reduction. Further reading can be found at De Paepe et al. [79].

5.5.2 Variable motif size

In all methods presented so far, the input for motif discovery is driven by the length of the desired subsequence. Some initiatives try to address this issue by enabling approaches that can work without constraining the length of the desired sequence or, by at least, establishing a particular range of length to search [183, 200]. Tang and Liao [259] find motifs of size k (fraction of bigger motif). Then, they concatenate discovered motifs to search for more complex ones. Mueen [190] presents an approach to discovering motifs of different lengths. It uses the concept of cover, like generating candidates in sequence mining. Linardi et al. [167] introduce a new framework that provides an exact motif discovery algorithm that finds all motifs and discords in a range of sequence lengths.

5.5.3 Ranking motifs and occurrences

After the motif discovering process, an important task in motif analysis is sorting the motifs according to their relevance [56]. A standard classification method is k-motif, which considers the number of occurrences of the motifs in a time series. Also, motifs can be sorted according to their relevance degree. For instance, some motifs can be similar to a straight line (*i.e.*, constant observations) and may not be relevant depending on the data domain. Such motifs can be low qualified or discarded to avoid distorting the analysis [69].

Some approaches to evaluate the significance and relevance of motifs were proposed in the literature. A statistical approach to assess the relevance of motifs is based on information gain, which measures how expected the motif is to occur [56]. The Log-odds consider the degree of how rare the motif is by comparing the amount

of occurrence with the expected chance of having occurrence based on probabilistic distribution [287]. Castro and Azevedo [56] proposed the estimation of expectation for the frequency of a motif based on Markov Chain models. The value is assessed, comparing actual frequency, and estimated based on hypothesis tests.

Some motif algorithms are naturally driven to rank motifs according to similarities. For example, MP approaches return pairs of motifs ranked by similarities [290]. Other approaches provide ways to rank motifs using other metrics. These initiatives might also be interfered with the type of time series studied. Borges et al. [43] provide an approach that ranks the motifs according to balance among the number of occurrences, the entropy of the motif signal (considering indexed motifs), and the density of their occurrences. The closer the occurrences are, the better ranked they become.

5.5.4 Big Data

When it comes to Big Data, some issues are commonly addressed. The first is related to enabling improvements in distance computation. For example, Rakthanmanon et al. [225] and Alaee et al. [15] improve DTW computation to support motif discovery. Alaee et al. [15] established a novel algorithm called SWAMP, which is based on MP. Besides, Mueen et al. [193] proposed a disk-aware algorithm to discover exact motifs in large time series databases.

Some other works improve the processing of indexed time series. Castro and Azevedo [55] provides iSAX to do motif discovery using PAA and SAX. Other works are also concerned with the speed of processing time series collected in streaming. Fuchs et al. [101] introduce SwiftMotif, which is focused on streaming data. Other approaches established novel methods to improve the management of large time series. Zhu et al. [308] introduce mSTAMP algorithm that is built on top of MP.

5.5.5 Novel data structures

In the context of data structure, there are many flavors of empowering motif discovery using MP, such as context matrix profile, STOMP, and AAMP. Context matrix profile establishes searching constraint for the MP [79]. It enables the discovery of structures that are temporal close to each other.

STOMP is similar to STAMP [143] in that it can be seen as highly optimized nested loop searches, with the repeated calculation of distance profiles as the inner loop. However, while STAMP must evaluate the distance profiles in random order (to allow its anytime behavior), STOMP performs an *ordered* search. By exploiting the locality of these searches, STOMP can reduce the time complexity by a factor of $O(n \log n)$.

AAMP [10] is an efficient algorithm for computing the matrix profile with the pure (non-normalized) Euclidean distance. It avoids the need to do `Z-Score` normalization.

Chapter 6
Evaluation metrics

Considering the broad set of detection methods available, evaluating their performance in verifying if they can detect events correctly and if they miss important events is necessary. Particularly, correctly judging whether detection is made close to the actual events involves introducing time tolerance in event detection evaluation. In this way, Section 6.1 presents the general metrics adopted in event detection. Section 6.2 characterizes the concept of time tolerance. Section 6.3 describes a metric that introduces time tolerance. Finally, Section 6.4 briefly presents other initiatives in event detection metrics.

6.1 Basic metrics

In time series analysis, it is often possible to observe a significant change in behavior at a certain point or time interval. Such behavior change generally characterizes the occurrence of an event [117]. An event can represent a phenomenon with a defined meaning in a domain. Event detection is identifying the occurrence of such events based on data analysis. It is recognized as a basic function in surveillance and monitoring systems. Moreover, it becomes even more relevant for applications based on time series and sensor data analysis [217].

There are many strategies for event detection [18, 77, 265, 302]. Regardless of the adopted detection strategy, an important aspect of any event detection method is how the events are reported. Typically, assuming that events are labeled, standard classification metrics (around since the 1950s), including Accuracy, Recall, Precision, and F1, are usually adopted [129, 159, 260]. Although Accuracy is a specific metric [118], the expression detection accuracy is henceforth used to refer to the ability of a method to detect events correctly. Classification metrics focus mainly on an analysis of detection accuracy. On the other hand, inaccuracy in event detection does not necessarily mean a bad result, especially when detections are sufficiently close to events. This property is particularly true when labels are marked manually, which provides intrinsic imprecision in the labeling process.

The assessment of event detection performance is often important to infer the adequability of a method to a particular application. In this case, detection performance refers to how good a labeling event detection method is at predicting the class label of data instances [233, 260]. The standard classification evaluation metrics are summarized in Table 6.1, including accuracy, recall, specificity, precision, F1, and F_β [118].

Table 6.1 Standard event detection performance metrics [118]

Metric	Formula
accuracy, recognition rate	$\frac{TP+TN}{P+N}$
error rate, misclassification rate	$\frac{FP+FN}{P+N}$
precision	$\frac{TP}{TP+FP}$
recall, sensitivity, true positive rate	$\frac{TP}{P}$
F_1, F-score, harmonic mean of precision and recall	$\frac{2 \times precision \times recall}{precision + recall}$
specificity, true negative rate	$\frac{TN}{N}$
fallout, false positive rate	$\frac{FP}{N}$
F_β, where β is a non-negative real number	$\frac{(1+\beta^2) \times precision \times recall}{beta^2 \times precision + recall}$

The metric formulas in Table 6.1 depend on the measures P and N. Respectively, they refer to the number of positive instances (data instances of the main class of interest) and negative instances (all others). In event detection, the positive instances indicate the occurrence of an event ($event = yes$), while the negative instances indicate typical observations ($event = no$). Moreover, Table 6.1 presents four other important measures to compare the class labels yielded by the detection method with the known class labels of the data instances. They are TP, TN, FP, and FN [118].

The TP measure is the number of true positives, which refer to the positive instances that were correctly labeled by the method. Analogously, TN is the number of true negatives, *i.e.*, negative instances correctly labeled by the method. On the other hand, the measure FP is the number of false positives, that is, the negative instances that were incorrectly labeled as positive. Finally, FN is the number of false negatives, referring to the positive instances mislabeled as negative [118, 233]. The four measures are summarized in a confusion matrix as presented in Table 6.2,

which is a useful tool for analyzing how well a detection method can recognize the positive (event) class. TP and TN refer to correct labeling, while FP and FN refer to mislabels. Totals are also provided. Besides P and N, P' is the number of data instances labeled as positive, and N' is the number of instances labeled as negative. Given a time series X, the total number of instances is $TP+TN+FP+TN = P+N = P' + N' = |X|$.

Table 6.2 Standard confusion matrix for event detection

		Detected events: yes	Detected events: no	Total
Known events	yes	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P+N$

The standard (classification-based) event detection performance metrics listed in Table 6.1 enable specific analysis regarding the performance of the detection methods. The accuracy metric reflects how well the method recognizes data instances of both classes (positive and negative). The error rate is the complement of the accuracy. The sensitivity and specificity metrics reflect how well the method can recognize the positive instances ($event = yes$) and the negative instances ($event = no$), respectively. The fallout is the complement of the specificity. The precision and recall metrics are widely adopted [260]. Precision reflects exactness (the percentage of instances labeled as positive that are so). In contrast, recall reflects completeness (the percentage of positive instances labeled as such). A perfect precision score (1.0) means that every instance of the method labeled as an event is a known event. A perfect recall score (1.0) means that every known event was labeled as such. Precision and recall are combined in the F_1 and the F_β metrics. The F_1 metric is the harmonic mean of precision and recall, giving them equal weight. The F_β metric, on the other hand, assigns β times as much weight to recall as to precision [118]. Both F_1 and F_β metrics help gauge the quality of event detection [260].

Consider an example of event detection depicted in Table 6.3. The example has ten observations, of which nine events were detected. Only between time 2 to 6 the observations were labeled as events. In this way, true positives (TP), true negatives (TN), and false positives (FP) were respectively 5, 1, 4. There were no false negatives ($FN = 0$). This way, accuracy, precision, recall, and F_1 were, respectively, 0.6, 0.55, 1, and 0.71.

Event detection methods frequently use the Operating characteristic curve (ROC) to visualize the ability of a detection method to discriminate between positive instances (events) and negative instances (nonevents) based on the true positive rate (tpr) versus false positive rate (fpr). Detection methods whose curves come closer to the upper left corner of the ROC space are the most accurate [118]. Likewise, the accuracy of the methods can be compared by the Area under the curve (AUC) defined in Equation 6.1, which summarizes the ROC curve into a metric [233]. A perfect method can generate an AUC of 1. Conversely, a random method has a performance

Table 6.3 Example of detections in a time series with ten observations

Time	Detector	Actual
1	*T*	*F*
2	*T*	*T*
3	*T*	*T*
4	*T*	*T*
5	*T*	*T*
6	*T*	*T*
7	*T*	*F*
8	*T*	*F*
9	*T*	*F*
10	*F*	*F*

equal to 0.5. The ROC curve for the example of Table 6.3 is depicted in blue in Figure 6.1. The AUC curve equals 0.6, which is slightly better than a random method (in red). A perfect detector is plotted as green.

$$AUC = \int_0^1 tpr(fpr)\,\mathrm{d}fpr \in [0,1], \tag{6.1}$$



Fig. 6.1 ROC curves for Example of Table 6.3.

The problem of benchmarking event detection performance is challenging [57, 59, 159, 207]. Aminikhanghahi and Cook [21] review traditional metrics for the evaluation of detection performance of change point methods. Aminikhanghahi and Cook [21] observed the usage of Accuracy, Sensitivity (recall), G-mean, F-measure, ROC, PR-Curve, and MSE. Detection evaluation measures have also been investigated in the areas of sequence data anomaly detection [61], time series mining and representation [83], sensor-based human activity learning [277]. Metrics found in the literature are mainly designed to evaluate the detection of punctual anomalies.

However, many real-world event occurrences extend over a period (interval-based). Motivated by this, Tatbul et al. [260] expand the well-known Precision and Recall metrics to measure the accuracy of detection algorithms over event ranges.

6.2 Time tolerance

This section gives an example of the problem of evaluating inaccurate event detections. It characterizes the need to introduce time tolerance while evaluating detection methods. Consider, for example, a time series X containing an event at time t, represented in Figure 6.2. Given detection methods A and B applied to X, a user must select one as the most adequate for the underlying application. Both of them made detecting close to E_1. Method A detects an event at time 12, while Method B detects an event at time 10. None of the methods could correctly detect the event at time 11. Besides, only method B made a detection close to the event E_2 (at time 21). Based on the basic metrics, the user would deem both inaccurate and disposable.

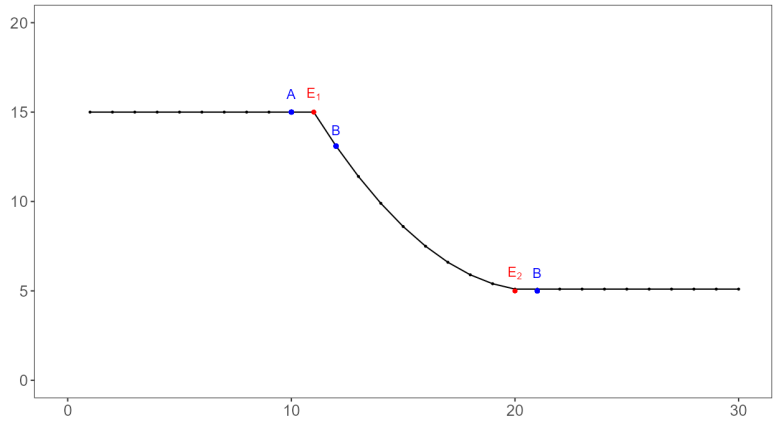


Fig. 6.2 Example regarding the problem of evaluating the detection of events at time 11 (E_1) and 20 (E_2). Method A and B detect an event at times 10 and 12 concerning E_1. Method B was the only method that detected an event close to E_2.

However, inaccuracy in detecting an event can often result from its preceding or lingering effects. Take the adoption of a new policy in a business. While a domain specialist may consider the moment of policy enforcement as a company event, its effects on profit may only be detectable a few months later. On the other hand, preparations for policy adoption may be detectable in the antecedent months. Moreover, when accurate detections are not achievable, which is common, detection applications demand events to be identified as soon as possible [159], or early enough to allow necessary actions to be taken, mitigating possible critical system failures or

help mitigate urban problems resulting from extreme weather events, for example. In this context, the results of Methods *A* and *B* would be valuable to the user. Note that while the method *B* seems to anticipate the event, its detection is made after the occurrence of the event. On the other hand, the detection of method *A* came temporally closer to the event, possibly more representative of its effects. Since this analysis uses the entire time series, it is impossible to guarantee such property. From Figure 6.2, it is only possible to infer that both results from method *A* and *B* were detected at time 30 (since the last observation seen so far is displayed at time 30), which for an online event detection could be too late (see Section 7.4 for further discussion).

Evaluating event detection is particularly challenging. Some issues regarding the quality of available benchmark datasets provide a false perspective of progress in the area [282]. Besides, the detection accuracy metrics usually adopted must be revised for the task [246]. Standard classification metrics do not consider the concept of time, which is fundamental in the context of time series analysis, and do not reward early detection [8], for example, or any relevant neighboring detections. For the remainder of this paper, neighboring or close detections refer to detections whose temporal distance to events is within a desired threshold. Current metrics only reward true positives (exact matches in event detection). All other results are "harshly" and equally discredited.

In this case, there is a demand to introduce time tolerance in the usual concept of detection accuracy and evaluate the methods while considering neighboring detections. However, state-of-the-art metrics designed for scoring anomaly detection [159] are still limited [246], also being biased towards results preceding events, such as the ones produced by the method *B*. A step forward in addressing some of these issues is provided by Salles et al. [237], which introduce time tolerance in event detection while preserving compatibility with traditional classification metrics.

6.3 SoftED

This section provides an overview of Soft evaluation event detection (`SoftED`) as defined in Salles et al. [237]. `SoftED` is a metric designed to assess the performance of methods detecting events in time series with time tolerance. The fundamental idea behind `SoftED` is introducing flexibility into traditional classification metrics (hard metrics).

Figure 6.3 illustrates the concept of `SoftED`, highlighting the key distinction between standard hard evaluation and the proposed soft evaluation. The standard hard evaluation involves binary outcomes, indicating whether a detection precisely matches an actual event. In contrast, the soft evaluation measures the degree to which a detection is associated with a specific event.

`SoftED` introduces a time tolerance for events through a distance-based approach. The relevance of detection to a specific event is determined by an event membership

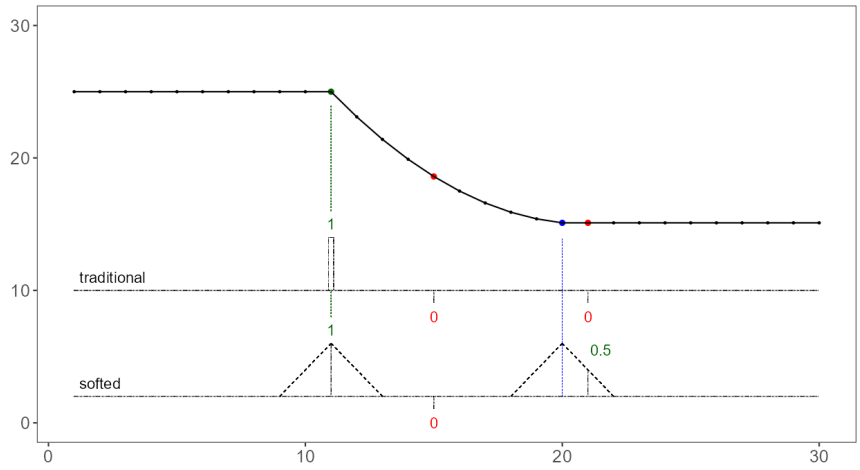


Fig. 6.3 Comparison between the standard "hard" evaluation and the "soft" evaluation of event detection.

function $\mu_{e_j}(t)$, as defined in Equation 6.2. The function considers an acceptable time range represented by the constant k.

$$\mu_{e_j}(t_{d_i}) = \max\left(\min\left(\frac{t_{d_i} - (t_{e_j} - k)}{k}, \frac{(t_{e_j} + k) - t_{d_i}}{k}\right), 0\right) \tag{6.2}$$

Figure 6.4a depicts the event membership function $\mu_{e_j}(t)$. For each detection d_i, the function evaluates the temporal closeness to the corresponding event e_j.

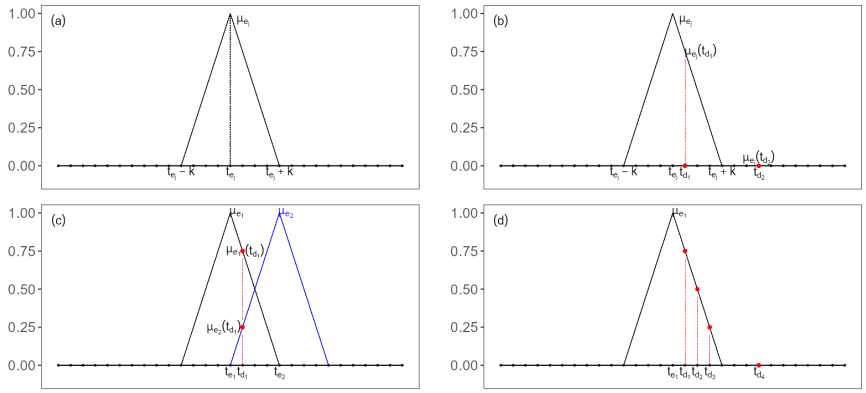


Fig. 6.4 `SoftED` evaluation mapping. (a) represents an event membership function $\mu_{e_j}(t)$. (b) represents $\mu_{e_j}(t)$ for detections d_1 and d_2. (c) illustrates one detection to many events, motivating the first constraint of `SoftED`. (d) shows many detections to a single event, motivating the second constraint of `SoftED`.

The degree to which a detection d_i is relevant to an event e_j is determined by $\mu_{e_j}(t_{d_i})$, with a higher value indicating temporal closeness to a true positive (TP) regarding e_j. This concept is illustrated in Figure 6.4b, where $\mu_{e_j}(t)$ is evaluated for two detections, d_1 and d_2, produced by a specific detection method. In this context, `SoftED` evaluates the extent to which a detection represents an event or, in other words, its temporal closeness to a hard TP. For instance, detection d_1 is closer to a TP. At the same time, d_2 falls outside the tolerance range and might be considered a false positive.

`SoftED` introduces two constraints to maintain consistency with standard hard metrics. The first constraint stipulates that a given detection d_i should have only one associated score. The second constraint ensures that the total score associated with a specific event e_j must not exceed 1.

Consider the scenario in Figure 6.4c, where one detection is evaluated for multiple close events. The detection d_1 is assessed for events e_1, e_2, and e_3, resulting in three distinct membership evaluations ($\mu_{e_1}(t_{d_1})$, $\mu_{e_2}(t_{d_1})$, and $\mu_{e_3}(t_{d_1})$). However, to maintain consistency with hard metrics, a given detection d_1 should not have more than one score. Otherwise, d_1 would be rewarded multiple times, potentially surpassing the score of a perfect match, which is 1.

`SoftED` attributes each detection d_i to the event e_j that maximizes the membership evaluation $\mu_{e_j}(t_{d_i})$. This attribution is denoted by E_{d_i} (Equation 6.3). In Figure 6.4c, d_1 is attributed to event e_1 due to the maximum membership evaluation of $\mu_{e_1}(t_{d_1})$. The set of detections attributed to each event e_j, D_{e_j}, is defined by Equation 6.4. The inclusion of a detection d_i in the set D_{e_j} is conditioned on the membership evaluation being greater than 0 ($\mu_{e_j}(t_{d_i}) > 0$).

$$E_{d_i} = \arg\max_{e_j} \left(\mu_{e_j}(t_{d_i}) \right) \tag{6.3}$$

$$D_{e_j} = \{ d_i \mid E_{d_i} \supset e_j \wedge \mu_{e_j}(t_{d_i}) > 0 \} \tag{6.4}$$

Additionally, a specific detection method should not be rewarded more than once for detecting the same event e_j to ensure that the total score for event detections does not surpass the perfect reference score of 1. Consider the scenario in Figure 6.4d, where multiple detections are attributed to the same event. Event e_1 is present in the sets E_{d_1}, E_{d_2}, E_{d_3}, and E_{d_4}. Moreover, D_{e_1} contains d_1, d_2, and d_3. However, to maintain consistency with hard metrics, the total score for D_{e_1} ($\mu_{e_1}(t_{d_1}) + \mu_{e_1}(t_{d_2}) + \mu_{e_1}(t_{d_3})$) should not surpass the score of a perfect match. `SoftED` attributes to event e_j the detection d_i (contained in D_{e_j}) that maximizes the membership evaluation $\mu_{e_j}(t_{d_i})$. This attribution is denoted by $\hat{d}_{e_j}$ (Equation 6.5). According to Figure 6.4d, e_1 is best represented by detection d_1, given the maximum membership evaluation of $\mu_{e_1}(t_{d_1})$. Consequently, `SoftED` computes the associated score for each event e_j as $es(e_j)$ (Equation 6.6).

$$\hat{d}_{e_j} = \arg\max_{d_i} \left(\{ \mu_{e_j}(t_{d_i}) \mid d_i \in D_{e_j} \} \right) \tag{6.5}$$

$$es(e_j) = \mu_{e_j}(t_{\hat{d}_{e_j}}) \tag{6.6}$$

Finally, each detection d_i produced by a specific detection method is scored by $ds(d_i)$ (Equation 6.7). Representative detections ($d_i = \hat{d}_{e_j}$) are scored based on $es(e_j)$, while all other detections receive a score of 0. This definition ensures that the total score for detections of a specific method does not surpass the number of real events m in the time series X. Equation 6.8 maintains the reference to a perfect detection Recall score according to traditional hard metrics. Additionally, it penalizes false positives and multiple detections for the same event e_j.

$$ds(d_i) = \begin{cases} es(e_j), & \text{if } \exists e_j \in E \mid d_i = \hat{d}_{e_j} \\ 0, & \text{otherwise} \end{cases} \tag{6.7}$$

$$\sum_{i=1}^{n} ds(d_i) \leq m \tag{6.8}$$

The scores computed for each detection d_i, $ds(d_i)$, derive soft versions of the hard metrics TP, FP, TN, and FN, as outlined in Table 6.4. The soft metrics, including TP_s, FP_s, TN_s, and FN_s, maintain the same properties and scale as traditional hard metrics. They enable the derivation of soft versions of common scoring methods such as Sensitivity, Specificity, Precision, Recall, and F1. Importantly, `SoftED` metrics retain the same interpretability while accounting for time tolerance, which is crucial in event detection applications.

Table 6.4 `SoftED` Metrics

$\text{TP}_s =$	$\sum_{i=1}^{n} ds(d_i)$	$\text{FN}_s =$	$m - \text{TP}_s$
$\text{FP}_s =$	$\sum_{i=1}^{n} (1 - ds(d_i))$	$\text{TN}_s =$	$(\lvert t \rvert - m) - \text{FP}_s$

An implementation of `SoftED` metrics in R is publicly available in Harbinger (see Section 8.3), allowing the computation of both hard and soft metrics.

6.4 Other initiatives for event detection metrics

The Numenta anomaly benchmark (`NAB`) provides a common scoring algorithm for evaluating and comparing the efficacy of anomaly detection methods [159]. The `NAB` score metric is computed based on anomaly windows of observations centered around each event in a time series. Given an anomaly window, `NAB` uses the sigmoid scoring function to compute the weights of each anomaly detection. It rewards earlier detections within a given window and penalizes FP. Also, `NAB` allows the definition

of application profiles: standard, reward low FP, and reward low FN. The standard profile gives relative weights to TP, FP, and FN based on the window size.

Nonetheless, the `NAB` scoring system presents challenges for its usage in real-world applications. For example, the anomaly window size is automatically defined as 10% of the time series size, divided by the number of events it contains, and values generally not known in advance, especially in streaming environments. Furthermore, Singh and Olinsky [246] pointed out some issues with definitions and arbitrary constants in the scoring equations. Finally, score values increase with the number of events and detections. Every user can tweak the weights in application profiles, making it difficult to interpret and benchmark results from other users or setups.

In addition to `NAB` [159], this section presents other works related to the problem of analyzing and comparing event detection performance [59]. Recent works focus on developing benchmarks to evaluate univariate anomaly detection methods [40, 133, 280]. Jacob et al. [133] provide a comprehensive benchmark for explainable anomaly detection over high-dimensional time series. In contrast, the benchmark developed by Boniol et al. [40] allows the user to assess the advantages and limitations of both anomaly detection methods and detection accuracy metrics.

Standard classification metrics are generally used for evaluating the ability of an algorithm to distinguish normal from abnormal data samples [59, 260]. Aminikhanghahi and Cook [21] review traditional metrics for change point detection, such as Sensitivity, G-mean, F-Measure, ROC, PR-Curve, and MSE. Detection evaluation measures have also been investigated in the areas of sequence data anomaly detection [61], time series mining and representation [83], and sensor-based human activity learning [277].

Metrics found in the literature are mainly designed to evaluate the detection of punctual anomalies. However, many real-world event occurrences extend over an interval (range-based). Motivated by this, Tatbul et al. [260] and Paparrizos et al. [206] extend the well-known Precision and Recall metrics and the AUC-based metrics, respectively, to measure the accuracy of detection algorithms over range-based anomalies. Other recent metrics developed for detecting range-based time series anomalies are also included in the benchmark of Boniol et al. [40]. In addition, Wenig et al. [280] published a benchmarking toolkit for algorithms designed for detecting anomalous subsequences in time series [39, 42].

Chapter 7
Online event detection

In monitoring systems, online event detection is a mandatory subject. Characterizing online and offline event detection is important to comprehend the area properly. Section 7.1 clarifies the difference between online and offline event detection. In online detection, there is additional complexity in memory management, concept drift detection, evaluation of the need for retraining models, and inferring which detection method is better suited for this problem (Section 7.2). Section 7.3 clarifies online prediction, a specialized case of online event detection. Such a challenging context of online event detection requires revisiting event detection metrics (Section 7.4). Finally, Section 7.5 presents the challenges regarding online event detection.

7.1 Characterizing the online versus offline scenario

Event detection is the process of discovering events in a time series. With this process, we may be interested in identifying past events (offline detection) [205, 240, 306], detecting events in real-time (online detection) [8, 29, 194], or even predicting future events before they happen (online prediction) [188, 233, 302]. Event detection is recognized as a basic function in surveillance and monitoring systems. It has gained much attention in research for application domains involving large datasets from critical systems [131, 217, 270].

Several methods have been developed and are surveyed in the literature to address the task of time series event detection [77, 264, 265]. Each detection method presents different characteristics or makes assumptions about the time series distribution. For example, in the scenario of event detection using classification methods, a dataset is separated into training and test sets. In the case of time series, the test set contains recent data [130]. The model is built using the training set. It is common to partition the training data set using a cross-validation strategy to optimize hyperparameters. Once the model is built, it is evaluated using a test set [118]. This traditional approach is depicted in Figure 7.1a.

However, observations occur continuously as a streaming data source for event detection. Even when the entire streaming is stored in a single dataset organized into batches, the time stamp of the data needs to be considered. Training should occur using past data to predict more recent data, mimicking data streaming scenarios. It is depicted in Figure 7.1b, where the i-th batch (training data) is used to build a model for further evaluation with more recent data at the next batch $(i + 1)$ (test data). It is worth mentioning that the i-th batch corresponds to a dataset sample in the time interval associated with i. While studying concept drift, such methodology is mandatory [107, 132].

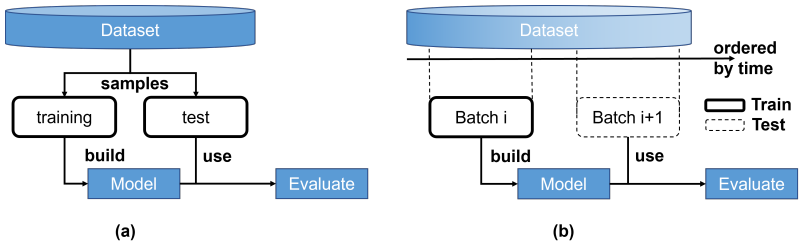


Fig. 7.1 Characterization of training and testing during classification: (a) traditional scenario; (b): streaming scenario (adopted from Giusti et al. [107])

7.2 Online detection

Online detection is identifying the occurrence of events as observations come, such as in a streaming scenario. Online applications impose particular challenges for time series modeling. In contrast to offline event detection, the entire time series is unavailable (or at least the notion of the entire time series varies according to time). The system must analyze each observation in sequential order as they arrive, and any processing or learning must be done online.

Let X be an online or streaming time series composed of a continuous input: $< \ldots, x_{t-2}, x_{t-1}, x_t, x_{t+1}, x_{t+2}, \cdots >$. At each point in time t, a model trained on previous observations $< \ldots, x_{t-2}, x_{t-1} >$ is used to determine whether the current behavior of the system is unusual. This determination should be made by the arrival of the next input x_{t+1}. For this, the time series model might be continuously updated [8].

Unlike offline event detection, data is not split into static train/test sets, and algorithms cannot look ahead. Practical applications impose additional challenges. Typically, the sensor streams are large and at high velocity, leaving little opportunity for expert intervention; manual parameter tweaking and data labeling are usually not viable. Thus, operating in an unsupervised (or semi-supervised) way is needed [8]. Furthermore, in streaming applications, early detection of events is often impor-

tant, where events are identified as soon as possible. Detection of events can give information that is crucial to decision-making. This information must be given early enough to be actionable, preventing possible critical system failures. However, there is still a tradeoff between fast detections and false positives. An algorithm that often makes inaccurate detections is likely to be ignored [217].

Applications aiming at the uninterrupted monitoring of events in a network infrastructure usually generate fast data [184]. In fast data, information undergoes dynamic changes within specific time intervals, ranging from seconds to milliseconds. A substantial volume of data arrives continuously in the pipeline. Streamed data analytics is critical for event analysis and decision-making [29].

Despite detecting events in real-time data having practical and substantial applications across several industries, providing resilient solutions is a challenge [29]. Current approaches for online detection can be categorized into two major categories regarding their modeling approaches. The first category encompasses static models trained on large data samples and deployed on data streams. It remains the same during the entire execution. In the second category, the event detection model is initialized on a data sample and then learned incrementally as new data arrives, adopting a dynamic strategy.

Besides, accommodating large volumes of streaming data in the main memory of a machine is often infeasible. In that case, full-memory algorithm approaches, depicted in Figure 7.2a, that consider all observations (batches) seen so far, are impractical [132]. Hence, solutions that maintain only a fraction of the data are required.

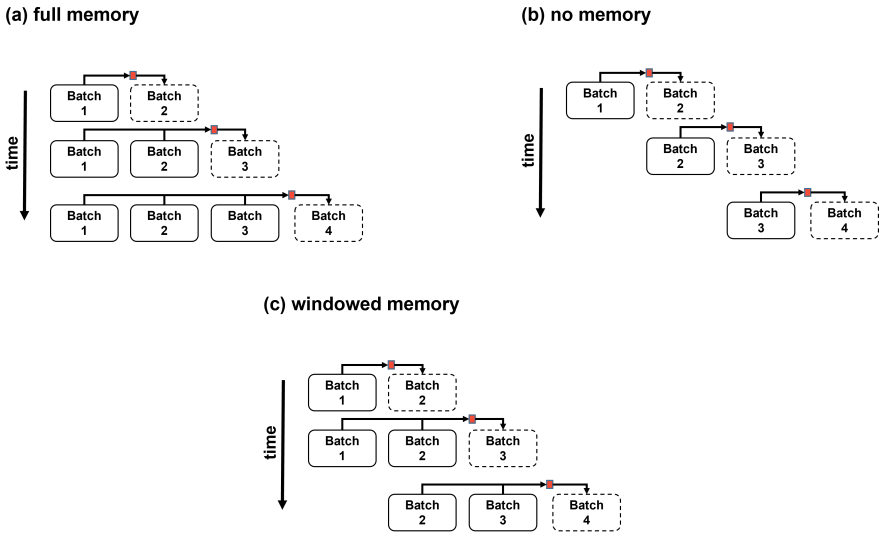


Fig. 7.2 Memory management for online event detection: full-memory, no-memory, or windowed memory. Solid batches are used for training. Dashed batches are being tested. Adapted from Iwashita and Papa [132].

In this case, data models can be retrained using recent batches of data, or they can be trained incrementally by continuous updates. Incremental algorithms process input examples individually (or batch by batch) and update the data model after receiving each example. Typically, for any new batch of data, the model update is based on the previous one (Figure 7.2b). However, incremental algorithms may also access previous or most representative samples. In this case, these algorithms are said to have partial (windowed) memory, as depicted in Figure 7.2c [104].

Learning algorithms often need to operate in dynamic environments that can change unexpectedly. A desirable property of these algorithms is their ability to incorporate new data. This property, in turn, brings us to the stability-plasticity dilemma [87]. How can a learning system be designed to remain adaptive in response to significant changes yet remain stable in response to irrelevant changes?

If the data-generating process is not stationary, the underlying concept, which is being predicted, may change over time. Learning in nonstationary environments requires adaptive or evolving approaches that can monitor the underlying changes and adapt a model to accommodate these changes. Considering that this holds for most practical applications, there is a significant need for algorithms that are both effective and efficient in learning from and adapting to environments that undergo evolution or drift [87]. The ability to adapt to such changes is a natural extension of the incremental learning systems that learn predictive models, example by example. Adaptive learning algorithms can be seen as the next step to incremental learning ones that can adapt to the evolution of the data-generating process over time [238].

There is an intrinsic relationship between nonstationarity and concept drift [104]. Whenever there is a concept drift, it is possible to characterize a time t in which the concept of sequences before t differs from the concept of sequences after t. Concept drifts happen unexpectedly, but changes may take different forms. It can happen abruptly, incrementally, gradually, or even previously seen concepts may reoccur after some time [132]. In such cases, models must adapt to a new definition of *typical* data in an unsupervised process [8]. In that case, adaptive learning involves the real-time updating of predictive models while operating to respond to concept drifts [104].

Adaptive learning algorithms are primarily based on either an active or passive approach. Algorithms following the active approach aim at detecting concept drift. It is related to the detection of change point events, which may be described as the points in which we observe a drift in concept. Algorithms following the passive approach update the model every time new data is given as input, regardless of whether there is drift. They might find it difficult to detect change points [87].

Overall, the requirements described suggest that event detection for online applications is a fundamentally different problem than offline event detection. Furthermore, most existing event detection algorithms do not directly apply to streaming applications [8]. Thus, there is currently a demand for developing reliable methods for online detection.

Over the past decade, there has been a growing focus on online detection applications, driven by the rising need for analyzing IoT sensor data and automated system monitoring [29]. In addition to event detection on streaming data, some problems

demand fast and real-time reactions. It is known as fast event detection. In most cases, this algorithm presents a complexity of $O(N \log N)$ for detecting the most significant event, while previous algorithms present at least $O(N^2)$.

A few survey papers review online detection methods. It includes Zhang et al. [301] that studied anomaly detection over wireless sensor networks data. Ariyaluran Habeeb et al. [29] survey real-time big data processing methods for anomaly detection. Munir et al. [194] also provide a comparative analysis of traditional and deep learning-based anomaly detection methods for streaming data.

7.3 Online prediction

Event detection and online detection are applied to identify historical or ongoing events. Online prediction, on the other hand, corresponds to the problem of predicting events in the future [302]. The anticipation of events can help mitigate potential damages and allow informed decision-making. Online prediction is similar to the problem of time series prediction. However, instead of predicting the next n observations from analyzing past time series data, it focuses on correctly predicting whether a target event will occur within a future window of time [22, 141].

Online prediction methods take as input time series data containing historical events, build a predictive model, and output online predictions. To build a predictive model on the input data, generally, such methods apply techniques from machine learning, data mining, pattern recognition, or statistics [302]. The most common online prediction techniques typically rely on decision trees, support vector machines, and artificial neural networks [188]. There has been considerable research on new online prediction methods in recent years. Nevertheless, most of them are designed for specific application domains, although their approaches can usually be generalized. Overall, online prediction methods are still in their infancy [302].

Generally, online prediction methods for time series adopt one of two main strategies: (i) time series classification or (ii) time series prediction (regression).

7.3.1 Online prediction using time series classification

Online prediction using time series classification is commonly adopted [286]. In this case, sequences and features extracted from preceding observations are analyzed to deduce temporal patterns that can be used for event anticipation. Then, the prediction process involves applying the learned model in new time series observations to predict event labels.

Once prohibitively challenging, work on time series online prediction is growing rapidly in the big data era [302]. However, methods that are adherent to this first strategy are still particularly affected by several common issues that demand special attention [188]:

- Data complexity: time series data commonly present complex behavior, demanding advanced analysis techniques [229].
- Noise: omnipresent in real-world time series, noise tends to decrease online prediction accuracy. It demands noise reduction techniques to be applied before data analysis.
- Unlabeled or unbalanced data: given many unlabeled instances or an unbalanced dataset, supervised learning methods demand time and cost to gather labels (computationally or by experts) and instances of the rare anomalous class.
- Parameterization: some methods need to set initial values and thresholds that usually demand trialing and optimization.
- Detection time: methods must predict in time to allow sufficient anticipation of events.
- Computational feasibility: the demand for memory and CPU also drives the possible methods to be adopted [257].
- Evaluation: a comprehensive evaluation and comparison of time series mining methods for significant online prediction are challenging.

The issues and challenging data properties are frequently observed, especially in online time series applications.

7.3.2 Online prediction using time series prediction

Among the proposed approaches for addressing events is combining two analysis problems: event detection and time series prediction. In this case, events are detected from predicted time series observations. Event detection from time series prediction has been mainly studied in the very recent and prominent area of online prediction [302]. An approach based on event detection from time series prediction can be exploited for the problem of online detection. In that case, streaming observations are identified as anomalies based on what the time series model expects. On the other hand, Seth and Chaudhary [242] provide frameworks for predictive analysis of sensor data streams focusing on predicting insider trading events.

In this case, the historical time series $X = < x_1, \ldots, x_t >$ is used to predict the future time series sequences $\hat{X} = < x_{t+1}, \ldots, x_{t+h} >$. The time series prediction usually requires data preprocessing, as described in Chapter 2. When the model is trained, prediction occurs. The second step involves identifying events in the predicted time series $\hat{X}$ using unsupervised or supervised event detection methods [302]. In this context, online prediction based on time series prediction differs from time series prediction alone. The latter outputs future values of the time series, while the former outputs a predicted event, *i.e.*, the future time of occurrence of an event.

Survey papers presented analytical reviews of research and methods for event monitoring and anticipation in time series [180, 188]. Particularly, some works focus on methods for streaming time series. Salfner et al. [233] present a survey of prediction methods for online failure. In contrast, Zhao [302] presents a systematic survey of methods for time series events in the Big Data era. As methods for event

detection from time series prediction are typically motivated by specific application domains, Zhao [302] lists some other surveys in domains such as flood events, social unrest, wind power ramp forecasting, tornado events, temporal events without location, online failures, and business failures. Both Salfner et al. [233] and Zhao [302] provide a taxonomy for event detection from time series prediction.

7.4 Online detection evaluation

This section evolves from the concepts presented in Chapter 6. Consider detection methods A and B applied to an online time series X. This example examines detection methods for time series X at each new observation. Figure 7.3 clarifies the difference between offline and online evaluations. At time 11, characterized in Figure 7.3a, the event E_1 appears in the time series. However, neither method A nor method B detected it.

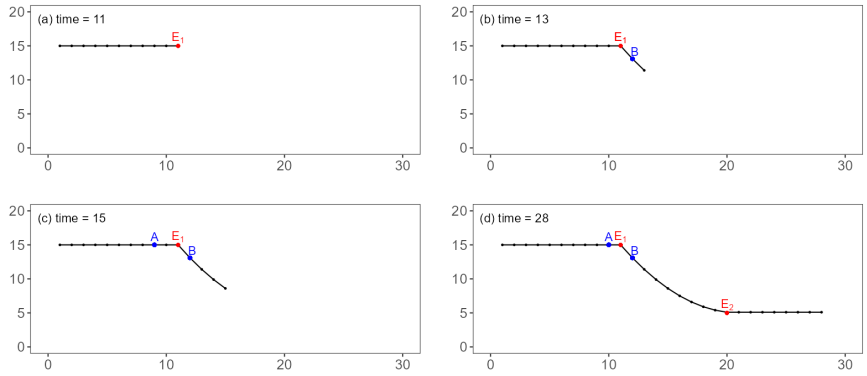


Fig. 7.3 Evaluation of two event detection methods in an online time series: time 11 (a), time 13 (b), time 15 (c), time 28 (d)

In Figure 7.3b, at time 13, the method B makes a detection close to the event E_1 (difference of one time unit). Besides, it took two time units to detect it. At this moment, the method A did not detect it.

At time 15, depicted in Figure 7.3c, the method A finally makes a detection close to the event E_1. The difference between detection and the event is two time units. It is not as close as method B. Besides, the method A took four time units to detect it. So, detecting an event before an event does not necessarily mean anticipating the detection. It is only related to the imprecision of the detection. Anticipating is related to when the detection occurred. The detection should be done before the occurrence of an event to characterize anticipation. It would be related to event prediction.

Finally, at time 28, both methods did not detect the event E_2. At this time, both methods made detection close to event E_1 (one time unit different). Method A presents inconsistent detections. At time 15, it detected time nine related to event E_1. However, at time 28, it detected time 10 as related to the event E_1. During online event detection, the actual detections might change. How to address such inconsistency is still an open problem.

7.5 Open challenges in online event detection

Generally, the solutions to online event detection using classification demand a long time to gather sufficient data to identify patterns. The gathered datasets are usually unbalanced, making learning rare anomalous patterns difficult. Also, in nonstationarity, the learned time series patterns can change gradually or even abruptly.

In this context, an approach based on event detection using time series regression is advantageous since its application only demands the availability of a time series long enough to train a regression model with the required accuracy. Such a model is then used to guide the identification of anomalous observations.

Currently, most event detection methods can only discover events with detection lags between the previously analyzed data points and the newly arrived ones. They must inspect the new observation before determining if an event occurred [21]. Moreover, although the offline and online problems are fundamentally different, online solutions may adapt and benefit from established methods and techniques of offline event detection, especially in the context of big data [229].

It is not uncommon that the same time series may contain different events, such as anomalies, change points, and motifs. Nonetheless, most event detection methods specialize in identifying only one, disregarding other types. Most event detection works in literature are designed to identify anomalies in time series. Consequently, the isolated application of highly specialized methods may compartmentalize information on the events of a time series in the sense that each adopted detection method only highlights events of a specific nature. For example, most anomaly detection methods can disregard change points in a time series. In that case, combining different specialized detection methods is a good idea to understand the events in a time series better.

There is a demand for a consensual definition and formalization of different types of events. Nonetheless, it is possible to observe that while identifying some events can be based solely on previous time series observations, the full detection and distinction between event types can only be confirmed based on an analysis of afterward time series behavior. It poses a new challenge for online applications, where type distinction can only be performed with a specific lag of k observations.

Finally, assessing the detection performance of event detection methods is often important to infer their feasibility to a particular time series and application. They are particularly considering many methods [53]. Besides, it is also important to reduce false positive and false negative situations [204]. Although most papers base their

detection quality evaluation on standard classification metrics, they are not suitable for a qualitative analysis of the temporal bias of detection algorithms, that is, their tendency to anticipate or delay event detections. Moreover, classification metrics are not applicable for online time series event detection, where events must be detected immediately. Hence, there is still a demand for metrics for scoring the performance of online event detection methods that incorporate the time dimension, which is essential to time series.

Chapter 8
Epilogue

Event detection encompasses anomaly detection, change point detection, and motif discovery. Each of these research areas has opportunities for studying. Besides, providing an interplay among them is also important. There are numerous research initiatives in event detection going on. Due to that, it is not possible to provide a "conclusion" on the subject. Rather, this chapter provides closure to the book by summarizing how the research areas are being studied (Section 8.1) and pinpointing the main challenges in the subject (Section 8.2). Finally, Section 8.3 briefly describes Harbinger, a framework for event detection that enables an integrated view of the subject.

8.1 Summary of research area

Time series event detection is a broad and active area. It encompasses anomaly and change point detection and motif discovery. There are many research papers on the broad area of event detection. However, there are even more papers in each one of the individual areas.

A systematic mapping study in the area was conducted to enable a better comprehension of the scientific production in event detection. Four major queries were executed at Scopus. The analysis adopted only articles (journal papers), conference papers, reviews (journal papers of literature review or surveys), and books. The query was evaluated on the title, abstract, and keywords of documents indexed in Scopus. These queries were executed on November 15th, 2023. It is worth mentioning that event detection, anomalies, concept drift, and motifs are broad concepts. Since the scope of the book is driven to time series, all queries included the "time series" constraint to target these concepts whenever they tackled time series context.

The first query focused on the general concept of event detection (including event detection and event prediction), *i.e.*, ("event detection" or "event prediction") and "time series". The query returned 693 documents. Table 8.1 summarizes publications according to the areas.

Table 8.1 Publications according to event detection areas

Type	Event Detection	Anomaly	Change Point	Motif
Article	364	6679	1719	388
Conference paper	324	2825	555	296
Review	5	110	29	9
Book	-	21	7	-

The second query focused on the anomalies, *i.e.*, ("anomaly") and "time series". It is by far the most studied area. The query returned 9635. This area has 6679 articles, 2825 conference papers, 110 reviews, and 21 books. The third query focused on change points in time series. The query included change point and concept drift, *i.e.*, ("change point" or "concept drift") and "time series". It returned 2310 documents. There are 1719 articles, 555 conference papers, 29 reviews, and seven books related to change points. Finally, the last query focused on motif, *i.e.*, "motif" and "time series". It returned 693 documents. The area has 388 articles, 296 conference papers, and nine reviews.

Figure 8.1 clarifies the evolution of the research area. The y-axis presents the number of documents (in logarithm scale) indexed at Scopus. Since anomaly is the main studied subarea, it is reasonable that the first paper started before 1960. After 1980, there was a significant increase in published documents related to change points. The general view of event detection started at the beginning of 2000. Finally, motif started its studies in 2003 in a steady rhythm. It is also interesting to observe that these areas have similar exponential growth captured by the log scale in the y-axis.

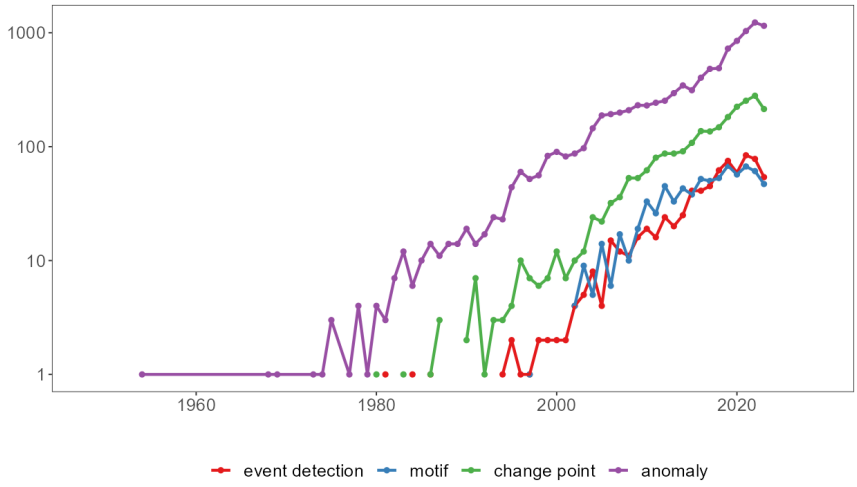


Fig. 8.1 Number of publications (in logarithm scale) event detection according to its main areas

Since it is expected to observe some intersection of documents in these areas, Table 8.2 clarifies this question. Two interesting points are presented in Table 8.2. The first is how general event detection is connected to subareas (anomaly, change point, and motif). In this scenario, it is possible to observe that the major intersection of event detection is related to anomalies (89 out of 693 documents). It includes more than 12% of documents. Nonetheless, both change points and motifs appear related to event detection.

Table 8.2 Intersection between areas

Areas	Amount
Event detection - Anomaly	89
Event detection - Change point	13
Event detection - Motif	4
Anomaly - Change point	215
Anomaly - Motif	65
Change point - Motif	7

The second analysis is related to how anomalies, change points, and motifs might relate. This investigation is particularly interesting since some change point methods can also find anomalies. As expected, the major intersections of the area are related to anomalies with change points. Still, other relationships are present, as can be observed between anomalies and motifs (65 documents) and change points and motifs (7 documents).

In order to provide a qualitative perspective of the subjects presented in the documents, Figure 8.2 depicts a word cloud for the 100 frequent words (after removing English stop words and applying basic text mining preprocessing) of the queried documents. Since the query string always included “time series”, these words were also removed from the analysis. There are four word clouds. Figure 8.2a is related to general event detection (including event prediction). Figures 8.2b, 8.2c, and 8.2d are respectively related to anomalies, change points (with concept drift) and motifs.

One important difference between general event detection and other subareas is that it has fewer application area terms. It has more terms related to data mining, machine learning, and systems. However, many climate-related terms exist in the area of anomalies, meaning many articles published are domain-driven. Besides, change point documents commonly contain many statistical terms, as statistical theories support many of these methods.

Finally, motif documents have a different general vocabulary with many terms associated with patterns, subsequences, and specific data mining functions. It is even worth mentioning that in Figures 8.2a, 8.2b, and 8.2c the word detection is notable. However, discovery is the related concept in Figure 8.2d that stands out.



Fig. 8.2 Word cloud for the top 100 frequent words in event detection: major area of event detection (a), anomaly (b), change point (c), motif (d)

8.2 Summary of challenges

Many research challenges were presented throughout the book. The goal of this section is to summarize the major ones. They are depicted in Figure 8.3.

Multidimensional time series

The first one is related to multidimensional time series. Despite many methods being developed, there is room for further investigation. There are opportunities in data preprocessing, including transformation, decomposition, and dimensionality reduction. There are also opportunities for novel machine learning methods to better explore the relationship among time series, taking advantage of proper data preprocessing.

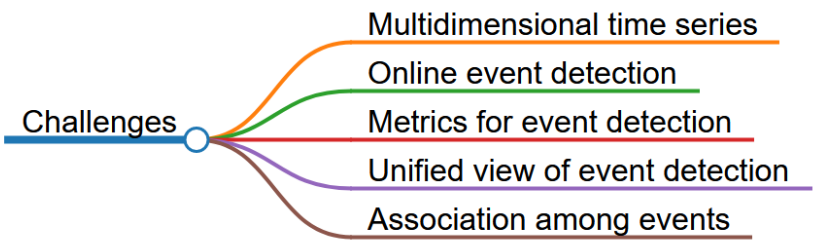


Fig. 8.3 Summary of challenges in event detection

There are also opportunities for ensemble models that combine both univariate and multivariate methods.

Online event detection

Online event detection is a challenging subject. The level of challenge is different according to the area. Detecting change points and motifs as fast as possible is an open problem. Anomalies might be initially viewed as a simpler activity when compared to change points and motifs since some perturbations in time series might be identified as soon as the observations are collected. However, even anomalies might be more difficult to detect in scenarios containing change points.

Metrics for event detection

Chapter 6 started the discussion of challenges regarding the introduction of time tolerance. This discussion allows for establishing metrics for punctual events. However, proper metrics for interval events are still open.

Another challenge regards the perception of detected events that might vary through time in online event detection. In this case, as time goes by, it is possible to associate a score with each detected event. The higher the score, the higher the chance of being an event.

Unified view of event detection

Event detection is a broad area. It is related to anomaly detection, change point detection, and motif discovery. However, most initiatives, such as anomaly detection, are targeted to a single area, which might have a limited view of the phenomena the time series represents. Anomalies, change points, and motifs are intrinsic parts of non-

stationary time series. Properly combining different methods might empower time series analysis and provide more confidence in understanding observed phenomena.

Associating among events

This challenge is similar to sequence mining, a data mining technique that discovers patterns in ordered datasets. Applying such principles is a two-fold goal. First, identify the relationship between events. Then, use this relationship to predict new events once sequential conditions occur. Many applications could take advantage of such a method.

8.3 Harbinger

Considering all the challenges presented, testing state-of-the-art methods to solve event detection problems becomes important. A solution that evaluates many alternative methods and seamlessly enables the development of novel methods is a mandatory activity to empower the area. In this spirit, Harbinger was developed [236] and is made available as an R Package[1].

Harbinger serves as a framework designed for detecting events within time series. It offers a unified environment that identifies time series anomalies, change points, and motifs. The framework encompasses a diverse array of methods for event detection, accompanied by functions for visualizing and assessing the detected events.

Regarding anomaly detection, the methods employed rely on deviations in machine learning models such as `Conv1D`, `ELM`, `MLP`, `LSTM`, `RFR`, and `SVM`. Machine learning classification models like `DTree`, `KNN`, `MLP`, `NB`, `RF`, and `SVM` are also utilized. Clustering techniques, including `K-means`, `DBScan`, and `DTW`, as well as statistical approaches like `ARIMA`, `FBIAD`, and `GARCH`, are also incorporated.

The framework leverages methods based on `AMOC`, `PELT`, `Chow test`,`GFT`, `SCP`, `CF` for change point methods. It also includes methods for concept drift, such as `Page-Hinkley`.

`SAX` and `MP` methods are employed in the motif discovery. Specific techniques tailored for multivariate series are also available. The evaluation of detections encompasses both traditional and soft computing (`SoftED`) approaches.

New methods are constantly being introduced at Harbinger. Additionally, all examples presented in this book used Harbinger are made available on GitHub[2].

[1] Harbinger R package: `https://cefet-rj-dal.github.io/harbinger`

[2] `https://github.com/eogasawara/TSED`

References

[1] Abdel Wahab, O.: Intrusion Detection in the IoT Under Data and Concept Drifts: Online Deep Learning Approach. IEEE Internet of Things Journal **9**(20), 19,706 – 19,716 (2022). DOI 10.1109/JIOT.2022.3167005

[2] Abdi, H., Williams, L.J.: Principal component analysis. Wiley Interdisciplinary Reviews: Computational Statistics **2**(4), 433 – 459 (2010). DOI 10.1002/wics.101

[3] Abhaya, A., Patra, B.K.: An efficient method for autoencoder based outlier detection. Expert Systems with Applications **213** (2023). DOI 10.1016/j.eswa.2022.118904

[4] Afzalan, M., Jazizadeh, F., Wang, J.: Self-configuring event detection in electricity monitoring for human-building interaction. Energy and Buildings **187**, 95 – 109 (2019). DOI 10.1016/j.enbuild.2019.01.036

[5] Aggarwal, C.C.: Data Mining: The Textbook. Springer (2015)

[6] Aggarwal, C.C.: Outlier Analysis. Springer International Publishing (2016)

[7] Aguilar, D.L., Medina-Perez, M.A., Loyola-Gonzalez, O., Choo, K.K.R., Bucheli-Susarrey, E.: Towards an Interpretable Autoencoder: A Decision-Tree-Based Autoencoder and its Application in Anomaly Detection. IEEE Transactions on Dependable and Secure Computing **20**(2), 1048 – 1059 (2023). DOI 10.1109/TDSC.2022.3148331

[8] Ahmad, S., Lavin, A., Purdy, S., Agha, Z.: Unsupervised real-time anomaly detection for streaming data. Neurocomputing **262**, 134 – 147 (2017). DOI 10.1016/j.neucom.2017.04.070

[9] Ahmed, M., Mahmood, A.N., Islam, M.R.: A survey of anomaly detection techniques in financial domain. Future Generation Computer Systems **55**, 278 – 288 (2016). DOI 10.1016/j.future.2015.01.001

[10] Akbarinia, R., Cloez, B.: Efficient Matrix Profile Computation Using Different Distance Functions (2019). DOI 10.48550/arXiv.1901.05708. URL `http://arxiv.org/abs/1901.05708`

[11] Akoglu, L., Tong, H., Koutra, D.: Graph based anomaly detection and description: A survey. Data Mining and Knowledge Discovery **29**(3), 626 – 688 (2015). DOI 10.1007/s10618-014-0365-y

[12] Akpinar, M., Yumusak, N.: Year ahead demand forecast of city natural gas using seasonal time series methods. Energies **9**(9) (2016). DOI 10.3390/en9090727

[13] Al Shalabi, L., Shaaban, Z.: Normalization as a Preprocessing Engine for Data Mining and the Approach of Preference Matrix. In: Proceedings of International Conference on Dependability of Computer Systems, DepCoS-RELCOMEX 2006, pp. 207 – 214 (2006). DOI 10.1109/DEPCOS-RELCOMEX.2006.38

[14] Alaee, S., Kamgar, K., Keogh, E.: Matrix profile XXII: Exact discovery of time series motifs under DTW. In: Proceedings - IEEE International Conference on Data Mining, ICDM, vol. 2020-November, pp. 900 – 905 (2020). DOI 10.1109/ICDM50108.2020.00099

[15] Alaee, S., Mercer, R., Kamgar, K., Keogh, E.: Time series motifs discovery under DTW allows more robust discovery of conserved structure. Data Mining and Knowledge Discovery **35**(3), 863 – 910 (2021). DOI 10.1007/s10618-021-00740-0

[16] Alamr, A., Artoli, A.: Unsupervised Transformer-Based Anomaly Detection in ECG Signals. Algorithms **16**(3) (2023). DOI 10.3390/a16030152

[17] Alevizos, E., Skarlatidis, A., Artikis, A., Paliouras, G.: Probabilistic complex event recognition: A survey. ACM Computing Surveys **50**(5) (2017). DOI 10.1145/3117809

[18] Alimohammadi, H., Nancy Chen, S.: Performance evaluation of outlier detection techniques in production timeseries: A systematic review and meta-analysis. Expert Systems with Applications **191** (2022). DOI 10.1016/j.eswa.2021.116371

[19] Alippi, C., Roveri, M.: Just-in-time adaptive classifiers - Part I: Detecting nonstationary changes. IEEE Transactions on Neural Networks **19**(7), 1145 – 1153 (2008). DOI 10.1109/TNN.2008.2000082

[20] Amador Coelho, R., Bambirra Torres, L.C., Leite de Castro, C.: Concept drift detection with quadtree-based spatial mapping of streaming data. Information Sciences **625**, 578 – 592 (2023). DOI 10.1016/j.ins.2022.12.085

[21] Aminikhanghahi, S., Cook, D.J.: A survey of methods for time series change point detection. Knowledge and Information Systems **51**(2), 339 – 367 (2017). DOI 10.1007/s10115-016-0987-z

[22] Amodeo, G., Blanco, R., Brefeld, U.: Hybrid models for future event prediction. In: International Conference on Information and Knowledge Management, Proceedings, pp. 1981 – 1984 (2011). DOI 10.1145/2063576.2063870

[23] An, X., Jiang, D., Liu, C., Zhao, M.: Wind farm power prediction based on wavelet decomposition and chaotic time series. Expert Systems with Applications **38**(9), 11,280 – 11,285 (2011). DOI 10.1016/j.eswa.2011.02.176

[24] Andrews, D.W.K.: Tests for Parameter Instability and Structural Change With Unknown Change Point. Econometrica **61**(4), 821–856 (1993). DOI 10.2307/2951764

[25] Andrienko, G., Andrienko, N., Mladenov, M., Mock, M., Poelitz, C.: Extracting events from spatial time series. In: Proceedings of the Interna-

tional Conference on Information Visualisation, pp. 48 – 53 (2010). DOI 10.1109/IV.2010.17

[26] Ané, T., Ureche-Rangau, L., Gambet, J.B., Bouverot, J.: Robust outlier detection for Asia-Pacific stock index returns. Journal of International Financial Markets, Institutions and Money **18**(4), 326 – 343 (2008). DOI 10.1016/j.intfin.2007.03.001

[27] Angelov, P., Zhou, X.: On line learning fuzzy rule-based system structure from data streams. In: IEEE International Conference on Fuzzy Systems, pp. 915 – 922 (2008). DOI 10.1109/FUZZY.2008.4630479

[28] Angiulli, F., Pizzuti, C.: Fast outlier detection in high dimensional spaces. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) **2431 LNAI**, 15 – 27 (2002). DOI 10.1007/3-540-45681-3_2

[29] Ariyaluran Habeeb, R.A., Nasaruddin, F., Gani, A., Targio Hashem, I.A., Ahmed, E., Imran, M.: Real-time big data processing for anomaly detection: A Survey. International Journal of Information Management **45**, 289 – 307 (2019). DOI 10.1016/j.ijinfomgt.2018.08.006

[30] Atto, A.M., Berthoumieu, Y.: Wavelet packets of nonstationary random processes: Contributing factors for stationarity and decorrelation. IEEE Transactions on Information Theory **58**(1), 317 – 330 (2012). DOI 10.1109/TIT.2011.2167496

[31] Babüroğlu, E.S., Durmuşoğlu, A., Dereli, T.: Concept drift from 1980 to 2020: a comprehensive bibliometric analysis with future research insight. Evolving Systems (2023). DOI 10.1007/s12530-023-09503-2

[32] Bai, J., Perron, P.: Computation and analysis of multiple structural change models. Journal of Applied Econometrics **18**(1), 1 – 22 (2003). DOI 10.1002/jae.659

[33] Bayram, F., Ahmed, B.S., Kassler, A.: From concept drift to model degradation: An overview on performance-aware drift detectors. Knowledge-Based Systems **245** (2022). DOI 10.1016/j.knosys.2022.108632

[34] Belcaid, A., Belkbir, H.: Constrained energy variation for change point detection. Multidimensional Systems and Signal Processing **33**(1), 63 – 80 (2022). DOI 10.1007/s11045-021-00785-w

[35] Benkabou, S.E., Benabdeslem, K., Canitia, B.: Unsupervised outlier detection for time series by entropy and dynamic time warping. Knowledge and Information Systems **54**(2), 463 – 486 (2018). DOI 10.1007/s10115-017-1067-8

[36] Bentkus, V.: On Hoeffding's inequalities. Annals of Probability **32**(2), 1650 – 1673 (2004). DOI 10.1214/009117904000000360

[37] Bifet, A., Gavaldà, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 7th SIAM International Conference on Data Mining, pp. 443 – 448 (2007). DOI 10.1137/1.9781611972771.42

[38] Blázquez-García, A., Conde, A., Mori, U., Lozano, J.A.: A Review on Outlier/Anomaly Detection in Time Series Data. ACM Computing Surveys **54**(3) (2021). DOI 10.1145/3444690

[39] Boniol, P., Palpanas, T.: Series2graph: Graph-based subsequence anomaly detection for time series. Proceedings of the VLDB Endowment **13**(11), 1821 – 1834 (2020). DOI 10.14778/3407790.3407792

[40] Boniol, P., Paparrizos, J., Kang, Y., Palpanas, T., Tsay, R.S., Elmore, A.J., Franklin, M.J.: Theseus: Navigating the Labyrinth of Time-Series Anomaly Detection. Proceedings of the VLDB Endowment **15**(12), 3702 – 3705 (2022). DOI 10.14778/3554821.3554879

[41] Boniol, P., Paparrizos, J., Palpanas, T.: New Trends in Time-Series Anomaly Detection. In: Advances in Database Technology - EDBT, vol. 26, pp. 847 – 850 (2023). DOI 10.48786/edbt.2023.80

[42] Boniol, P., Paparrizos, J., Palpanas, T., Franklin, M.J.: Sand in action: Subsequence anomaly detection for streams. Proceedings of the VLDB Endowment **14**(12), 2867 – 2870 (2021). DOI 10.14778/3476311.3476365

[43] Borges, H., Dutra, M., Bazaz, A., Coutinho, R., Perosi, F., Porto, F., Masseglia, F., Pacitti, E., Ogasawara, E.: Spatial-time motifs discovery. Intelligent Data Analysis **24**(5), 1121 – 1140 (2020). DOI 10.3233/IDA-194759

[44] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., Ljung, G.M.: Time Series Analysis: Forecasting and Control. John Wiley & Sons (2015)

[45] Bozdogan, H.: Model selection and Akaike's Information Criterion (AIC): The general theory and its analytical extensions. Psychometrika **52**(3), 345 – 370 (1987). DOI 10.1007/BF02294361

[46] Bracewell, R.N., Bracewell, R.: The Fourier Transform and Its Applications. McGraw Hill (2000)

[47] Braei, M., Wagner, S.: Anomaly Detection in Univariate Time-series: A Survey on the State-of-the-Art (2020). DOI 10.48550/arXiv.2004.00433. URL `http://arxiv.org/abs/2004.00433`

[48] Breiman, L.: Random forests. Machine Learning **45**(1), 5 – 32 (2001). DOI 10.1023/A:1010933404324

[49] Brockwell, P.J., Davis, R.A.: Introduction to Time Series and Forecasting. Springer International Publishing (2016)

[50] Buda, T.S., Assem, H., Xu, L.: ADE: An ensemble approach for early Anomaly Detection. In: Proceedings of the IM 2017 - 2017 IFIP/IEEE International Symposium on Integrated Network and Service Management, pp. 442 – 448 (2017). DOI 10.23919/INM.2017.7987310

[51] Buhler, J., Tompa, M.: Finding motifs using random projections. Journal of Computational Biology **9**(2), 225 – 242 (2002). DOI 10.1089/10665270252935430

[52] Buza, K.: Time series classification and its applications. In: ACM International Conference Proceeding Series (2018). DOI 10.1145/3227609.3227690

[53] Calikus, E., Nowaczyk, S., Sant'Anna, A., Dikmen, O.: No free lunch but a cheaper supper: A general framework for streaming anomaly detection. Expert Systems with Applications **155** (2020). DOI 10.1016/j.eswa.2020.113453

[54] Carmona, R.: Statistical Analysis of Financial Data in R. Springer Science & Business Media (2013)

[55] Castro, N., Azevedo, P.: Multiresolution motif discovery in time series. In: Proceedings of the 10th SIAM International Conference on Data Mining, SDM 2010, pp. 665 – 676 (2010). DOI 10.1137/1.9781611972801.73

[56] Castro, N.C., Azevedo, P.J.: Significant motifs in time series. Statistical Analysis and Data Mining **5**(1), 35 – 53 (2012). DOI 10.1002/sam.11134

[57] Cauteruccio, F., Cinelli, L., Corradini, E., Terracina, G., Ursino, D., Virgili, L., Savaglio, C., Liotta, A., Fortino, G.: A framework for anomaly detection and classification in Multiple IoT scenarios. Future Generation Computer Systems **114**, 322 – 335 (2021). DOI 10.1016/j.future.2020.08.010

[58] Cerdà-Alabern, L., Iuhasz, G., Gemmi, G.: Anomaly detection for fault detection in wireless community networks using machine learning. Computer Communications **202**, 191 – 203 (2023). DOI 10.1016/j.comcom.2023.02.019

[59] Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3) (2009). DOI 10.1145/1541880.1541882

[60] Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection for discrete sequences: A survey. IEEE Transactions on Knowledge and Data Engineering **24**(5), 823 – 839 (2012). DOI 10.1109/TKDE.2010.235

[61] Chandola, V., Mithal, V., Kumar, V.: Comparative evaluation of anomaly detection techniques for sequence data. In: Proceedings - IEEE International Conference on Data Mining, ICDM, pp. 743 – 748 (2008). DOI 10.1109/ICDM.2008.151

[62] Chatzikonstanti, V.: Breaks and outliers when modelling the volatility of the U.S. stock market. Applied Economics **49**(46), 4704 – 4717 (2017). DOI 10.1080/00036846.2017.1293785

[63] Chawla, S., Gionisy, A.: k-means-: A unified approach to clustering and outlier detection. In: Proceedings of the 2013 SIAM International Conference on Data Mining, SDM 2013, pp. 189 – 197 (2013). DOI 10.1137/1.9781611972832.21

[64] Chen, J., Gupta, A.K.: Parametric Statistical Change Point Analysis. Springer Science & Business Media (2013)

[65] Chen, X., Huang, K., Jiang, H.: Detecting Changes in the Spatiotemporal Pattern of Bike Sharing: A Change-Point Topic Model. IEEE Transactions on Intelligent Transportation Systems **23**(10), 18,361 – 18,377 (2022). DOI 10.1109/TITS.2022.3161623

[66] Cheng, C., Sa-Ngasoongsong, A., Beyca, O., Le, T., Yang, H., Kong, Z., Bukkapatnam, S.T.: Time series forecasting for nonlinear and non-stationary processes: A review and comparative study. IIE Transactions (Institute of Industrial Engineers) **47**(10), 1053 – 1071 (2015). DOI 10.1080/0740817X.2014.999180

[67] Chib, S.: Estimation and comparison of multiple change-point models. Journal of Econometrics **86**(2), 221 – 241 (1998). DOI 10.1016/S0304-4076(97)00115-2

[68] Chicco, D., Warrens, M.J., Jurman, G.: The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. PeerJ Computer Science **7**, 1 – 24 (2021). DOI 10.7717/PEERJ-CS.623

[69] Chiu, B., Keogh, E., Lonardi, S.: Probabilistic discovery of time series motifs. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 493 – 498 (2003). DOI 10.1145/956750.956808

[70] Chow, G.C.: Tests of Equality Between Sets of Coefficients in Two Linear Regressions. Econometrica **28**(3), 591–605 (1960). DOI 10.2307/1910133

[71] Clements, M.P., Hendry, D.F.: A Companion to Economic Forecasting. John Wiley & Sons (2008)

[72] Coles, S.: An Introduction to Statistical Modeling of Extreme Values. Springer Science & Business Media (2013)

[73] Conejo, A.J., Plazas, M.A., Espínola, R., Molina, A.B.: Day-ahead electricity price forecasting using the wavelet transform and ARIMA models. IEEE Transactions on Power Systems **20**(2), 1035 – 1042 (2005). DOI 10.1109/TPWRS.2005.846054

[74] Cook, A.A., Misirli, G., Fan, Z.: Anomaly Detection for IoT Time-Series Data: A Survey. IEEE Internet of Things Journal **7**(7), 6481 – 6494 (2020). DOI 10.1109/JIOT.2019.2958185

[75] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. MIT Press (2009)

[76] Cryer, J.D., Chan, K.S.: Time Series Analysis: With Applications in R. Springer Science & Business Media (2008)

[77] Darban, Z.Z., Webb, G.I., Pan, S., Aggarwal, C.C., Salehi, M.: Deep Learning for Time Series Anomaly Detection: A Survey (2022). DOI 10.48550/arXiv.2211.05244. URL http://arxiv.org/abs/2211.05244

[78] Daw, C., Finney, C., Tracy, E.: A review of symbolic analysis of experimental data. Review of Scientific Instruments **74**(2), 915 – 930 (2003). DOI 10.1063/1.1531823

[79] De Paepe, D., Vanden Hautte, S., Steenwinckel, B., De Turck, F., Ongenae, F., Janssens, O., Van Hoecke, S.: A generalized matrix profile framework with support for contextual series analysis. Engineering Applications of Artificial Intelligence **90** (2020). DOI 10.1016/j.engappai.2020.103487

[80] Desforges, M., Jacob, P., Cooper, J.: Applications of probability density estimation to the detection of abnormal conditions in engineering. Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science **212**(8), 687 – 703 (1998). DOI 10.1243/0954406981521448

[81] Desobry, F., Davy, M., Doncarli, C.: An online Kernel change detection algorithm. IEEE Transactions on Signal Processing **53**(8), 2961 – 2974 (2005). DOI 10.1109/TSP.2005.851098

[82] Diab, D.M., Assadhan, B., Binsalleeh, H., Lambotharan, S., Kyriakopoulos, K.G., Ghafir, I.: Anomaly detection using dynamic time warping. In: Proceedings - 22nd IEEE International Conference on Computational Science and Engineering and 17th IEEE International Conference on Embedded and Ubiquitous Computing, CSE/EUC 2019, pp. 193 – 198 (2019). DOI 10.1109/CSE/EUC.2019.00045

[83] Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., Keogh, E.: Querying and mining of time series data: Experimental comparison of representations and distance measures. In: Proceedings of the VLDB Endowment, vol. 1, pp. 1542 – 1552 (2008). DOI 10.14778/1454159.1454226

[84] Ding, J., Xiang, Y., Shen, L., Tarokh, V.: Multiple Change Point Analysis: Fast Implementation and Strong Consistency. IEEE Transactions on Signal Processing **65**(17), 4495 – 4510 (2017). DOI 10.1109/TSP.2017.2711558

[85] Ding, X., Li, Y., Belatreche, A., Maguire, L.P.: An experimental evaluation of novelty detection methods. Neurocomputing **135**, 313 – 327 (2014). DOI 10.1016/j.neucom.2013.12.002

[86] DIsabato, S., Roveri, M.: Learning Convolutional Neural Networks in presence of Concept Drift. In: Proceedings of the International Joint Conference on Neural Networks, vol. 2019-July (2019). DOI 10.1109/IJCNN.2019.8851731

[87] Ditzler, G., Roveri, M., Alippi, C., Polikar, R.: Learning in Nonstationary Environments: A Survey. IEEE Computational Intelligence Magazine **10**(4), 12 – 25 (2015). DOI 10.1109/MCI.2015.2471196

[88] Dixit, P., Bhattacharya, P., Tanwar, S., Gupta, R.: Anomaly detection in autonomous electric vehicles using AI techniques: A comprehensive survey. Expert Systems **39**(5) (2022). DOI 10.1111/exsy.12754

[89] Dudek, G.: Neural networks for pattern-based short-term load forecasting: A comparative study. Neurocomputing **205**, 64 – 74 (2016). DOI 10.1016/j.neucom.2016.04.021

[90] Egilmez, H.E., Ortega, A.: Spectral anomaly detection using graph-based filtering for wireless sensor networks. In: ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings, pp. 1085 – 1089 (2014). DOI 10.1109/ICASSP.2014.6853764

[91] Elliott, G., Rothenberg, T.J., Stock, J.H.: Efficient tests for an autoregressive unit root. Econometrica **64**(4), 813 – 836 (1996). DOI 10.2307/2171846

[92] Ermshaus, A., Schäfer, P., Leser, U.: ClaSP: parameter-free time series segmentation. Data Mining and Knowledge Discovery **37**(3), 1262–1300 (2023). DOI 10.1007/s10618-023-00923-x

[93] Esling, P., Agon, C.: Time-series data mining. ACM Computing Surveys **45**(1) (2012). DOI 10.1145/2379776.2379788

[94] Ester, M., Kriegel, H.P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, KDD'96, pp. 226–231. AAAI Press, Portland, Oregon (1996)

[95] Fanaee-T, H., Gama, J.: Event labeling combining ensemble detectors and background knowledge. Progress in Artificial Intelligence **2**(2-3), 113 – 127 (2014). DOI 10.1007/s13748-013-0040-3

[96] Flach, M., Gans, F., Brenning, A., Denzler, J., Reichstein, M., Rodner, E., Bathiany, S., Bodesheim, P., Guanche, Y., Sippel, S., Mahecha, M.D.: Multivariate anomaly detection for Earth observations: A comparison of algorithms and feature extraction techniques. Earth System Dynamics **8**(3), 677 – 696 (2017). DOI 10.5194/esd-8-677-2017

[97] Flandrin, P., Gonçalvès, P., Rilling, G.: Detrending and denoising with empirical mode decompositions. In: 2004 12th European Signal Processing Conference, pp. 1581–1584 (2004)

[98] Flandrin, P., Rilling, G., Gonçalvés, P.: Empirical mode decomposition as a filter bank. IEEE Signal Processing Letters **11**(2 PART I), 112 – 114 (2004). DOI 10.1109/LSP.2003.821662

[99] Flórez, A., Rodríguez-Moreno, I., Artetxe, A., Olaizola, I.G., Sierra, B.: CatSight, a direct path to proper multi-variate time series change detection: perceiving a concept drift through common spatial pattern. International Journal of Machine Learning and Cybernetics **14**(9), 2925 – 2944 (2023). DOI 10.1007/s13042-023-01810-z

[100] Frías-Blanco, I., Del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., Caballero-Mota, Y.: Online and non-parametric drift detection methods based on Hoeffding's bounds. IEEE Transactions on Knowledge and Data Engineering **27**(3), 810 – 823 (2015). DOI 10.1109/TKDE.2014.2345382

[101] Fuchs, E., Gruber, T., Nitschke, J., Sick, B.: On-line motif detection in time series with SwiftMotif. Pattern Recognition **42**(11), 3015 – 3031 (2009). DOI 10.1016/j.patcog.2009.05.004

[102] Gabarda, S., Cristóbal, G.: Detection of events in seismic time series by time-frequency methods. IET Signal Processing **4**(4), 413 – 420 (2010). DOI 10.1049/iet-spr.2009.0125

[103] Gama, J., Medas, P., Castillo, G., Rodrigues, P.: Learning with drift detection. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) **3171**, 286 – 295 (2004). DOI 10.1007/978-3-540-28645-5_29

[104] Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Computing Surveys **46**(4) (2014). DOI 10.1145/2523813

[105] Garcia, S., Luengo, J., Herrera, F.: Data Preprocessing in Data Mining. Springer (2014)

[106] Gedda, R., Beilina, L., Tan, R.: Change Point Detection for Process Data Analytics Applied to a Multiphase Flow Facility. CMES - Computer Modeling in Engineering and Sciences **134**(3), 1737 – 1759 (2023). DOI 10.32604/cmes.2022.019764

[107] Giusti, L., Carvalho, L., Gomes, A.T., Coutinho, R., Soares, J., Ogasawara, E.: Analyzing flight delay prediction under concept drift. Evolving Systems (2022). DOI 10.1007/s12530-021-09415-z

[108] Gonçalves, P.M., De Carvalho Santos, S.G., Barros, R.S., Vieira, D.C.: A comparative study on concept drift detectors. Expert Systems with Applications **41**(18), 8144 – 8156 (2014). DOI 10.1016/j.eswa.2014.07.019

[109] Gran, A., Veiga, H.: Wavelet-based detection of outliers in financial time series. Computational Statistics and Data Analysis **54**(11), 2580 – 2593 (2010). DOI 10.1016/j.csda.2009.12.010

[110] Greff, K., Srivastava, R.K., Koutnik, J., Steunebrink, B.R., Schmidhuber, J.: LSTM: A Search Space Odyssey. IEEE Transactions on Neural Networks and Learning Systems **28**(10), 2222 – 2232 (2017). DOI 10.1109/TNNLS.2016.2582924

[111] Grubbs, F.E.: Procedures for Detecting Outlying Observations in Samples. Technometrics **11**(1), 1 – 21 (1969). DOI 10.1080/00401706.1969.10490657

[112] Guerra-Manzanares, A., Bahsi, H.: On the relativity of time: Implications and challenges of data drift on long-term effective android malware detection. Computers and Security **122** (2022). DOI 10.1016/j.cose.2022.102835

[113] Guha, S., Rastogi, R., Shim, K.: Rock: a robust clustering algorithm for categorical attributes. Information Systems **25**(5), 345 – 366 (2000). DOI 10.1016/S0306-4379(00)00022-3

[114] Gujarati, D.N.: Essentials of Econometrics. SAGE (2021)

[115] Guo, T., Dong, J., Li, H., Gao, Y.: Simple convolutional neural network on image classification. In: 2017 IEEE 2nd International Conference on Big Data Analysis, ICBDA 2017, pp. 721 – 724 (2017). DOI 10.1109/ICBDA.2017.8078730

[116] Gupta, M., Gao, J., Aggarwal, C.C., Han, J.: Outlier Detection for Temporal Data: A Survey. IEEE Transactions on Knowledge and Data Engineering **26**(9), 2250 – 2267 (2014). DOI 10.1109/TKDE.2013.184

[117] Guralnik, V., Srivastava, J.: Event Detection from Time Series Data. In: Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '99, pp. 33–42. ACM, New York, NY, USA (1999). DOI 10.1145/312129.312190

[118] Han, J., Pei, J., Tong, H.: Data Mining: Concepts and Techniques. 4th edition edn. Morgan Kaufmann, Cambridge, MA (2022)

[119] Han, M., Chen, Z., Li, M., Wu, H., Zhang, X.: A survey of active and passive concept drift handling methods. Computational Intelligence **38**(4), 1492 – 1535 (2022). DOI 10.1111/coin.12520

[120] Hanssens, D.M., Parsons, L.J., Schultz, R.L.: Market Response Models: Econometric and Time Series Analysis. Springer Science & Business Media (2012)

[121] Haykin, S.O.: Neural Networks and Learning Machines. Pearson Education (2011)

[122] Herbert, T., Mangler, J., Rinderle-Ma, S.: Generating Reliable Process Event Streams and Time Series Data Based on Neural Networks. Lecture Notes in Business Information Processing **421**, 81 – 95 (2021). DOI 10.1007/978-3-030-79186-5_6

[123] Hinkley, D.V.: Inference about the change-point in a sequence of random variables. Biometrika **57**(1), 1 – 17 (1970). DOI 10.1093/biomet/57.1.1

[124] Hlávka, Z., Hušková, M., Meintanis, S.G.: Change-point methods for multivariate time-series: paired vectorial observations. Statistical Papers **61**(4), 1351 – 1383 (2020). DOI 10.1007/s00362-020-01175-3

[125] Hodge, V.J., Austin, J.: A survey of outlier detection methodologies. Artificial Intelligence Review **22**(2), 85 – 126 (2004). DOI 10.1023/B:AIRE.0000045502.10941.a9

[126] Hoens, T., Polikar, R., Chawla, N.V.: Learning from streaming data with concept drift and imbalance: An overview. Progress in Artificial Intelligence **1**(1), 89 – 101 (2012). DOI 10.1007/s13748-011-0008-0

[127] Huang, G.B., Zhu, Q.Y., Siew, C.K.: Extreme learning machine: Theory and applications. Neurocomputing **70**(1-3), 489 – 501 (2006). DOI 10.1016/j.neucom.2005.12.126

[128] Huang, N.E., Shen, Z., Long, S.R., Wu, M.C., Snin, H.H., Zheng, Q., Yen, N.C., Tung, C.C., Liu, H.H.: The empirical mode decomposition and the Hubert spectrum for nonlinear and non-stationary time series analysis. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences **454**(1971), 903 – 995 (1998). DOI 10.1098/rspa.1998.0193

[129] Hwang, W.S., Yun, J.H., Kim, J., Kim, H.C.: Time-series aware precision and recall for anomaly detection considering variety of detection result and addressing ambiguous labeling. In: International Conference on Information and Knowledge Management, Proceedings, pp. 2241 – 2244 (2019). DOI 10.1145/3357384.3358118

[130] Hyndman, R.J., Athanasopoulos, G.: Forecasting: principles and practice. OTexts (2018)

[131] Ibidunmoye, O., Hernández-Rodriguez, F., Elmroth, E.: Performance anomaly detection and bottleneck identification. ACM Computing Surveys **48**(1) (2015). DOI 10.1145/2791120

[132] Iwashita, A.S., Papa, J.P.: An Overview on Concept Drift Learning. IEEE Access **7**, 1532 – 1547 (2019). DOI 10.1109/ACCESS.2018.2886026

[133] Jacob, V., Song, F., Stiegler, A., Rad, B., Diao, Y., Tatbul, N.: Exathlon: A benchmark for explainable anomaly detection over time series. Proceedings of the VLDB Endowment **14**(11), 2613 – 2626 (2021). DOI 10.14778/3476249.3476307

[134] Jain, A.K.: Data clustering: 50 years beyond K-means. Pattern Recognition Letters **31**(8), 651 – 666 (2010). DOI 10.1016/j.patrec.2009.09.011

[135] James, G.M., Witten, D., Hastie, T., Tibshirani, R.: An Introduction to Statistical Learning: With Applications in R. Springer Nature (2021)

[136] Jensen, S.K., Pedersen, T.B., Thomsen, C.: Time Series Management Systems: A Survey. IEEE Transactions on Knowledge and Data Engineering **29**(11), 2581 – 2600 (2017). DOI 10.1109/TKDE.2017.2740932

[137] Jones, M., Nikovski, D., Imamura, M., Hirata, T.: Exemplar learning for extremely efficient anomaly detection in real-valued time series. Data Mining and Knowledge Discovery **30**(6), 1427 – 1454 (2016). DOI 10.1007/s10618-015-0449-3

[138] Joo, T.W., Kim, S.B.: Time series forecasting based on wavelet filtering. Expert Systems with Applications **42**(8), 3868 – 3874 (2015). DOI 10.1016/j.eswa.2015.01.026

[139] Jouini, J.: Bootstrap methods for single structural change tests: Power versus corrected size and empirical illustration. Statistical Papers **51**(1), 85–109 (2010). DOI 10.1007/s00362-008-0123-6

[140] Kammammettu, S., Li, Z.: Change point and fault detection using Kantorovich Distance. Journal of Process Control **80**, 41 – 59 (2019). DOI 10.1016/j.jprocont.2019.05.012

[141] Kattan, A., Fatima, S., Arif, M.: Time-series event-based prediction: An unsupervised learning framework based on genetic programming. Information Sciences **301**, 99 – 123 (2015). DOI 10.1016/j.ins.2014.12.054

[142] Kaufman, L., Rousseeuw, P.J.: Finding Groups in Data: An Introduction to Cluster Analysis. John Wiley & Sons (2009)

[143] Keogh, E., Kasetty, S.: On the Need for Time Series Data Mining Benchmarks: A Survey and Empirical Demonstration. Data Mining and Knowledge Discovery **7**(4), 349 – 371 (2003). DOI 10.1023/A:1024988512476

[144] Keogh, E., Lin, J.: Clustering of time-series subsequences is meaningless: Implications for previous and future research. Knowledge and Information Systems **8**(2), 154 – 177 (2005). DOI 10.1007/s10115-004-0172-7

[145] Keogh, E., Lin, J., Fu, A.: HOT SAX: Efficiently finding the most unusual time series subsequence. In: Proceedings - IEEE International Conference on Data Mining, ICDM, pp. 8 – 15 (2005). DOI 10.1109/ICDM.2005.79

[146] Keogh, E., Lin, J., Fu, A.W., Van Herle, H.: Finding unusual medical time-series subsequences: Algorithms and applications. IEEE Transactions on Information Technology in Biomedicine **10**(3), 429 – 439 (2006). DOI 10.1109/TITB.2005.863870

[147] Keogh, E., Lonardi, S., Ratanamahatana, C.A.: Towards parameter-free data mining. In: KDD-2004 - Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 206 – 215 (2004). DOI 10.1145/1014052.1014077

[148] Keogh, E., Ratanamahatana, C.A.: Exact indexing of dynamic time warping. Knowledge and Information Systems **7**(3), 358 – 386 (2005). DOI 10.1007/s10115-004-0154-9

[149] Khamassi, I., Sayed-Mouchaweh, M., Hammami, M., Ghédira, K.: Discussion and review on evolving data streams and concept drift adapting. Evolving Systems **9**(1), 1 – 23 (2018). DOI 10.1007/s12530-016-9168-2

[150] Kieu, T., Yang, B., Jensen, C.S.: Outlier detection for multidimensional time series using deep neural networks. In: Proceedings - IEEE International Conference on Mobile Data Management, vol. 2018-June, pp. 125 – 134 (2018). DOI 10.1109/MDM.2018.00029

[151] Killick, R., Eckley, I.A.: Changepoint: An R package for changepoint analysis. Journal of Statistical Software **58**(3), 1 – 19 (2014). DOI 10.18637/jss.v058.i03

[152] Killick, R., Fearnhead, P., Eckley, I.: Optimal detection of changepoints with a linear computational cost. Journal of the American Statistical Association **107**(500), 1590 – 1598 (2012). DOI 10.1080/01621459.2012.737745

[153] Kim, H., Kim, B., Chung, D., Yoon, J., Ko, S.K.: SoccerCPD: Formation and Role Change-Point Detection in Soccer Matches Using Spatiotemporal Tracking Data. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 3146 – 3156 (2022). DOI 10.1145/3534678.3539150

[154] Kiskin, I., Zilli, D., Li, Y., Sinka, M., Willis, K., Roberts, S.: Bioacoustic detection with wavelet-conditioned convolutional neural networks. Neural Computing and Applications **32**(4), 915 – 927 (2020). DOI 10.1007/s00521-018-3626-7

[155] Kloska, M., Grmanova, G., Rozinajova, V.: Expert enhanced dynamic time warping based anomaly detection. Expert Systems with Applications **225** (2023). DOI 10.1016/j.eswa.2023.120030

[156] Lahmiri, S.: Interest rate next-day variation prediction based on hybrid feedforward neural network, particle swarm optimization, and multiresolution techniques. Physica A: Statistical Mechanics and its Applications **444**, 388 – 396 (2016). DOI 10.1016/j.physa.2015.09.061

[157] Lam, H.T., Pham, N.D., Calders, T.: Online discovery of top-k similar motifs in time series data. In: Proceedings of the 11th SIAM International Conference on Data Mining, SDM 2011, pp. 1004 – 1015 (2011). DOI 10.1137/1.9781611972818.86

[158] Lampert, C.H., Blaschko, M.B., Hofmann, T.: Beyond sliding windows: Object localization by efficient subwindow search. In: 26th IEEE Conference on Computer Vision and Pattern Recognition, CVPR (2008). DOI 10.1109/CVPR.2008.4587586

[159] Lavin, A., Ahmad, S.: Evaluating real-time anomaly detection algorithms - The numenta anomaly benchmark. In: Proceedings - 2015 IEEE 14th International Conference on Machine Learning and Applications, ICMLA 2015, pp. 38 – 44 (2016). DOI 10.1109/ICMLA.2015.141

[160] Laxhammar, R., Falkman, G.: Inductive conformal anomaly detection for sequential detection of anomalous sub-trajectories. Annals of Mathematics and Artificial Intelligence **74**(1-2), 67 – 94 (2015). DOI 10.1007/s10472-013-9381-7

[161] Lazarevic, A., Kumar, V.: Feature bagging for outlier detection. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 157 – 166 (2005). DOI 10.1145/1081870.1081891

[162] Li, J., Izakian, H., Pedrycz, W., Jamal, I.: Clustering-based anomaly detection in multivariate time series data. Applied Soft Computing **100** (2021). DOI 10.1016/j.asoc.2020.106919

[163] Lim, B., Zohren, S.: Time-series forecasting with deep learning: A survey. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences **379**(2194) (2021). DOI 10.1098/rsta.2020.0209

[164] Lima, J., Salles, R., Porto, F., Coutinho, R., Alpis, P., Escobar, L., Pacitti, E., Ogasawara, E.: Forward and Backward Inertial Anomaly Detector: A Novel Time Series Event Detection Method. In: Proceedings of the International

Joint Conference on Neural Networks, vol. 2022-July (2022). DOI 10.1109/IJCNN55064.2022.9892088

[165] Lin, J., Keogh, E., Lonardi, S., Chiu, B.: A symbolic representation of time series, with implications for streaming algorithms. In: Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, DMKD '03, pp. 2 – 11 (2003). DOI 10.1145/882082.882086

[166] Lin, J., Keogh, E., Wei, L., Lonardi, S.: Experiencing SAX: A novel symbolic representation of time series. Data Mining and Knowledge Discovery **15**(2), 107 – 144 (2007). DOI 10.1007/s10618-007-0064-z

[167] Linardi, M., Zhu, Y., Palpanas, T., Keogh, E.: Matrix profile goes MAD: variable-length motif and discord discovery in data series. Data Mining and Knowledge Discovery **34**(4), 1022 – 1071 (2020). DOI 10.1007/s10618-020-00685-w

[168] Liu, A., Song, Y., Zhang, G., Lu, J.: Regional concept drift detection and density synchronized drift adaptation. In: IJCAI International Joint Conference on Artificial Intelligence, vol. 0, pp. 2280 – 2286 (2017). DOI 10.24963/ijcai.2017/317

[169] Liu, B., Zhao, H., Liu, Y., Wang, S., Li, J., Li, Y., Lang, J., Gu, R.: Discovering multi-dimensional motifs from multi-dimensional time series for air pollution control. Concurrency and Computation: Practice and Experience **32**(11) (2020). DOI 10.1002/cpe.5645

[170] Liu, J., Yang, D., Zhang, K., Gao, H., Li, J.: Anomaly and change point detection for time series with concept drift. World Wide Web (2023). DOI 10.1007/s11280-023-01181-z

[171] Liu, S., Smith, K., Che, H.: A multivariate based event detection method and performance comparison with two baseline methods. Water Research **80**, 109 – 118 (2015). DOI 10.1016/j.watres.2015.05.013

[172] Loh, W.K., Kim, S.W., Whang, K.Y.: Index Interpolation: An Approach to Subsequence Matching Supporting Normalization Transform in Time-Series Databases. In: International Conference on Information and Knowledge Management, Proceedings, vol. 2000-January, pp. 480 – 487 (2000). DOI 10.1145/354756.354856

[173] Los, C.: Financial Market Risk: Measurement and Analysis. Routledge (2003)

[174] Loureiro, D., ao Amado, C., Martins, A., Vitorino, D., Mamade, A., Coelho, S.T.: Water distribution systems flow monitoring and anomalous event detection: A practical approach. Urban Water Journal **13**(3), 242 – 252 (2016). DOI 10.1080/1573062X.2014.988733

[175] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under Concept Drift: A Review. IEEE Transactions on Knowledge and Data Engineering **31**(12), 2346 – 2363 (2019). DOI 10.1109/TKDE.2018.2876857

[176] Lughofer, E., Angelov, P.: Handling drifts and shifts in on-line data streams with evolving fuzzy systems. Applied Soft Computing Journal **11**(2), 2057 – 2068 (2011). DOI 10.1016/j.asoc.2010.07.003

[177] Machado, E., Serqueira, M., Ogasawara, E., Ogando, R., Maia, M.A.G., Da Costa, L.N., Campisano, R., Paiva Guedes, G., Bezerra, E.: Exploring machine

learning methods for the Star/Galaxy Separation Problem. In: Proceedings of the International Joint Conference on Neural Networks, vol. 2016-October, pp. 123 – 130 (2016). DOI 10.1109/IJCNN.2016.7727189

[178] McGovern, A., Rosendahl, D.H., Brown, R.A., Droegemeier, K.K.: Identifying predictive multi-dimensional time series motifs: An application to severe weather prediction. Data Mining and Knowledge Discovery **22**(1-2), 232 – 258 (2011). DOI 10.1007/s10618-010-0193-7

[179] McKenna, S.A., Hart, D., Klise, K., Cruz, V., Wilson, M.: Event detection from water quality time series. In: Restoring Our Natural Habitat - Proceedings of the 2007 World Environmental and Water Resources Congress (2007). DOI 10.1061/40927(243)518

[180] Mehrmolaei, S., Keyvanpourr, M.R.: A brief survey on event prediction methods in time series. Advances in Intelligent Systems and Computing **347**, 235 – 246 (2015). DOI 10.1007/978-3-319-18476-0_24

[181] Memarzadeh, M., Matthews, B., Avrekh, I.: Unsupervised anomaly detection in flight data using convolutional variational auto-encoder. Aerospace **7**(8) (2020). DOI 10.3390/AEROSPACE7080115

[182] Mills, T.C., Markellos, R.N.: The Econometric Modelling of Financial Time Series. Cambridge University Press (2008)

[183] Minnen, D., Isbell, C.L., Essa, I., Starner, T.: Discovering multivariate motifs using subsequence density estimation and greedy mixture learning. In: Proceedings of the National Conference on Artificial Intelligence, vol. 1, pp. 615 – 620 (2007)

[184] Mishne, G., Dalton, J., Li, Z., Sharma, A., Lin, J.: Fast data in the era of big data: Twitter's real-time related query suggestion architecture. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 1147 – 1157 (2013). DOI 10.1145/2463676.2465290

[185] Misra, P., Siddharth, S.: Machine learning and time series: Real world applications. In: Proceeding - IEEE International Conference on Computing, Communication and Automation, ICCCA 2017, vol. 2017-January, pp. 389 – 394 (2017). DOI 10.1109/CCAA.2017.8229832

[186] Moghram, I., Rahman, S.: Analysis and evaluation of five short-term load forecasting techniques. IEEE Transactions on Power Systems **4**(4), 1484 – 1491 (1989). DOI 10.1109/59.41700

[187] Moghtaderi, A., Borgnat, P., Flandrin, P.: Trend filtering: Empirical mode decompositions versus l1 and hodrick-prescott. Advances in Adaptive Data Analysis **3**(1-2), 41 – 61 (2011). DOI 10.1142/S1793536911000751

[188] Molaei, S.M., Keyvanpour, M.R.: An analytical review for event prediction system on time series. In: 2015 2nd International Conference on Pattern Recognition and Image Analysis, IPRIA 2015 (2015). DOI 10.1109/PRIA.2015.7161635

[189] Morana, C.: Multivariate modelling of long memory processes with common components. Computational Statistics and Data Analysis **52**(2), 919 – 934 (2007). DOI 10.1016/j.csda.2006.12.010

[190] Mueen, A.: Enumeration of time series motifs of all lengths. In: Proceedings - IEEE International Conference on Data Mining, ICDM, pp. 547 – 556 (2013). DOI 10.1109/ICDM.2013.27

[191] Mueen, A.: Time series motif discovery: Dimensions and applications. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery **4**(2), 152 – 159 (2014). DOI 10.1002/widm.1119

[192] Mueen, A., Keogh, E., Zhu, Q., Cash, S., Westover, B.: Exact discovery of time series motifs. In: Society for Industrial and Applied Mathematics - 9th SIAM International Conference on Data Mining 2009, Proceedings in Applied Mathematics, vol. 1, pp. 469 – 480 (2009)

[193] Mueen, A., Keogh, E., Zhu, Q., Cash, S.S., Westover, M.B., Bigdely-Shamlo, N.: A disk-aware algorithm for time series motif discovery. Data Mining and Knowledge Discovery **22**(1-2), 73 – 105 (2011). DOI 10.1007/s10618-010-0176-8

[194] Munir, M., Chattha, M.A., Dengel, A., Ahmed, S.: A comparative analysis of traditional and deep learning-based anomaly detection methods for streaming data. In: Proceedings - 18th IEEE International Conference on Machine Learning and Applications, ICMLA 2019, pp. 561 – 566 (2019). DOI 10.1109/ICMLA.2019.00105

[195] Munir, M., Siddiqui, S.A., Dengel, A., Ahmed, S.: DeepAnT: A Deep Learning Approach for Unsupervised Anomaly Detection in Time Series. IEEE Access **7**, 1991 – 2005 (2019). DOI 10.1109/ACCESS.2018.2886457

[196] NASA: Global Warming vs. Climate Change. Tech. rep., https://climate.nasa.gov/global-warming-vs-climate-change/ (2023)

[197] Nelson, C.R., Plosser, C.R.: Trends and random walks in macroeconmic time series. Some evidence and implications. Journal of Monetary Economics **10**(2), 139 – 162 (1982). DOI 10.1016/0304-3932(82)90012-5

[198] no, A.C., naki Inza, I., Lozano, J.A.: Analyzing rare event, anomaly, novelty and outlier detection terms under the supervised classification framework. Artificial Intelligence Review **53**(5), 3575 – 3594 (2020). DOI 10.1007/s10462-019-09771-y

[199] NOAA: Climate at a Glance Global Time Series. Tech. rep., https://www.ncei.noaa.gov/access/monitoring/climate-at-a-glance/global/time-series (2023)

[200] Nunthanid, P., Niennattrakul, V., Ratanamahatana, C.A.: Discovery of variable length time series motif. In: ECTI-CON 2011 - 8th Electrical Engineering/ Electronics, Computer, Telecommunications and Information Technology (ECTI) Association of Thailand - Conference 2011, pp. 472 – 475 (2011). DOI 10.1109/ECTICON.2011.5947877

[201] Ogasawara, E., Martinez, L.C., De Oliveira, D., ao, G.Z., Pappa, G.L., Mattoso, M.: Adaptive Normalization: A novel data normalization approach for non-stationary time series. In: Proceedings of the International Joint Conference on Neural Networks (2010). DOI 10.1109/IJCNN.2010.5596746

[202] Ogasawara, E., Murta, L., ao, G.Z., Mattoso, M.: Neural networks cartridges for data mining on time series. In: Proceedings of the International Joint

Conference on Neural Networks, pp. 2302 – 2309 (2009). DOI 10.1109/IJCNN.2009.5178615

[203] Olteanu, M., Rossi, F., Yger, F.: Meta-survey on outlier and anomaly detection. Neurocomputing **555** (2023). DOI 10.1016/j.neucom.2023.126634

[204] Ozkan, H., Ozkan, F., Kozat, S.S.: Online Anomaly Detection Under Markov Statistics With Controllable Type-I Error. IEEE Transactions on Signal Processing **64**(6), 1435 – 1445 (2016). DOI 10.1109/TSP.2015.2504345

[205] Pang, G., Shen, C., Cao, L., Van Den Hengel, A.: Deep Learning for Anomaly Detection: A Review. ACM Computing Surveys **54**(2) (2021). DOI 10.1145/3439950

[206] Paparrizos, J., Boniol, P., Palpanas, T., Tsay, R.S., Elmore, A., Franklin, M.J.: Volume Under the Surface: A New Accuracy Evaluation Measure for Time-Series Anomaly Detection. Proceedings of the VLDB Endowment **15**(11), 2774 – 2787 (2022). DOI 10.14778/3551793.3551830

[207] Paparrizos, J., Kang, Y., Boniol, P., Tsay, R.S., Palpanas, T., Franklin, M.J.: TSB-UAD: An End-to-End Benchmark Suite for Univariate Time-Series Anomaly Detection. Proceedings of the VLDB Endowment **15**(8), 1697 – 1711 (2022). DOI 10.14778/3529337.3529354

[208] Park, C.H.: Outlier and anomaly pattern detection on data streams. Journal of Supercomputing **75**(9), 6118 – 6128 (2019). DOI 10.1007/s11227-018-2674-1

[209] Park, P., Di Marco, P., Shin, H., Bang, J.: Fault detection and diagnosis using combined autoencoder and long short-term memory network. Sensors (Switzerland) **19**(21) (2019). DOI 10.3390/s19214612

[210] Parra, L., Deco, G., Miesbach, S.: Statistical Independence and Novelty Detection with Information Preserving Nonlinear Maps. Neural Computation **8**(2), 260 – 269 (1996). DOI 10.1162/neco.1996.8.2.260

[211] Parzen, E.: On Estimation of a Probability Density Function and Mode. The Annals of Mathematical Statistics **33**(3), 1065–1076 (1962)

[212] Patel, P., Keogh, E., Lin, J., Lonardi, S.: Mining motifs in massive time series databases. In: Proceedings - IEEE International Conference on Data Mining, ICDM, pp. 370 – 377 (2002)

[213] Percival, D.B., Walden, A.T.: Wavelet Methods for Time Series Analysis. Cambridge University Press (2006)

[214] Perelman, L., Arad, J., Housh, M., Ostfeld, A.: Event detection in water distribution systems from multivariate water quality time series. Environmental Science and Technology **46**(15), 8212 – 8219 (2012). DOI 10.1021/es3014024

[215] Pesaranghader, A., Viktor, H.L.: Fast hoeffding drift detection method for evolving data streams. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) **9852 LNAI**, 96 – 111 (2016). DOI 10.1007/978-3-319-46227-1_7

[216] Pesaranghader, A., Viktor, H.L., Paquet, E.: McDiarmid Drift Detection Methods for Evolving Data Streams. In: Proceedings of the International Joint Conference on Neural Networks, vol. 2018-July (2018). DOI 10.1109/IJCNN.2018.8489260

[217] Pimentel, M.A., Clifton, D.A., Clifton, L., Tarassenko, L.: A review of novelty detection. Signal Processing **99**, 215 – 249 (2014). DOI 10.1016/j.sigpro.2013.12.026

[218] Ploberger, W., Krämer, W.: The Cusum Test with Ols Residuals. Econometrica **60**(2), 271–285 (1992). DOI 10.2307/2951597

[219] Polz, J., Chwala, C., Graf, M., Kunstmann, H.: Rain event detection in commercial microwave link attenuation data using convolutional neural networks. Atmospheric Measurement Techniques **13**(7), 3835 – 3853 (2020). DOI 10.5194/amt-13-3835-2020

[220] Prabuchandran, K., Singh, N., Dayama, P., Agarwal, A., Pandit, V.: Change point detection for compositional multivariate data. Applied Intelligence **52**(2), 1930 – 1955 (2022). DOI 10.1007/s10489-021-02321-6

[221] Pratama, M., Lu, J., Lughofer, E., Zhang, G., Er, M.J.: An Incremental Learning of Concept Drifts Using Evolving Type-2 Recurrent Fuzzy Neural Networks. IEEE Transactions on Fuzzy Systems **25**(5), 1175 – 1192 (2017). DOI 10.1109/TFUZZ.2016.2599855

[222] Provotar, O.I., Linder, Y.M., Veres, M.M.: Unsupervised Anomaly Detection in Time Series Using LSTM-Based Autoencoders. In: 2019 IEEE International Conference on Advanced Trends in Information Theory, ATIT 2019 - Proceedings, pp. 513 – 517 (2019). DOI 10.1109/ATIT49449.2019.9030505

[223] Pyle, D.: Data Preparation for Data Mining. Morgan Kaufmann (1999)

[224] Python, P.: Pandas Python package. Tech. rep., https://pandas.pydata.org/ (2023)

[225] Rakthanmanon, T., Campana, B., Mueen, A., Batista, G., Westover, B., Zhu, Q., Zakaria, J., Keogh, E.: Addressing big data time series: Mining trillions of time series subsequences under dynamic time warping. ACM Transactions on Knowledge Discovery from Data **7**(3) (2013). DOI 10.1145/2500489

[226] Rayana, S., Akoglu, L.: Less is more: Building selective anomaly ensembles. ACM Transactions on Knowledge Discovery from Data **10**(4) (2016). DOI 10.1145/2890508

[227] Ren, H., Xu, B., Wang, Y., Yi, C., Huang, C., Kou, X., Xing, T., Yang, M., Tong, J., Zhang, Q.: Time-series anomaly detection service at Microsoft. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 3009 – 3017 (2019). DOI 10.1145/3292500.3330680

[228] Ren, J., Xia, F., Lee, I., Noori Hoshyar, A., Aggarwal, C.: Graph Learning for Anomaly Analytics: Algorithms, Applications, and Challenges. ACM Transactions on Intelligent Systems and Technology **14**(2) (2023). DOI 10.1145/3570906

[229] Rettig, L., Khayati, M., Cudre-Mauroux, P., Piorkowski, M.: Online anomaly detection over Big Data streams. In: Proceedings - 2015 IEEE International Conference on Big Data, IEEE Big Data 2015, pp. 1113 – 1122 (2015). DOI 10.1109/BigData.2015.7363865

[230] Ross, G.J., Adams, N.M., Tasoulis, D.K., Hand, D.J.: Exponentially weighted moving average charts for detecting concept drift. Pattern Recognition Letters **33**(2), 191 – 198 (2012). DOI 10.1016/j.patrec.2011.08.019

[231] Sakurada, M., Yairi, T.: Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: ACM International Conference Proceeding Series, vol. 02-December-2014, pp. 4 – 11 (2014). DOI 10.1145/2689746.2689747

[232] Sakurai, G.Y., Lopes, J.F., ao, B.B.Z., Barbon Junior, S.: Benchmarking Change Detector Algorithms from Different Concept Drift Perspectives. Future Internet **15**(5) (2023). DOI 10.3390/fi15050169

[233] Salfner, F., Lenk, M., Malek, M.: A survey of online failure prediction methods. ACM Computing Surveys **42**(3) (2010). DOI 10.1145/1670679.1670680

[234] Salles, R., Assis, L., Guedes, G., Bezerra, E., Porto, F., Ogasawara, E.: A framework for benchmarking machine learning methods using linear models for univariate time series prediction. In: Proceedings of the International Joint Conference on Neural Networks, vol. 2017-May, pp. 2338 – 2345 (2017). DOI 10.1109/IJCNN.2017.7966139

[235] Salles, R., Belloze, K., Porto, F., Gonzalez, P.H., Ogasawara, E.: Nonstationary time series transformation methods: An experimental review. Knowledge-Based Systems **164**, 274 – 291 (2019). DOI 10.1016/j.knosys.2018.10.041

[236] Salles, R., Lima, J., Baroni, L., Castro, A., Carvalho, L., Borges, H., Carvalho, D., Coutinho, R., Bezerra, E., Pacitti, E., Porto, F., Ogasawara, E., for Technological Education of Rio de Janeiro, F.C.: harbinger: An Unified Time Series Event Detection Framework (2023). URL `https://cran.r-project.org/web/packages/harbinger/index.html`

[237] Salles, R., Lima, J., Coutinho, R., Pacitti, E., Masseglia, F., Akbarinia, R., Chen, C., Garibaldi, J., Porto, F., Ogasawara, E.: SoftED: Metrics for Soft Evaluation of Time Series Event Detection (2023). DOI 10.48550/arXiv.2304.00439. URL `http://arxiv.org/abs/2304.00439`

[238] Sayed-Mouchaweh, M., Lughofer, E.: Learning in Non-Stationary Environments: Methods and Applications. Springer Science & Business Media (2012)

[239] Schmidl, S., Wenig, P., Papenbrock, T.: Anomaly Detection in Time Series: A Comprehensive Evaluation. Proceedings of the VLDB Endowment **15**(9), 1779 – 1797 (2022). DOI 10.14778/3538598.3538602

[240] Sebestyen, G., Hangan, A., Czako, Z., Kovacs, G.: A taxonomy and platform for anomaly detection. In: 2018 IEEE International Conference on Automation, Quality and Testing, Robotics, AQTR 2018 - THETA 21st Edition, Proceedings, pp. 1 – 6 (2018). DOI 10.1109/AQTR.2018.8402710

[241] Seekell, D.A., Carpenter, S.R., Pace, M.L.: Conditional heteroscedasticity as a leading indicator of ecological regime shifts. American Naturalist **178**(4), 442 – 451 (2011). DOI 10.1086/661898

[242] Seth, T., Chaudhary, V.: A Predictive Analytics Framework for Insider Trading Events. In: Proceedings - 2020 IEEE International Conference on Big Data, Big Data 2020, pp. 218 – 225 (2020). DOI 10.1109/BigData50022.2020.9377791

[243] Shi, X., Gallagher, C., Lund, R., Killick, R.: A comparison of single and multiple changepoint techniques for time series data. Computational Statistics and Data Analysis **170** (2022). DOI 10.1016/j.csda.2022.107433

[244] Shumway, R.H., Stoffer, D.S.: Time Series Analysis and Its Applications: With R Examples. Springer (2017)

[245] Siffer, A., Fouque, P.A., Termier, A., Largouet, C.: Anomaly detection in streams with extreme value theory. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, vol. Part F129685, pp. 1067 – 1075 (2017). DOI 10.1145/3097983.3098144

[246] Singh, N., Olinsky, C.: Demystifying Numenta anomaly benchmark. In: Proceedings of the International Joint Conference on Neural Networks, vol. 2017-May, pp. 1570 – 1577 (2017). DOI 10.1109/IJCNN.2017.7966038

[247] Son, N.T., Anh, D.T.: Discovery of time series k-motifs based on multidimensional index. Knowledge and Information Systems **46**(1), 59 – 86 (2016). DOI 10.1007/s10115-014-0814-3

[248] Song, W., Wang, L., Xiang, Y., Zomaya, A.Y.: Geographic spatiotemporal big data correlation analysis via the Hilbert–Huang transformation. Journal of Computer and System Sciences **89**, 130 – 141 (2017). DOI 10.1016/j.jcss.2017.05.010

[249] Souto, G., Liebig, T.: On event detection from spatial time series for urban traffic applications. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) **9580**, 221 – 233 (2016). DOI 10.1007/978-3-319-41706-6_11

[250] Stefanakos, C., Schinas, O.: Forecasting bunker prices; a nonstationary, multivariate methodology. Transportation Research Part C: Emerging Technologies **38**, 177 – 194 (2014). DOI 10.1016/j.trc.2013.11.017

[251] Su, Y., Liu, R., Zhao, Y., Sun, W., Niu, C., Pei, D.: Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2828 – 2837 (2019). DOI 10.1145/3292500.3330672

[252] Sun, G., Chen, T., Wei, Z., Sun, Y., Zang, H., Chen, S.: A carbon price forecasting model based on variational mode decomposition and spiking neural networks. Energies **9**(1) (2016). DOI 10.3390/en9010054

[253] Taha, A., Hadi, A.S.: Anomaly detection methods for categorical data: A review. ACM Computing Surveys **52**(2) (2019). DOI 10.1145/3312739

[254] Takahashi, C.C., Braga, A.P.: A Review of Off-Line Mode Dataset Shifts. IEEE Computational Intelligence Magazine **15**(3), 16 – 27 (2020). DOI 10.1109/MCI.2020.2998231

[255] Takeuchi, J.I., Yamanishi, K.: A unifying framework for detecting outliers and change points from time series. IEEE Transactions on Knowledge and Data Engineering **18**(4), 482 – 492 (2006). DOI 10.1109/TKDE.2006.1599387

[256] Talagala, P.D., Hyndman, R.J., Smith-Miles, K., Kandanaarachchi, S., noz, M.A.M.: Anomaly Detection in Streaming Nonstationary Temporal Data.

Journal of Computational and Graphical Statistics **29**(1), 13 – 27 (2020). DOI 10.1080/10618600.2019.1617160

[257] Tan, S.C., Ting, K.M., Liu, T.F.: Fast anomaly detection for streaming data. In: IJCAI International Joint Conference on Artificial Intelligence, pp. 1511 – 1516 (2011). DOI 10.5591/978-1-57735-516-8/IJCAI11-254

[258] Tanaka, Y., Iwamoto, K., Uehara, K.: Discovery of time-series motif from multi-dimensional data based on MDL principle. Machine Learning **58**(2-3), 269 – 300 (2005). DOI 10.1007/s10994-005-5829-2

[259] Tang, H., Liao, S.S.: Discovering original motifs with different lengths from time series. Knowledge-Based Systems **21**(7), 666 – 671 (2008). DOI 10.1016/j.knosys.2008.03.022

[260] Tatbul, N., Lee, T.J., Zdonik, S., Alam, M., Gottschlich, J.: Precision and recall for time series. In: Advances in Neural Information Processing Systems, vol. 2018-December, pp. 1920 – 1930 (2018)

[261] Thudumu, S., Branch, P., Jin, J., Singh, J.: A comprehensive survey of anomaly detection techniques for high dimensional big data. Journal of Big Data **7**(1) (2020). DOI 10.1186/s40537-020-00320-x

[262] Tian, H., Khoa, N.L.D., Anaissi, A., Wang, Y., Chen, F.: Concept drift adaption for online anomaly detection in structural health monitoring. In: International Conference on Information and Knowledge Management, Proceedings, pp. 2813 – 2821 (2019). DOI 10.1145/3357384.3357816

[263] Tiao, G.: Asymptotic behaviour of temporal aggregates of time series. Biometrika **59**(3), 525 – 531 (1972). DOI 10.1093/biomet/59.3.525

[264] Torkamani, S., Lohweg, V.: Survey on time series motif discovery. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery **7**(2) (2017). DOI 10.1002/widm.1199

[265] Truong, C., Oudre, L., Vayatis, N.: Selective review of offline change point detection methods. Signal Processing **167** (2020). DOI 10.1016/j.sigpro.2019.107299

[266] ts R: Time-Series Objects. Tech. rep., https://stat.ethz.ch/R-manual/R-devel/library/stats/html/ts.html (2023)

[267] Tsay, R.S.: Analysis of Financial Time Series. Wiley (2010)

[268] Vahdatpour, A., Amini, N., Sarrafzadeh, M.: Toward unsupervised activity discovery using multi-dimensional motif detection in time series. In: IJCAI International Joint Conference on Artificial Intelligence, pp. 1261 – 1266 (2009)

[269] Vishwakarma, G.K., Paul, C., Elsawah, A.: An algorithm for outlier detection in a time series model using backpropagation neural network. Journal of King Saud University - Science **32**(8), 3328 – 3336 (2020). DOI 10.1016/j.jksus.2020.09.018

[270] Wang, C., Viswanathan, K., Choudur, L., Talwar, V., Satterfield, W., Schwan, K.: Statistical techniques for online anomaly detection in data centers. In: Proceedings of the 12th IFIP/IEEE International Symposium on Integrated Network Management, IM 2011, pp. 385 – 392 (2011). DOI 10.1109/INM.2011.5990537

[271] Wang, H., Bah, M.J., Hammad, M.: Progress in Outlier Detection Techniques: A Survey. IEEE Access **7**, 107,964 – 108,000 (2019). DOI 10.1109/ACCESS.2019.2932769

[272] Wang, H., Liu, L., Dong, S., Qian, Z., Wei, H.: A novel work zone short-term vehicle-type specific traffic speed prediction model through the hybrid EMD–ARIMA framework. Transportmetrica B **4**(3), 159 – 186 (2016). DOI 10.1080/21680566.2015.1060582

[273] Wang, L., Chng, E., Li, H.: A tree-construction search approach for multivariate time series motifs discovery. Pattern Recognition Letters **31**(9), 869 – 875 (2010). DOI 10.1016/j.patrec.2010.01.005

[274] Wang, P., Jin, N., Davies, D., Woo, W.L.: Model-centric transfer learning framework for concept drift detection. Knowledge-Based Systems **275** (2023). DOI 10.1016/j.knosys.2023.110705

[275] Wang, Y., Vuran, M.C., Goddard, S.: Analysis of event detection delay in wireless sensor networks. In: Proceedings - IEEE INFOCOM, pp. 1296 – 1304 (2011). DOI 10.1109/INFCOM.2011.5934912

[276] Wang, Z., Lin, X., Mishra, A., Sriharsha, R.: Online Changepoint Detection on a Budget. In: IEEE International Conference on Data Mining Workshops, ICDMW, vol. 2021-December, pp. 414 – 420 (2021). DOI 10.1109/ICDMW53433.2021.00057

[277] Ward, J.A., Lukowicz, P., Gellersen, H.W.: Performance metrics for activity recognition. ACM Transactions on Intelligent Systems and Technology **2**(1) (2011). DOI 10.1145/1889681.1889687

[278] Webb, G.I., Hyde, R., Cao, H., Nguyen, H.L., Petitjean, F.: Characterizing concept drift. Data Mining and Knowledge Discovery **30**(4), 964 – 994 (2016). DOI 10.1007/s10618-015-0448-4

[279] Wei, W.W.S.: Time Series Analysis Univariate and Multivariate Methods. Pearson Education (2018)

[280] Wenig, P., Schmidl, S., Papenbrock, T.: TimeEval: A Benchmarking Toolkit for Time Series Anomaly Detection Algorithms. Proceedings of the VLDB Endowment **15**(12), 3678 – 3681 (2022). DOI 10.14778/3554821.3554873

[281] Witten, I.H., Frank, E., Hall, M.A., Pal, C.J.: Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann (2016)

[282] Wu, R., Keogh, E.J.: Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress. IEEE Transactions on Knowledge and Data Engineering **35**(3), 2421 – 2429 (2023). DOI 10.1109/TKDE.2021.3112126

[283] Wu, Z., Huang, N.E., Long, S.R., Peng, C.K.: On the trend, detrending, and variability of nonlinear and nonstationary time series. Proceedings of the National Academy of Sciences of the United States of America **104**(38), 14,889 – 14,894 (2007). DOI 10.1073/pnas.0701020104

[284] Xiang, Q., Zi, L., Cong, X., Wang, Y.: Concept Drift Adaptation Methods under the Deep Learning Framework: A Literature Review. Applied Sciences (Switzerland) **13**(11) (2023). DOI 10.3390/app13116515

[285] Xuan, P.T., Anh, D.T.: An efficient hash-based method for time series motif discovery. In: Multi-disciplinary Trends in Artificial Intelligence, vol. 11248 LNAI, pp. 205 – 211 (2018). DOI 10.1007/978-3-030-03014-8_17

[286] Yadav, R., Pradhan, A.K., Kamwa, I.: Real-Time Multiple Event Detection and Classification in Power System Using Signal Energy Transformations. IEEE Transactions on Industrial Informatics **15**(3), 1521 – 1531 (2019). DOI 10.1109/TII.2018.2855428

[287] Yang, J., Wang, W., Yu, P.S.: Mining surprising periodic patterns. Data Mining and Knowledge Discovery **9**(2), 189 – 216 (2004). DOI 10.1023/B:DAMI.0000031631.84034.af

[288] Yang, W., Zurbenko, I.: Nonstationarity. Wiley Interdisciplinary Reviews: Computational Statistics **2**(1), 107 – 115 (2010). DOI 10.1002/wics.64

[289] Yeh, C.C.M., Kavantzas, N., Keogh, E.: Matrix profile VI: Meaningful multidimensional motif discovery. In: Proceedings - IEEE International Conference on Data Mining, ICDM, vol. 2017-November, pp. 565 – 574 (2017). DOI 10.1109/ICDM.2017.66

[290] Yeh, C.C.M., Zhu, Y., Ulanova, L., Begum, N., Ding, Y., Dau, H.A., Silva, D.F., Mueen, A., Keogh, E.: Matrix profile I: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets. In: Proceedings - IEEE International Conference on Data Mining, ICDM, pp. 1317 – 1322 (2017). DOI 10.1109/ICDM.2016.89

[291] Yeh, C.C.M., Zhu, Y., Ulanova, L., Begum, N., Ding, Y., Dau, H.A., Zimmerman, Z., Silva, D.F., Mueen, A., Keogh, E.: Time series joins, motifs, discords and shapelets: a unifying view that exploits the matrix profile. Data Mining and Knowledge Discovery **32**(1), 83 – 123 (2018). DOI 10.1007/s10618-017-0519-9

[292] Yeo, I.K., Johnson, R.A.: A new family of power transformations to improve normality or symmetry. Biometrika **87**(4), 954 – 959 (2000). DOI 10.1093/biomet/87.4.954

[293] You, X., Zhang, M., Ding, D., Feng, F., Huang, Y.: Learning to Learn the Future: Modeling Concept Drifts in Time Series Prediction. In: International Conference on Information and Knowledge Management, Proceedings, pp. 2434 – 2443 (2021). DOI 10.1145/3459637.3482271

[294] Yu, Q., Luo, Y., Chen, C., Wang, X.: Trajectory outlier detection approach based on common slices sub-sequence. Applied Intelligence **48**(9), 2661 – 2680 (2018). DOI 10.1007/s10489-017-1104-z

[295] Yuan, L., Li, H., Xia, B., Gao, C., Liu, M., Yuan, W., You, X.: Recent Advances in Concept Drift Adaptation Methods for Deep Learning. In: IJCAI International Joint Conference on Artificial Intelligence, pp. 5654 – 5661 (2022)

[296] Zeileis, A.: A unified approach to structural change tests based on ML scores, F statistics, and OLS residuals. Econometric Reviews **24**(4), 445 – 466 (2005). DOI 10.1080/07474930500406053

[297] Zeileis, A., Kleiber, C., Walter, K., Hornik, K.: Testing and dating of structural changes in practice. Computational Statistics and Data Analysis **44**(1-2), 109 – 123 (2003). DOI 10.1016/S0167-9473(03)00030-6

[298] Zeileis, A., Leisch, F., Hornik, K., Kleiber, C.: Strucchange: An R package for testing for structural change in linear regression models. Journal of Statistical Software **7**, 1 – 38 (2002). DOI 10.18637/jss.v007.i02

[299] Zhang, C., Song, D., Chen, Y., Feng, X., Lumezanu, C., Cheng, W., Ni, J., Zong, B., Chen, H., Chawla, N.V.: A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data. In: 33rd AAAI Conference on Artificial Intelligence, AAAI 2019, pp. 1409 – 1416 (2019)

[300] Zhang, J., Wu, C., Wang, Y.: Human fall detection based on body posture spatio-temporal evolution. Sensors (Switzerland) **20**(3) (2020). DOI 10.3390/s20030946

[301] Zhang, Y., Meratnia, N., Havinga, P.: Outlier detection techniques for wireless sensor networks: A survey. IEEE Communications Surveys and Tutorials **12**(2), 159 – 170 (2010). DOI 10.1109/SURV.2010.021510.00088

[302] Zhao, L.: Event Prediction in the Big Data Era: A Systematic Survey. ACM Computing Surveys **54**(5) (2021). DOI 10.1145/3450287

[303] Zhao, L., Wang, L.: Price Trend Prediction of Stock Market Using Outlier Data Mining Algorithm. In: Proceedings - 2015 IEEE 5th International Conference on Big Data and Cloud Computing, BDCloud 2015, pp. 93 – 98 (2015). DOI 10.1109/BDCloud.2015.19

[304] Zhao, Q., Zhang, Y., Shi, Y., Li, J.: Analyzing and Visualizing Anomalies and Events in Time Series of Network Traffic. Advances in Intelligent Systems and Computing **936**, 15 – 25 (2020). DOI 10.1007/978-3-030-19861-9_2

[305] Zhou, X., Angelov, P.: Autonomous visual self-localization in completely unknown environment using evolving fuzzy rule-based classifier. In: Proceedings of the 2007 IEEE Symposium on Computational Intelligence in Security and Defense Applications, CISDA 2007, pp. 131 – 138 (2007). DOI 10.1109/CISDA.2007.368145

[306] Zhou, Y., Arghandeh, R., Zou, H., Spanos, C.J.: Nonparametric Event Detection in Multiple Time Series for Power Distribution Networks. IEEE Transactions on Industrial Electronics **66**(2), 1619 – 1628 (2019). DOI 10.1109/TIE.2018.2840508

[307] Zhou, Y., Ren, H., Li, Z., Wu, N., Al-Ahmari, A.M.: Anomaly detection via a combination model in time series data. Applied Intelligence **51**(7), 4874 – 4887 (2021). DOI 10.1007/s10489-020-02041-3

[308] Zhu, Y., Yeh, C.C.M., Zimmerman, Z., Kamgar, K., Keogh, E.: Matrix Profile XI: SCRIMP++: Time Series Motif Discovery at Interactive Speeds. In: Proceedings - IEEE International Conference on Data Mining, ICDM, vol. 2018-November, pp. 837 – 846 (2018). DOI 10.1109/ICDM.2018.00099

[309] Zipfel, J., Verworner, F., Fischer, M., Wieland, U., Kraus, M., Zschech, P.: Anomaly detection for industrial quality assurance: A comparative evaluation

of unsupervised deep learning models. Computers and Industrial Engineering **177** (2023). DOI 10.1016/j.cie.2023.109045

[310] Zou, B., Yang, K., Kui, X., Liu, J., Liao, S., Zhao, W.: Anomaly detection for streaming data based on grid-clustering and Gaussian distribution. Information Sciences **638** (2023). DOI 10.1016/j.ins.2023.118989

Index

ACF, 22, 23
ADFT, 22, 23
ADWIN, 88
AIC, 31, 63
AMOC, 75–77, 79, 132
AN, 38
AR, 44, 46, 81
AR(1), 19, 22
ARCH, 6
ARIMA, 6, 10, 14, 33, 44, 45, 47, 48, 53, 57, 58, 63, 81, 82, 132
ARMA, 44, 46
AUC, 109, 110

BCT, 27, 28
BIC, 31
BPT, 22, 23

CF, 75, 81, 82, 132
Chow, 75, 78, 79, 132
Conv1D, 12, 47, 48, 57, 60, 132
CUSUM, 79, 89
CWT, 33

DBScan, 61, 69, 132
DDM, 86, 87
DFT, 31, 33, 104
DIF, 28–30, 44
DT, 28–30
DTree, 60, 132
DTW, 61, 96–98, 104, 132
DWT, 33, 34

ECDD, 88
EDist, 96, 97, 101
EE, 70
ELM, 12, 47, 57, 132
EMD, 33–36, 57
EMMA, 99
ETS, 44
EV, 70
EWMA, 88

FBIAD, 52, 53, 132
FFT, 31, 32, 57, 102
FHDDM, 87
FT, 31

GARCH, 6, 44, 46, 68, 132
GEV, 71
GFT, 75, 79, 80, 132
GRU, 65
GST, 1

HAAR, 33, 34
HDDM, 87
HDist, 96, 97, 100
HHT, 36
HItem, 97
HPF, 33
HW, 44, 46

IID, 46, 70
IMF, 34–36

K-means, 12, 61, 62, 69, 132
KNN, 69, 132
KNN-CAD, 12
KZFT, 32

LMM, 44, 45
LogReg, 60
LPF, 33
LR, 26, 27, 29, 57, 63

LSTM, 12, 14, 47–49, 57, 60, 132
LT, 27, 28

MA, 29, 44
MAS, 27, 29, 57
MBIC, 77
MDDM, 87
MDL, 104
MEP, 96
MGT, 10, 11, 26, 95, 96
Min-Max, 24, 37, 38
MLE, 76
MLM, 36, 40, 46–48
MLP, 47, 57, 132
MP, 101–103, 105, 132
MRA, 33
MSE, 43, 63

NAB, 115, 116
NB, 60, 132
NNET, 12, 47, 48, 60
NOAA, 1, 2

OLS, 78, 79

PAA, 95, 96, 99, 100, 104, 105
PACF, 22
Page-Hinkley, 89, 90, 132
PCA, 53, 54, 65, 69, 91, 92, 103
PCT, 29, 30
PDF, 63, 64, 76
PELT, 75–77, 132
PP, 40, 43, 48
PPT, 22, 23

R2, 43, 63
RBF, 47, 57
RF, 60, 132
RFR, 47, 48, 57, 132
ROC, 109
RSS, 75, 78

SARIMA, 44
SAX, 9, 69, 95, 96, 99, 100, 104, 105, 132
SCP, 75, 80, 81, 132
SM, 44
sMAPE, 43, 63
SoftED, 112–115, 132
SOM, 62
STAMP, 102
SVM, 12, 47, 48, 57, 60, 91, 132
SW, 24, 36–40, 42, 46, 56, 57, 59–61, 80–82, 99, 100

TC, 4, 5, 7
TF, 44
THW, 44

VAE, 65
VMD, 33, 35

WFT, 31–33
WPT, 33
WT, 33, 57

YGT, 2, 6, 8–11, 18, 19, 22, 23, 25–36, 38, 40, 45, 48, 49, 102, 103

Z-Score, 24, 37, 95, 99, 100, 106